

OBJECT ORIENTED PROGRAMMING THROUGH JAVA [23CS0508]

LECTURE NOTES

B.TECH II YEAR – I SEM (R23)



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

SIDDHARTH INSTITUTE OF ENGINEERING & TECHNOLOGY

(AUTONOMOUS)

(Approved by AICTE, New Delhi & Affiliated to JNTUA, Ananthapuramu)

(Accredited by NBA for Civil, EEE, Mech., ECE & CSE Accredited by NAAC with 'A' Grade)

Puttur -517583, Chittoor District, A.P. (India)

UNIT-1

I. Program Structure in Java:

1. Basic concepts
2. Principles
3. Program Structure in Java: Introduction
4. Writing Simple Java Programs
5. Elements or Tokens in Java Programs
6. Java Statements
7. Command Line Arguments
8. User Input to Programs
9. Escape Sequences, Comments
10. Programming Style.

II. Data Types Variables and Operators:

1. Introduction Data Types in Java
2. Declaration of Variables Data Types
3. Type Casting
4. Scope of Variable Identifier
5. Literal Constants, Symbolic Constants
6. Formatted Output with printf() Method
7. Static Variables and Methods
8. Attribute Final
9. Introduction to Operators
10. Precedence and Associativity of Operators
11. Assignment Operator (=)
12. Basic Arithmetic Operators
13. Increment (++) and Decrement (- -) Operators
14. Ternary Operator Relational Operators
15. Boolean Logical Operators
16. Bitwise Logical Operators.

III. Control Statements:

1. Introduction
2. if Expression
3. Nested if Expressions
4. if-else Expressions
5. Ternary Operator?:
6. Switch Statement
7. Iteration Statements
8. while Expression
9. do-while Loop 10. for Loop
11. Nested for Loop
12. For-Each for Loop
13. Break Statement
14. Continue Statement.

1. Introduction

Java is one of the most popular programming languages used to create Web applications and platforms. It was designed for flexibility, allowing developers to write code that would run on any machine, regardless of architecture or platform. According to the Java home page, more than 3 billion computers and 4 billion mobile phones worldwide run Java.

Java is an Object Oriented and high-level programming language, originally developed by Sun Microsystems and released in 1995. Java runs on a variety of platforms, such as Windows, Mac OS, Linux etc.

In Java, there are three types of programs as follows:

Stand-alone application programs : These programs are made and run on users computers

Applet programs : These programs are meant to run in a web page on the Internet.

Java Servlets : These programs run in computers that provide web services. They are also often called server side programs or servlets.

Objects pass messages to each other. Each object decides what to do with a received message. OOP focuses on each object's states and behaviors.

What Are Objects?

An object is an entity that has states and behaviors.

For example, dog, cat, and vehicle. To illustrate, a dog has states like age, color, name, and behaviors like eating, sleeping, and running.

- ✓ State tells us how the object looks or what properties it has.
- ✓ Behavior tells us what the object does.
- ✓ We can actually represent a real world dog in a program as a software object by defining its states and behaviors.
- ✓ Software objects are the actual representation of real world objects. Memory is allocated in RAM whenever creating a logical object.
- ✓ An object is also referred to an instance of a class. Instantiating a class means the same thing as creating an object.
- ✓ The important thing to remember when creating an object is: the reference type should be the **same type** or a **super type** of the object type. We'll see what a reference type is later in this article.

What Are Classes?

A class is a template or blueprint from which objects are created.

Imagine a class as a cookie-cutter and objects as cookies.



Figure 1: Illustrates class and object relationship through cookie-cutter and cookies.

Classes define states as instance variables and behaviors as instance methods. Instance variables are also known as member variables.

Classes don't consume any space.

Principles of object-oriented programming

These are the four main principles of the object-oriented programming paradigm. Understanding them is essential to becoming a successful programmer.

1. Encapsulation
2. Inheritance
3. Abstraction
4. Polymorphism

Now let's look at each in more detail.

Abstraction is the concept of object-oriented programming that "shows" only essential attributes and "hides" unnecessary information. The main purpose of abstraction is hiding the unnecessary details from the users. Abstraction is selecting data from a larger pool to show only relevant details of the object to the user. It helps in reducing programming complexity and efforts. It is one of the most important concepts of OOPs.

For example, when you login to your Amazon account online, you enter your user_id and password and press login, what happens when you press login, how the input data sent to amazon server, how it gets verified is all abstracted away from the you.

Another example of abstraction: A car in itself is a well-defined object, which is composed of several other smaller objects like a gearing system, steering mechanism, engine, which are again have their own subsystems. But for humans car is a one single object, which can be managed by the help of its subsystems, even if their inner details are unknown.



ENCAPSULATION

The process of binding data and corresponding methods (behavior) together into a single unit is called **encapsulation in Java**.

In other words, encapsulation is a programming technique that binds the class members (variables and methods) together and prevents them from being accessed by other classes, thereby we can keep variables and methods safe from outside interference and misuse.

Every Java class is an example of encapsulation because we write everything within the class only that binds variables and methods together and hides their complexity from other classes.

Another example of encapsulation is a capsule. Basically, capsule encapsulates several combinations of medicine.

If combinations of medicine are variables and methods then the capsule will act as a class and the whole process is called Encapsulation as shown in the below figure.

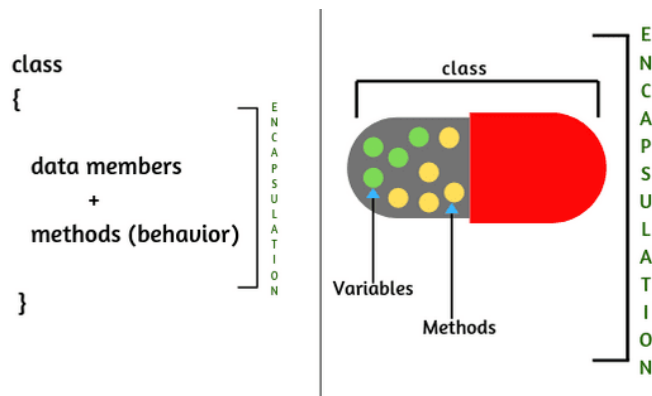


Fig: Encapsulation

In the encapsulation technique, we declare fields as private in the class to prevent other classes from accessing them directly. The required encapsulated data can be accessed by using public Java getter and setter method.

If the field is declared private in the class then it cannot be accessed by anyone from outside the class and hides the field within the class. Therefore, it is also called **data hiding**

Example 1:

School bag is one of the most real examples of Encapsulation. School bag can keep our books, pens, etc.

Realtime Example 2:

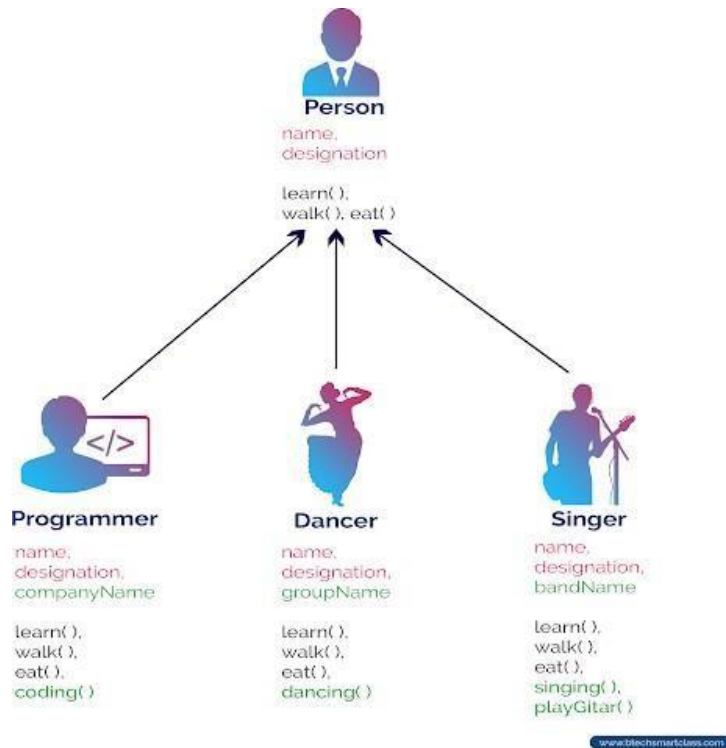
When you log into your email accounts such as Gmail, Yahoo Mail, or Rediff mail, there is a lot of internal processes taking place in the backend and you have no control over it.

When you enter the password for logging, they are retrieved in an encrypted form and verified, and then you are given access to your account.

You do not have control over it that how the password has been verified. Thus, it keeps our account safe from being misused.

INHERITANCE

- Inheritance is the mechanism by which an object acquires the some/all properties of another object.
- It supports the concept of hierarchical classification.
For example: Car is a four wheeler vehicle so assume that we have a class FourWheeler and a sub class of it named Car. Here Car acquires the properties of a class FourWheeler. Other classifications could be a jeep, tempo, van etc. FourWheeler defines a class of vehicles that have four wheels, and specific range of engine power, load carrying capacity etc. Car (termed as a sub-class) acquires these properties from FourWheeler, and has some specific properties, which are different from other classifications of FourWheeler, such as luxury, comfort, shape, size, usage etc. A car can have further classification such as an open car, small car, big car etc, which will acquire the properties from both Four Wheeler and Car, but will still have some specific properties. This way the level of hierarchy can be extended to any level.



POLYMORPHISM

Polymorphism in Java is a concept by which we can perform a *single action in different ways*. Polymorphism is derived from 2 Greek words: poly and morphs. The word "poly" means many and "morphs" means forms. So polymorphism means many forms.

There are two types of polymorphism in Java: compile-time polymorphism and runtime polymorphism. We can perform polymorphism in java by method overloading and method overriding.

If you overload a static method in Java, it is the example of compile time polymorphism. Here, we will focus on runtime polymorphism in java.

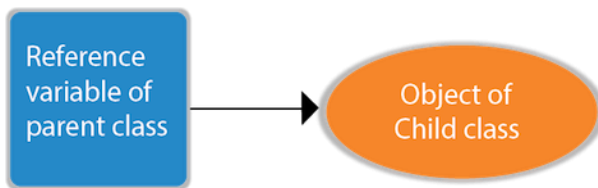
Runtime Polymorphism in Java

Runtime polymorphism or **Dynamic Method Dispatch** is a process in which a call to an overridden method is resolved at runtime rather than compile-time.

In this process, an overridden method is called through the reference variable of a superclass. The determination of the method to be called is based on the object being referred to by the reference variable.

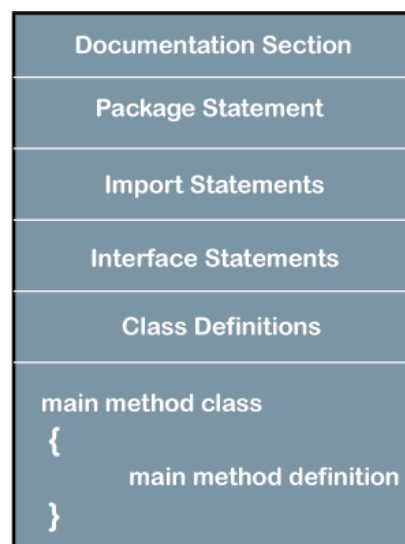
Let's first understand the upcasting before Runtime Polymorphism.

If the reference variable of Parent class refers to the object of Child class, it is known as upcasting. For example:



Program Structure in Java

Java is an [object-oriented programming](#), **platform-independent**, and **secure** programming language that makes it popular. Using the Java programming language, we can develop a wide variety of applications. So, before diving in depth, it is necessary to understand the **basic structure of Java program** in detail.



Structure of Java Program

Documentation Section

The documentation section is an important section but optional for a Java program. It includes **basic information** about a Java program. The information includes the **author's name**, **date of creation**, **version**, **program name**, **company name**, and **description** of the program. It improves the readability of the program. Whatever we write in the documentation section, the Java compiler ignores the statements during the execution of the program. To write the statements in the documentation section, we use **comments**. The comments may be **single-line**, **multi-line**, and **documentation** comments.

- **Single-line Comment:** It starts with a pair of forwarding slash (//).
For example: // HAI SAMBA SIVA

Multi-line Comment: It starts with a /* and ends with */. We write between these two symbols.
For example:

```
/* HAI SAMBA SIVA  
HOW ARE YOU */
```

Package Declaration

The package declaration is optional. It is placed just after the documentation section. In this section, we declare the **package name** in which the class is placed. Note that there can be **only one package** statement in a Java program. It must be defined before any class and interface declaration. It is necessary because a Java class can be placed in different packages and directories based on the module

they are used. For all these classes package belongs to a single parent directory. We use the keyword **package** to declare the package name. For example:

- **package javatpoint;** //where javatpoint is the package name
- **package com.javatpoint;** //where com is the root directory and javatpoint is the subdirectory

Import Statements

The package contains the many predefined classes and interfaces. If we want to use any class of a particular package, we need to import that class. The import statement represents the class stored in the other package. We use the **import** keyword to import the class. It is written before the class declaration and after the package statement. We use the import statement in two ways, either import a specific class or import all classes of a particular package. In a Java program, we can use multiple import statements. For example:

1. **import java.util.Scanner;** //it imports the Scanner class only
2. **import java.util.*;** //it imports all the class of the java.util package

Interface Section

It is an optional section. We can create an **interface** in this section if required. We use the **interface** keyword to create an interface. An [interface](#) is a slightly different from the class. It contains only **constants** and **method** declarations. Another difference is that it cannot be instantiated. We can use interface in classes by using the **implements** keyword. An interface can also be used with other interfaces by using the **extends** keyword.

Class Definition

In this section, we define the class. It is **vital** part of a Java program. Without the [class](#), we cannot create any Java program. A Java program may contain more than one class definition. We use the **class** keyword to define the class. The class is a blueprint of a Java program. It contains information about user-defined methods, variables, and constants. Every Java program has at least one class that contains the main() method. For example:

```
class Student //class definition
{

}
```

Class Variables and Constants

In this section, we define [variables](#) and **constants** that are to be used later in the program. In a Java program, the variables and constants are defined just after the class definition. The variables and constants store values of the parameters. It is used during the execution of the program. We can also decide and define the scope of variables by using the modifiers. It defines the life of the variables. For example:

```
class Student //class definition
{
    String sname; //variable
    int id;
    double percentage;
}
```

Main Method Class

In this section, we define the **main() method**. It is essential for all Java programs. Because the execution of all Java programs starts from the main() method. In other words, it is an entry point of the class. It must

be inside the class. Inside the main method, we create objects and call the methods. We use the following statement to define the main() method:

```
public class Student //class definition
{
    public static void main(String args[])
    {
        //statements
    }
}
```

2. Writing Simple Java Programs

Java is an Object oriented language. Every java program imports packages which are provides necessary classes and interfaces.

For example:

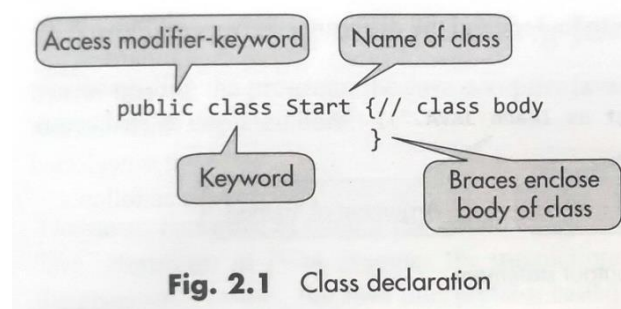
```
import java.util.*;
import java.io.*;
```

After import statement, every java program starts with the declaration of the class. A program may have one or more classes.

A class declaration starts with the keyword class, followed by the identifier or name of the class. Giving the name of a package at the top is optional.

Class declaration contains name of the class and body of the class. The body of the class may consist of several statements and is enclsed between the braces {}.

A sample of class declarations is as follows.



Here:

public is access specifier. This class is accessible to any outside code. Otherwise the class is accessible to only same package classes.

class is a keyword of java language which is used to declare the class.

The class body starts with the left brace { and ends with the right closing brace }.

// are comments which are neglected by the compiler.

A class body may comprise statements for declaration of variables, constants, expressions, and methods.

Simple program in Java. Save this code in Text Document as “**Start.java**”

```
public class Start()
{
```

```

public static void main()
{
    System.out.println("Hello World");
}
}

```

Here:

class is a keyword

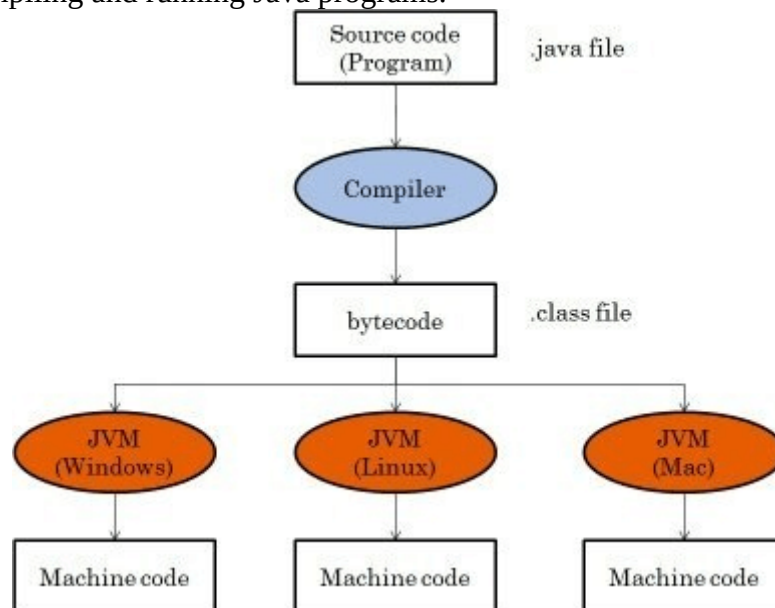
Start is a name the class. Here class is declared as public, so it is available to all the classes.

main() is the method which initiate and terminate the program (in java functions are called as Methods)

println() is method of object **"out"**. **"out"** is the object of class **"System"**.
println() prints the string "Hello World" on the screen/monitor.

Compiling and Running Java program

Java compiler first converts the source code into an intermediate code, known as bytecode or virtual machine code. To run the bytecode, we need the Java Virtual Machie (JVM). JVM exists only inside the computer memory and runs on top of the Operating System. The bytecode is not machine specific. The Java interpreter converts the bytecode into Machine code. The following diagram illustrates the process of compiling and running Java programs.



For compiling the program, the Java compiler javac is run, specifying the name of the source file on the command line as depicted here:

```
javac Start.java
```

The Java compiler creates a file called Start.class containing the bytecode version of the program. The java interpreter in JVM executes the instructions contained in

this intermediate Java bytecode version of the program. The Java interpreter is called with “java” at the command prompt.

```
C:\> java Start
```

Output:

```
Hello World
```

Here java command calls the Java interpreter which executes the Start.class (bytecode of Start.java).

3. Elements or Tokens in Java Programs

Java program contains different types of elements like white spaces, comments and tokens. A token is the smallest program element which is recognized by the compiler and which treats them as defined for the compiler. A program is a set of tokens which comprise the following elements:

Identifiers or names: Identifier is the name of variables, methods, classes etc.

Rules for framing Names or Identifiers.

- It should be a single word which contains alphabets a to z or A to Z, digits 0 to 9, underscore (_).
- It should not contain white spaces and special symbols.
- It should not be a keyword of Java.
- It should not be Boolean literal, that is, true or false.
- It should not be null literal.
- It should not start with a digit but it can start with an underscore.
- It can comprise one or more unicode characters which are characters as well as digits.

Conventions for Writing Names

- Names of packages are completely in lower-case letters such as mypackage, java.lang.
- Names of classes and interfaces start with an upper-case letter.
- Names of methods start with a lower-case character.
- Names of variables should start with a lower-case character.

Keywords: These are special words defined in Java and represent a set of instructions.

- The keywords represent groups of instructions for the compiler.
- These are special tokens that have a predefined meaning and their use is restricted.
- keywords cannot be used as names of variables, methods, classes, or packages.
- These are written in the lower case.
- Keywords of Java Language are as follows:

abstract	continue	for	new	switch
assert	default	goto	package	synchronized
boolean	do	if	private	this
break	double	implements	protected	throw
byte	else	import	public	throws
case	enum	instanceof	return	transient
catch	extends	int	short	try
char	final	interface	static	void
class	finally	long	strictfp	volatile
const	float	native	super	while

Literals: These are values represented by a set of character, digits, etc.

- A literal represents a value which may be of primitive type, String type, or null type.
- The value may be a number (either whole or decimal point number) or a sequence of characters which is called String literal, Boolean type, etc.
- A literal is a constant value.

Types of Literals:

i. Integer literals

- Sequences of digits.
- The whole numbers are described by different number systems such as decimal numbers, hexadecimal numbers, octal numbers, and binary numbers.
- Each number has a different set of digits.

Decimal Integer Literals

- These are sequences of decimal digits which are 0, 1, 2, 3, 4, 5, 6, 7, 8, and 9.
- Examples of such literals are 6, 453, 34789, etc.

Hex Integral Literals

- These are sequences of hexadecimal digits which are 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, and F.
- The values 10 to 15 are represented by A, B, C, D, E, and F or a, b, c, d, e, and f.
- The numbers are preceded by 0x or 0X. Examples are 0x56ab 0X6AF2, etc.

Octal Integer Literals

- These are sequences of octal digits which are 0, 1, 2, 3, 4, 5, 6, and 7.
- These numbers are preceded by 0. Examples of literals are 07122, 04, 043526.

Binary Literal

- These are sequences of binary digits.
- Binary numbers have only two digits—0 and 1 and a base 2.
- Examples of such literals are 0b0111001, 0b101, 0b1000, etc.

ii. Floating point literal

- These are floating decimal point numbers or fractional decimal numbers with base 10. Examples are 3.14159, 567.78, etc.

iii. Boolean literal

- These are Boolean values. There are only two values—true or false.

iv. Character literal

- These are the values in characters.
- Characters are represented in single quotes such as 'A', 'H', 'k', and so on.

v. String literal

- These are strings of characters in double quotes. Examples are "Delhi", "John", "AA", etc.

vi. Null literal

- There is only one value of Null Literal, that is, null.

Separators: These include comma, semicolon, period(.), Parenthesis (), Square brackets [], etc

<i>Symbol</i>	<i>Name</i>	<i>Description</i>
()	Parentheses	Parentheses are used for the following purposes: 1. To change precedence level in expressions 2. To contain a list of parameters in a definition of methods 3. To contain expression in control statements 4. To enclose cast types in explicit type casting
{ }	Braces	Braces are used for the following purposes: 1. To enclose definitions of classes and methods 2. To enclose blocks of statements 3. To initialize arrays and string objects 4. To enclose local scopes
[]	Square brackets	Square brackets are used for enclosing index values in array elements and for defining arrays
,	Comma	Commas are used for separating the following: 1. Variables in parameter or declaration lists 2. Identifiers in variable declarations 3. Expressions in for loop statement
;	Semicolon	A semicolon is used for terminating statements in a program.
.	Period	The period is used for the following: 1. To link an object of class with its method 2. To link classes with sub-packages and sub-packages to packages

Operators: Operators are mostly represented by symbols such as +, -, *, etc

Types of Operators:

Arithmetic Operators:

Operator	Description
+	Addition or Unary plus
-	Subtraction or Unary minus
*	Multiplication
/	Division
%	Modulus

Relational Operators:

Operator	Description
>	Greater than
>=	Greater than or equal to
<	Less than
<=	Less than or equal to
==	Equal to
!=	Not equal to

Logical Operators:

Operator	Description
&&	Greater than
	Greater than or equal to
!	Less than

Assignment Operators:

Operator	Description
+=	Add and assign to
-=	Subtract and assign to
*=	Multiply and assign to
/=	Divide and assign to
%=	Modulus and assign to

Increment / Decrement Operators:

Operator	Description
++	Increment by one
--	Decrement by one

Bitwise Operators:

Operator	Description
&	Bitwise AND
	Bitwise OR
^	Bitwise exclusive OR
~	Bitwise compliment
>>	Shift Right
<<	Shift Left
>>>	Shift right with Zero fill

Conditional Operators:

Operator	Description
?:	Used to construct Conditional expression

1. Java Statements

A Statement is a instruction to the computer. A program is a set of statements or instructions. The statements specify the sequence of actions to be performed when some method or constructor is invoked. The statements are executed in the sequence in the specified order. The important Java statements are as follows.

Statement	Description
Empty statement	These are used during program development.
Variable declaration Statement	It defines a variable that can be used to store the values.
Labeled statement	A block of statements is given a label. The labels should not be keywords, previously used labels, or already declared local variables.
Expression statement	Most of the statements come under this category. There are seven types of expression statements that include assignment, method call and allocation, pre-increment, post increment, pre-decrement, and post decrement statements.
Control statement	This comprises selection, iteration, and jump statements.
Selection statement	In these statements, one of the various control flows is selected when a certain condition expression is true. There are three types of selection statements including if, if-else, and switch.
Iteration statement	These involve the use of loops until some condition for the termination of loop is satisfied. There are three types of iteration statements that make use of while, do, and for
Jump statement	In these statements, the control is transferred to the beginning or end of the current block or to a labeled statement. There are four types of Jump statements including break, continue, return, and throw.
Synchronization Statement	These are used with multi-threading
Guarding statement	These are used to carry out the code safely that may cause exceptions (such as division by zero, and so on). These statements make use of try and catch block, and finally

2. Command Line Arguments

- A Java application can accept any number of arguments from the command line.
- These arguments can be passed at the time of running the program. This enables a programmer to check the behavior of a program for different values of inputs.
- When a Java application is invoked, the runtime system passes the command line arguments to the application's main method through an array of strings.
- It must be noted that the number of arguments
-
- in an array. To ensure this, we can make use of the length property of the array. This is illustrated in

Program 2.2: Example showing command line argument

```
class vehicle
{
    public static void main(String args[])
    {
        int x = args.length; for(int
        i=0; i<x; i++)
        {
            System.out.println(args[i]);
        }
    }
}
```

Output

(After compiling, type the following lines on the command prompt. It produces the output as)

```
C:\> Java vehicle "Car Cycle Motorbike" Car
Cycle Motorbike
```

Program 2.3: Illustration of command line

```
class Sum
{
    public static void main{ String args) int s=1;

    for(int i e; icargs length; 1++){ s=s+Integer.parseInt(args(1));

    System.out.prinln(" The addition of passed arguments 15" );

}
```

Output

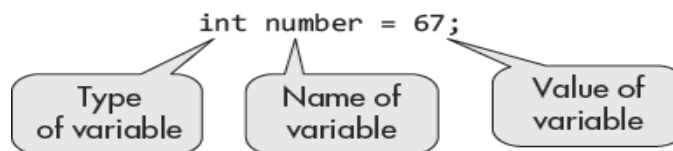
After compiling, the following lines are typed on the command prompt: It produces the

output as

C:\> Java Sun 20 15 17

Declaration of Variables

- A program may involve variables: variables are objects whose values may change in the program.
- A variable is declared by first writing its type, followed by its name or identifier as illustrated here.



- However, a variable should also be initialized, that is, a value should be assigned to it before it is used in an expression. The line ends with a semicolon (;) as shown in the above figure.

Examples:

```
double price = 28.5; char ch  
= 'C'; String name = "John";
```

Program 2.4 illustrates the declaration and output of some data types

```
class PrintOut  
{  
    public static void main (String args[])  
    {  
        String name = "Sunita"; //"name" is variable, value is "Sunita"  
        String str = "Hello"; //String set has value- "Hello!"  
  
        int length = 50; // Variable name "length", value 50  
        int width = 8; // Variable name "width" value 8  
  
        System.out.print("Name= " + name); System.out.print("Str = "+ str);  
        //print statement  
  
        System.out.println();  
  
        System.out.println("Length "+ length); System.  
        out.println("Width " + width);
```

```

        System.out.println(str + name); // println statement
    }
}

```

```
c:\>javac PrintOut.java
```

```

c:\>java PrintOut
Name= SunitaStr = Hello Length
50
Width 8 HelloSunita

```

4. User Input to Programs

- The class Scanner of package java.util to carry out input to the program.
- Java Scanner class is a text scanner that breaks the input into tokens using a delimiter. The delimiter is whitespace by default.
- Importing the class is the first step.

```
import java.util.Scanner;
```
- The object of the class Scanner is declared as follows.

```
Scanner scaninput = new Scanner (System. in);
```
- The object “scaninput” invokes the method `nextInt()` which reads the value typed by the user. This value is assigned to n. The value of m is similarly obtained.
- The other useful methods of Scanner class are `nextDouble()` and `nextLine()`.

Program 2.6: Illustration of a user's input from keyboard into program

```

import java.util.Scanner; public
class Arithmetic
{
    public static void main(String[] args)
    {
        Scanner scaninput = new Scanner (System. in); int n;
        int m;
        System.out. print( "Enter the value of n : "); n=scaninput.nextInt();
        System.out. print( "Enter the value of m : "); m=scaninput.nextInt();

        System.out.println("Sum of two numbers is =" + (n+m));

        System.out.println("Product of two numbers is =" + n*m);

        System.out.println("Modulus of (n % m) is =" + n%m);

        System.out.println("Division of two numbers is =" + n /m );
    }
}

```

OutputC:\>javac Arithmetic.java

```

C:\>java Arithmetic Enter the
value of n : 10 Enter the
value of m : 3 Sum of two
numbers is =13
Product of two numbers is =30 Modulus
of (n % m) is =1 Division of two
numbers is =3

C:\>

```

5. Escape Sequences

Escape Sequences character is preceded by a backslash (\) has a special meaning to the compiler. Escape sequences are as follows.

Table 2.7 Escape sequences

Escape sequences	Description	Example of code
\'	Single quote	"\'"
\"	Double quote	"\""
\\	Backslash	"\\"
\b	Back space	"\b"
\ddd	Octal character	"\132"
\uxxxx	Hexadecimal unicode character	"\u0042"
\f	Form feed	"\f"
\n	New line	"\n"
\r	Carriage return	"\r"
\t	Tab	"\t"

Program 2.9: Program on Escape Sequences

```

class Escape
{
    public static void main(String[] args)
    {
        int n=256, a=0, b=70;
        System.out.println("Value of a =" + a + "\n b= "+b +"\n");
        System.out.println("\u0041 \t" + " \u0042 \t"+"132");
        System.out.println("\nValue of b\n" = "+b);
        System.out.println("\'Value of n\' = " +n);
    }
}

```

Output:

```
C:\>javac Escape.java
```

```
C:\>java Escape
```

```
Value of a =0 b=
```

```
70
```

```
A          B          Z
```

```
"Value of b" = 70 'Value
```

```
of n' = 256
```

6. Comments

- Comments are Line of Text which is not a part of the compiled program.
- Comments are used for documentation to explain source code.
- They are added to the source code of the program.
- Java supports three types of comments as:

1. Single-line comment: These comments are started with two front slash characters (//)

Example:

```
// This is Single line comment
```

2. Multi-line comment : These comments are enclosed with /* and */ Example:

```
/* It is Multi line Comments */
```

3. Documentation comment: These comments are enclosed with /** and */. It is different from multi line comments in that it can be extracted by javadoc utility to generate an HTML document for the program.

Example:

```
/** It is documentation Comments */
```

7. Programming Style.

- An analysis of the programming exercises will throw some light on the look and feel of a program. The team members should easily understand each other's code.
- For a beginner, it is better to develop a habit of writing a program in a proper style so that there is no conflict between the current habits and the company's imposition of style rule at a later stage.
- There are no set patterns of good style and bad style: however, if the programmer takes care of a few requirements on the programs as discussed, the resulting style will be better

1. **The program should present a clean and orderly look. In order to develop a clean program, the programmer can adopt the following measures:**

- (a) The white spaces are neglected by the compiler.
- (b) Indenting is another method often used to improve the looks and readability
- (c) Use of Lambda expression and method reference of Java 8 enhances the look
- (d) Include a space before and after operators

2. The program should be easy to understand. The programmer should take care of the following aspects:

- (e) If it is team work, certain conventions about naming should be preset so that a team member can easily identify an item.
- (f) The makers of Java have a set of rules which are followed in the Java library. The same rules or an even better convention may be set.
- (g) Judicial use of comments can increase understandability. Use of too many comments makes confusion in the program.
- (h) It is better to use already defined and tested library methods rather than user-defined methods
- (i) Expressions like `z = x++ + - - c*++k;` should be avoided.

3. Debugging should be easy. The programmer may adopt the following measures to ensure easy debugging.

- (j) The vertical alignment of a similar item enhances the ability to find errors.
- (k) The code line should not be too long.
- (l) The variables should be declared close to the places of their use
- (m) If it is a big program, it should be divided into small segments. In Java, it is easy because the program may comprise separate classes.
- (n) The methods should not be too big.
- (o) Each source code file should have one class.
- (p) The braces should be vertically aligned, if possible.

4. The program should be easy to use. The following points

- (a) The input into program should be easy,
- (b) The output should be self-explanatory should be taken care of
- (c) The names used should imply the output type such as price, weight, length, and so on.
- (d) Confusing and long names must be avoided.

5. The program should be easy to maintain

- (a) The program should be easily modifiable to ensure simplicity in fixing errors.
- (b) The comments can help in modification of the program, fixing errors.

6. The program should be fail-safe. The failure of a program should not be catastrophic.

DATA TYPES VARIABLES AND OPERATORS

The Java language programs deal with the following entities:

1. Primitive data
2. Classes and objects
3. Interfaces and their references
4. Arrays
5. Methods

The primitive data and their types are defined independent of the classes and interfaces, and the arrays and methods derive their types from the first three entities.

An array is a collection of items that may be of primitive type, class objects, or references. The type of an array can be determined from the type of elements present in it.

Data Types in Java

- Data Type is the type of the data which computer accepts. Every variable and expression has a data type that is known at the compile time.
- The declaration of data type with a variable or expression limits the types of values that a variable can have or the expression it can evaluate.
- Java is an object-oriented programming language based on classes and interfaces.
- Java defines some primitive (basic) data types that are not reference types of any class or interface. Eight primitive (basic) types of data are defined in Java. The type names are also the keywords shown here in bold letters
 1. **Integral types**—byte, short, int, long
 2. **Floating point types** - float, double
 3. **Character type** -char
 4. **Boolean type** – Boolean values – True, False

There is a non-data type called void and no data can be of type void. This type is used for methods that do not return any value.

Java is a case-sensitive language. This means that it takes Area, area, and AREA are three different objects.

2. Declaration of Variables - Data Types

A variable is declared by first declaring its type followed by its identifier or name, which is given as follows:

```
type Identifier;
```

Here type is the primitive data type and Identifier is the name of the variable.

Declaration and Initialization of the variable is as follows.

```
type identifier = value;
```

Ex:

```
byte n; // declares a variable of type byte.
```

```
short m = 67; // declares and initiates a short number
```

```
int length; //declares length-a variable of type int
```

```
length = 50; // value is re-assigned after declaration
```

```
char ch = 'A'; // declares a character variable ch.
```

Non-primitive Types

These are the class and interface types. The name of a class or interface is the name of type.

A class object is declared as

```
Class_identifier object_identifier;
```

Similarly, an interface reference is declared as

```
Interface_identifierreference_identifier;
```

Example: String str "Delhi":

Data Types

i. Integers

Integers are whole numbers, that is, they represent numbers that do not have a fractional part. The integers can be declared in four types according to the size of memory allocated for them

byte

short

int

long

Table 3.1 Integer types and ranges

Type name	Memory allocated in bytes	Range of values of variables
byte	1 byte (8 bits)	-128 to 127
short	2 bytes (16 bits)	-32,768 to 32,767
int	4 bytes (32 bits)	-2,147,483,648 to 2,147,483,647
long	8 bytes (64 bits)	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807

Program 3.1 illustrates the integer data types.

```
class DataType
{
    public static void main (String args[])
    {
        byte a= 4, b=8 ;// variables of type byte
        short c = 67, d = 98; // variables of type short
        int e = 7000, f = 20000; // variables of type int
        long secondsInYear = 365 * 24 * 60 * 60; // long type
        //e=d+f;
        System.out.println("(b + a) = " + (b+a));
        System.out.println("b + a = " + b + a);
        System.out.println("c = "+ c + " \td = " + d);
        System.out.println("e = " + e + "\t f= " + f);
        System.out.println("Seconds in a year = "+ secondsInYear);
    }
}
```

Output:

```
C:\>javac DataType.java C:\
>java DataType
(b + a) = 12
b + a = 84
c = 67 d = 98
e = 7000 f= 20000
Seconds in a year = 31536000
```

ii. Characters

- A variable may have value in terms of a character in which the type of variable is char.
- These characters represent integer values.
- Java supports Unicode for the representation of characters.
- Unicode supports all the character sets of all the major languages of the world.
- The initial version of Unicode allocated 2 bytes for storing characters.
- The range of values for characters in the initial version of Unicode comprised from 'u0000' to 'uffff' that is from 0 to 65535 both end values inclusive.

Program 3 2 illustrates arithmetic operations on character constants and variables

```
class Datachar {
    public static void main (String args[])
    {
        char ch1='E', ch2, ch3 ;

        System.out.println("ch1 =" +ch1); // printing ch1 ch2=ch1++;
        System.out.println("ch2 =" + ch2); // printing ch2

        ch3=++ch1;
        System.out.println("ch3 =" + ch3); // printing ch3
    }
}
```

Output:

```
C:\>javac Datachar.java C:\
>java Datachar
ch1 =E
ch2 =E
ch3 =G
```

iii. Floating Point Numbers

- The numbers that are not whole numbers, or those that have fractional part, Examples are 3.141, 476.6, and so on.
- Java supports two types of such numbers.

Float: This type is used for single-precision decimal floating point numbers, that is, 7 digits

after the decimal point. These numbers are stored on 4 bytes.

Program : Illustration for working with float and double data

class FloatType

```
{
    public static void main (String args[])
    {
        float width =20.0f, length = 40.5f; float
        rectArea = length * width;

        System.out.println("Width = "+ width); System.out.println("Length = "+
        length); System.out.println("Rectangle Area = "+ rectArea);

        double dia=10.0, pi=3.14159;
        double areaCircle = pi* dia * dia / 4;

        System.out.println("Diameter " + dia); System.out.println("Area of Circle
        = " + areaCircle);
    }
}
```

Output

```
C:\>javac FloatType.java C:\>java
FloatType
Width = 20.0
Length = 40.5 Rectangle
Area = 810.0 Diameter
10.0
Area of Circle = 78.53975
```

iv. Boolean Type - Data

- For dealing with logical statements, variable of type boolean is supported.
- The value of boolean type variable is either *true* or *false*.
- The boolean type variables are *unsigned* as similar to char type variables.
- The variable may be declared as follows :
 boolean a;
 a = x > y;

If the aforementioned logical statement is correct, The value of a is true, otherwise the value of a is false. In Java true and false are not converted into numerical values, which is the case in other Languages. A boolean type variable is allocated to one byte, that is, 8 bits for storing its values Program

Program : illustrates the application of boolean type variables.

```
class Boolean
{
    public static void main (String args[])
    {
        double x= 5.5, y=10.5, p=4.0; int n=40,
        m=50;
        boolean a,b,c,d;

        a= x>y; b=
        y>p; c=
        y==x; d=
        x<=y;

        System.out.println("a = " + a + " and b =" +b);
        System.out.println("Now c= "+c+ " and d = "+d);
    }
}
```

C:\>javac Boolean.java

C:\>java Boolean

a = false and b =true

Now c= false and d = true

Type Casting

Converting one data type to another data type is called as Type Casting. There are two types of type casting. They are,

- i. Implicit Type casting
- ii. Explicit Type casting

i. Implicit Type casting

Implicit type casting is done automatically by a compiler when we assign a value to the variable.

Example:

```
int a=10;
double b;
b=a;
```

Here a is integer and d is double variables. Integer value is automatically converted into double by the compiler.

ii. Explicit Type casting

The explicit type casting is carried out by the following code:

```
type variable = (new_type) variable;
```

- It is illustrated by the following code lines:

```
double D = 6.865;
```

```
int A = (int) D;
```

- In such a conversion, there is loss of data

Program : Illustration of type casting.

```
class TypeCast
{
    public static void main (String args[])
    {

        int a=4, b = 8, c = 9, d,e; double x= 3.0,
        y=6.5, z,k;

        d=c/a;
        k=a+y;
        e = a + (int)y; z=(double)c/a;

        System.out.println("k = " + k +" and e =" +e); System.out.println("d= " + d + "
        and z  = "+ z);
    }
}
```

Output

```
C:\>javac TypeCast.java C:\
```

```
>java TypeCast
k = 10.5 and e =10
d= 2 and z  = 2.25
```

SCOPE OF VARIABLE IDENTIFIER

The scope and lifetime of a variable is the part of the program in which it is visible and holds the last entered value In Java, there are distinctly two types of scopes.

- (a) class scope and
- (b) method scope

- A variable declared in a class has class scope and scope of a variable declared in a method has method scope.
- The variables declared in a block have block scope.
- Thus, the variables defined in main() at the beginning have scope in entire main() method, however, those defined in a block have block scope.
- A block starts with the left brace ({) and ends with the right brace (}).

In the case of nested blocks of statements, the scope of variables is governed by the following rules.

1. The scope starts from the point the variable is defined in the block (declared and value assigned to it).
2. Although the variable may be defined anywhere in a block, it can be used only in the statements appearing after its definition Therefore, there is no use in defining a variable at the end of block.

If there are nested blocks, a variable defined in the outer block is visible in the inner blocks also and it cannot be redefined with the same name in the inner blocks

Program: Illustration of block scope

```
public class ScopeA
{
static int x = 5; // Variable scope within the Class
public static void main
(String args[])
{
System.out.println("Class Scope variable - outside main() x = " + x);
int y = 10;
System.out.println("Variable y Scope within main() y = " + y);
{ // Anonymous Block
int z = 20;
System.out.println("Variable z Scope within Anonymous Block z = " + z);
}
}
}
```

E:\>javac ScopeA.java

E:\>java ScopeA

Class Scope variable - outside main() x = 5
Variable y Scope within main() y = 10
Variable z Scope within Anonymous Block z = 20

E:\>java ScopeA

LITERAL CONSTANTS, SYMBOLIC CONSTANTS

Literal Constants

Literal Constants are as follows.

- Each character is a constant value.
- An array of characters or a string also represents a constant value.
- The digits in decimal system, octal system, or hexadecimal system represent constant values.
- The char literals are enclosed in single quotes (' '). The examples 'A' and 'B'. These may also be written as \u0041 and \u0042.
- The string literals are enclosed in double quotes.
Examples : "Delhi", "I am going out."
- The decimal integral literals examples are 5417, 684
- The octal number literals are prefixed with 0 (Zero). Example. 072 , 042
- The floating point literal examples are 684.62f, 5.245E+ 2f
- The literal of Boolean data type are true and false. In Java, neither true is converted to 1 nor

Symbolic Constants

A Symbolic constant is a variable whose value does not change throughout the program.

Some of the examples include PL NORTH, EAST etc.

<i>Name of constant</i>	<i>Value of constant</i>
Pi	3.145926535....
c (speed of light)	299,792,458 m/s
Gravitational Constant	$6.67300 \times 10^{-11} \text{ (m}^3 \text{ Kg}^{-1} \text{ s}^{-2}\text{)}$
Natural Log (e)	2.718281828

It is usually preferred to declare the symbolic constants using all the capital letters in a program as follows:

```
public static final int c = 299792458; public static final  
double PI = 3.21415;
```

Program: illustrates the use of symbolic constant

```
public class SymbolicConst  
{  
    public static final double PI = 3.1415926535; public static  
    void main (String args[])  
    {  
        double r = 25.0, perimeter; perimeter = 2*PI*r;  
        System.out.println("radius=" + r);  
        System.out.println("Perimeter of circle = " + perimeter);  
    }  
}
```

```
C:\>javac SymbolicConst.java
```

```
C:\>java SymbolicConst radius=25.0  
Perimeter of circle = 157.079632675
```

6. Formatted Output with printf() Method

In Java, the formatting of output may be carried out in two ways:

1. By using class Formatter
2. By method printf()

- The Formatter class is used to format the output.
- The printf() method is easy and simple to use, and hence, it is more popular than other methods.
- The following formatting objectives may be realized by using print method
 - i. Right and left justification
 - ii. Precision of floating point numbers by regulating the number of digits after the

- decimal point
- iii. Aligning a number of numbers in a column
- iv. Controlling the placement of characters and strings at desired locations
- v. Representing integers in octal and hexadecimal systems
- vi. Representing floating point numbers in exponential form or regular form
- vii. Representing the time and date in different formats.

The syntax of the method printf () method is as follows

System.out.printf("Formatting string" variables separated by comma);

- The formatting string specifies the output format for each variable that consists of percent (%) sign followed by a conversion letter.
- Thus, the format string for output of an integer and character is "X" and or respectively.
- The order of variables in variable list should match with the list of formats in formatting string.
- The following Table lists the conversion letters for different types of variables.

Table 3.4 Conversion characters for different types of variables

Type variable	Conversion letter	Formatting string
Integers (base 10)	d	"%d"
Integer (base 8) octal	o	"%o"
Integer (base 16) hexadecimal	X or x	"%X" or "%x"
Character	C or c	"%C" or "%c"
String	S or s	"%S" or "%s"
Floating point number base 10	f	"%f"
Floating point number base 10 exponential form	E or e	"%E" or "%e"
Floating point number base 16	A or a	"%A" or "%a"
Floating point number in exponential (e) or regular format (f) depending upon the magnitude of variable. If the magnitude lies between 10^{-3} and 10^7 , regular format is used. Outside these limits, it is formatted in exponential format.	G or g	"%G" or "%g"
For providing space between output values, the empty strings or tab "\t" may be used.	" " or \t	Example of code "%d %f %s" or "%d\t%f\t%s"

Program: illustration of formatting strings for output of different types of variables

```
class FormatPrintf
{
    public static void main(String args[])
    {
```

```

int n = 713; float x = 45.86f;
double d= 56.754;

String str = "Delhi"; char ch = 'A';
System.out.printf( "%d %f  %f  %c \t %s \n", n, x, d, ch, str);

//for conversion into hexadecimal number
System.out.printf("Hexadecimal value of 163 = %X \n", 163);

//for conversion into octal
System.out.printf("Octal value of 163 = %o\n", 163);
}
}
Output

```

C:\>javac FormatPrintf.java C:\>java

```

FormatPrintf
713    45.860001 56.754000 A        Delhi
Hexadecimal value of 163 = A3 Octal
value of 163 = 243

```

STATIC VARIABLES AND METHODS

Static Variables:

- **The static variables are class variables. Only one copy of such variables is kept in the memory and all the objects share that copy.**
- **The static variables are accessed through class reference**, whereas the instance variables are accessed through class object reference
- The variables in a class may be modified by modifier static.
- The non-static variables declared in a class are instance variables Each object of the class keeps a copy of the values of these variables.

Static Methods:

- The static methods are similar to class methods and can be invoked without any reference of object of class, however, class reference (name of class) is needed, as in the following example The method like sqrt() is declared as static method in Math class and is called

```
Math.sqrt(5); // Finds square root of 5
```

The static method is called using the method name that is preceded by the class name; in this case. Math and period ().

Program: illustration of using static methods of class Math

```
public class StaticMethods
```

```

{
    public static void main (String args[])
    {

        System.out.println("The Square root root of 16 = "+ Math.sqrt(16)); System.out.println("The cubroot
            root of 27 = "+ Math.cbrt(27));
        //printing five random variables
        for(int i =1; i<=5;i++)
            System.out.println("Random Number " + i + " = " +(int)(100 *Math.random()));
        }
    }
}

```

E:\>javac StaticMethods.java

E:\>java StaticMethods

```

The Square root root of 16 = 4.0 The cubroot
root of 27 = 3.0
Random Number 1 = 77
Random Number 2 = 69
Random Number 3 = 83
Random Number 4 = 2
Random Number 5 = 66

```

E:\>java StaticMethods

```

The Square root root of 16 = 4.0 The cubroot
root of 27 = 3.0
Random Number 1 = 67
Random Number 2 = 31
Random Number 3 = 10
Random Number 4 = 13
Random Number 5 = 40

```

Attribute Final

Final Variable:

The value of a variable declared final cannot be changed in the program. It makes the variable a constant. A few examples of declarations are as follows:

```
final double PI = 3.14159; // The value of PI cannot be changed in its scope
```

```
final int M = 900; // The value of M cannot be changed in its scope
```

```
final double X = 7.5643; // The value of x cannot be changed in its scope.
```

- As mentioned in the comments, the values of PI, M, and x cannot be changed in their respective scopes.

Final Method:

- The attribute final may be used for **methods** as well as for classes. These are basically connected with inheritance of classes.
- When final keyword is used with Java method, it becomes the final method.
- A final method cannot be overridden in a sub-class.

Final Class:

- A Java class with final modifier is called final class A final class cannot be sub-classed or inherited. Several classes in Java are final including String, Integer, and other wrapper classes.
- There are certain important points to be noted when using final keyword in Java
 - i. New value cannot be reassigned to a variable defined as final in Java.
 - ii. Final keyword can be applied to a member variable, local variable, method, or class.
 - iii. Final member variable must be initialized at the time of declaration.
 - iv. Final method cannot be overridden in Java
 - v. Final class cannot be inheritable in Java
 - vi. Final is different from finally keyword, which is used on Exception handling in Java

Example- 1:

```
public class Final
{
    public static void main (String args[])
    {

        int n =10; // Normal variable
        final int f = 20; // final variable

        System.out.println("n = "+ n);
        System.out.println("f = "+ f);

        n = 50; // Now the value 50 is assigned to variable n
        f = 60; // Error : f is final variable can not be changed
    }
}
```

Output:

```
C:\>javac Final.java
Final.java:13: error: cannot assign a value to final variable f
    f = 60; // Error : f is final variable can not be changed
    ^
1 error C:\>
```

Example-2:

```
public class Final
{
    public static void main (String args[])
    {
```

```
int n = 10; // Normal variable
final int f = 20; // final variable

System.out.println("n = " + n);
System.out.println("f = " + f);

n = 50; // Now the value 50 is assigned to variable n
System.out.println("n = " + n);
}
```

C:\>javac Final.java

C:\>java Final n =
10
f = 20
n = 50

Introduction to Operators

An operator is a symbol that tells the computer to perform certain mathematical and logical calculations.

-The different types of Java operators are,

- a) Arithmetic operators
- b) Relational operators
- c) Logical operators
- d) Increment or decrement operators
- e) Assignment operators
- f) Conditional operators
- g) Bitwise operators
- h) Special operators

Precedence and Associativity of Operators

If the expression contains several operators with the same precedence level, the expression is evaluated according to its *associativity*.

For example, in

$Z = 6 * 4 \% 5;$

Both the operators $*$ and $\%$ have same precedence level.

Highest						
++ (postfix)	-- (postfix)					
++ (prefix)	-- (prefix)	~	!	+ (unary)	- (unary)	(type-cast)
*	/	%				
+	-					
>>	>>>	<<				
>	>=	<	<=	instanceof		
==	!=					
&						
^						
&&						
?:						
->						
=	op=					
Lowest						

Assignment Operator (=)

[Assignment operators:](#)

-Assignment operators are used to assign the result of an expression to a variable.

Operator	Meaning
=	Assignment

-In addition, java has a set of short-hand assignment operators of the form

V OP=EXP;

- It is equivalent to

V=V OP EXP;

-The short-hand assignment operators are

+= , -= , *= , /= , %=

Example:

a+=b----- a=a+b

a-=b ----- a=a-b

a*= b ----- a=a*b

a/=b-----a=a/b

a%=b----- a=a%b

Basic Arithmetic Operators

Operator	Meaning
+	Addition or Unary plus
-	Subtraction or Unary minus
*	Multiplication
/	Division
%	Modulo division

-Integer division truncates any fractional part.

-The modulo division produces the remainder of an integer division.

Examples: a+b, a-b, a*b, a/b, a%b

Arithmetic Expression Types:

Integer arithmetic expression:

-An arithmetic operation involving only integer operands is called integer arithmetic.

int + int = int

int - int = int

int * int = int

int / int = int (truncates decimal part)

int % int = int (results remainder)

Real arithmetic expression:

- An arithmetic operation involving only real operands is called real arithmetic.

real + real = real

real - real = real

real * real = real

real / real = real

real % real = not allowed

Mixed-mode arithmetic expression:

-An arithmetic operation involving one operand is real and the other operand is integer, then it is called mixed-mode arithmetic.

real+int=real

real-int=real

real*int=real

real/int=real

real%int=not allowed

Increment (++) and Decrement (- -) Operators

- The increment operator is ++ which means +1
- The decrement operator is - - which means -1
- The operand must be either incremented or decremented by 1.

Example:

```
m=5;y=++m;      results m=6 and y=6.
m=5;y=m++;      results m=6 and y=5.
m=5;y=--m;      results m=4 and y=4.
m=5;y=m--;      results m=4 and y=5.
```

Ternary Operator or Conditional operators:

- It is also known as Ternary operator.
- The symbols used to construct a conditional expression are ? and :
- A conditional expression is of the form,

`exp1?exp2:exp3;`

Example:

```
(a>b)?System.out.println("a is greater"): System.out.println ("b is greater");
```

- It is equivalent to if-else statement in java

Example:

```
if(a>b)
    System.out.println("a is greater");
else
    System.out.println ("b is greater");
```

Relational Operators

- The comparison between two operands or expressions is done with the help of relational operators.

Operator	Meaning
<	Less than
<=	Less than or equal to
>	Greater than
>=	Greater than or equal to
==	Equality
!=	Inequality

Example:

```
a>b
a>=b
a<b
a<=b
a==b
a!=b
```

Boolean Logical Operators

-The java language has three logical operators.

Operator	Meaning
&&	Logical AND
	Logical OR
!	Logical NOT

-The logical operators &&, || are used when we want to test more than one condition.

&& - used when all the conditions must be true.

|| - used when any of the conditions must be true.

-A logical expression yields a value 0 or 1, according to the truth table.

Operand1	Operand2	Operand1&&operand2	Operand1 operand2
Non-zero	Non-zero	1	1
Non-zero	0	0	1
0	Non-zero	0	1
0	0	0	0

Example:

(a>b)&&(a>c)

(a>b)|| (a>c)

-The logical not is used as a negation or complement of the expression.

Example:

!(y<10) means (y>=10)

Bitwise Logical Operators.

Bitwise operators:

-‘C’ provides bitwise operators that operate on data at the bit-level.

-Bitwise operators interpret operands as string of bits.

-These bit strings are then interpreted according to data type.

-Bitwise operators are of 2 types.

i) Logical bitwise operators.

ii) Shift bitwise operators.

i) Bitwise Logical operators

Operator	Meaning
&	Bitwise AND
	Bitwise OR
^	Bitwise exclusive OR
~	One's complement

Bitwise AND operator:

-The Bitwise AND (&) is a binary operator that requires two integral operands(character or integer).

-It does a bit-by-bit comparison between two operands.

-The result of the comparison is 1 only when both bits are 1, otherwise it is 0.

First operand bit	Second operand bit	Result(&)
0	0	0
0	1	0

1	0	0
1	1	1

Bitwise OR operator:

- The Bitwise inclusive OR (|) is a binary operator that requires two integral operands(character or integer).
- It does a bit-by-bit comparison between two operands.
- The result of the comparison is 0 only when both bits are 0, otherwise it is 1.

First operand bit	Second operand bit	Result()
0	0	0
0	1	1
1	0	1
1	1	1

Bitwise exclusive OR:

- The Bitwise exclusive OR (^) is a binary operator that requires two integral operands(character or integer).
- It does a bit-by-bit comparison between two operands.
- The result of the comparison is 1 only if one of the operands is 1, otherwise it is 0.

First operand bit	Second operand bit	Result(^)
0	0	0
0	1	1
1	0	1
1	1	0

One's complement:

- The one's complement (~) is a unary operator applied to an integral value.
- The result is 1 when the original bit is 0 and it is 0 when the original bit is 1.

Original bit	Result(~)
0	1
1	0

ii) Shift Bitwise operators:

- The shift Bitwise operators move bits to the right or left.

Operator	Meaning
>>	Bitwise shift right
<<	Bitwise shift left
>>>	Shift right with zero fill

Bitwise shift-right operator:

- It moves some number of bits from right to left.
- It requires two integral operands.

Syntax:

Operand1 >> Operand2;

-Here, Operand1 is value to be shifted.

Operand2 is number of bits to be shifted.

Bitwise shift-left operator:

- It moves some number of bits from left to right.
- It requires two integral operands.

Syntax:

Operand1 << Operand2;

-Here, Operand1 is value to be shifted.

Operand2 is number of bits to be shifted.

Special Operators.

- Java supports some special operators such as,

a) instanceof operator

b) member selection operator

a) instanceof operator:

- The instanceof is an object reference operator and returns true if the object on the left-hand side is an instance of the class given on the right hand side.

- This operator allows us to determine whether the object belongs to a particular class or not.

Example:

person instanceof student

- It is true if the object person belongs to the class student, otherwise it is false.

b) member selection operator:

- The dot (.) operator is used to access the instance variables and methods of class objects.

Example:

person.age // Reference to the variable age

person.salary() // Reference to the method salary()

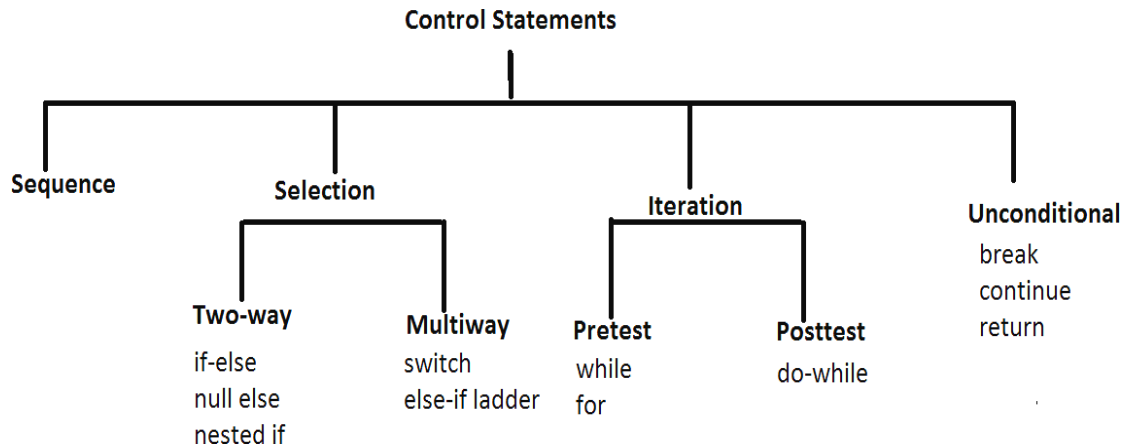
- It is also used to access classes and sub packages from a package.

CONTROL STATEMENTS

1. Introduction

CONTROL STATEMENTS: (FLOW OF CONTROL)

- A Control statement is a statement used to control the flow of execution in a Java Program.



SELECTION STATEMENTS:

-Also called as conditional or decision-making control statements.

-There are two types in Selection control statements.

- i) Two-way selection control statements
- ii) Multi-way selection control statements

i) Two-way selection control statements:

-The different two-way selection statements are,

- a) if-else statement
- b) null else statement
- c) Nested-if statement

2. if Expression - also called as “ null else statement ”

Syntax:

```
if(condition)
{
    statements;
}
next statement;
```

3. if-else Expressions

Syntax:

```
if(condition)
{
    true-block statements;
}
else
{
    false-block statements;
}
```

next statement;

Example:

```
import java.util.*;
```

```
class IfelseExample
```

```
{
    public static void main(String args[])
    {
        int i = 10;

        if (i < 15)
            System.out.println("i is smaller than 15");
        else
            System.out.println("i is greater than 15");
    }
}
```

4. Nested if Expressions

-if within if is called as Nested-if.

Syntax:

```
if(condition-1)
{
    if(condition-2)
    {
        Statement-1;
    }
    else
    {
        Statement-2;
    }
}
else
{

    if(condition-3)
    {
        Statement-3;
    }
    else
    {
        Statement-4;
    }
}
```

next statement;

Example:

```
if(a>b)
{
    if(a>c)
    {
        System.out.println("a is greater");
    }
    else
    {

        System.out.println ("c is greater");
    }
}
else
{

    if(b>c)
    {
        System.out.println("b is greater");
    }
    else
    {
        System.out.println("c is greater");
    }
}

System.out.println("a is greater");
System.out.println("b is greater");
```

b) else-if ladder statement:**Syntax:**

```
if(condition-1)
{
Statement-1;
}
else if(condition-2)
{
Statement-2;
}
.....
.....
else if(condition n-1)
{
Statement-(n-1);
}
else
{
Statement-n;
}
next statement;
```

Example:

```
if(a>b&& a>c&&a>d)
{
    System.out.println("a is greater");
}
else if(b>a&&b>c&&b>d)
{
    System.out.println("b is greater");
}
else if(c>a&&c>b&&c>d)
{
    System.out.println("c is greater");
}
else
{
    System.out.println("d is greater");
}
```

Ternary Operator?:

In Java, the **ternary operator** is a type of Java conditional operator. The meaning of **ternary** is composed of three parts.

The **ternary operator (? :)** consists of three operands. It is used to evaluate Boolean expressions. The operator decides which value will be assigned to the variable. It is the only conditional operator that accepts three operands.

It can be used instead of the if-else statement. It makes the code much more easy, readable, and shorter.

Syntax:

Expression1 ? Expression2 : Expression3

- The first expression is the test condition.
- If it evaluates true, the Expression2 is executed; otherwise Expression3 is executed.

Example-1:

```
public class Ternary
{
    public static void main (String args[])
    {
        int a=10;
        int b = (a<20)? 100 :200; // a < 20 if statement System.out.println("b= "+b);
    }
}
```

Output:

C:\>javac Ternary.java

C:\>java Ternary b=100

Example-2:

```
public class Ternary
{
    public static void main (String args[])
    {
        int a=10;
        int b = (a>20)? 100 :200; // a>20 – else statement System.out.println("b= "+b);
    }
}
```

Output:

C:\>javac Ternary.java

C:\>java Ternary b= 200

Switch Statement

Syntax:

```
switch(expression)
{
casevalue-1:statement-1;break;
case value-2:statement-2;break;
.....
.....
casevalue-n:statement-n;break;
default: default statement;
}
next statement;
```

Example:

```
switch(digit)
{
case 0: System.out.println("ZERO");break;
case 1: System.out.println("ONE");break;
case 2: System.out.println("TWO");break;
case 3: System.out.println("THREE");break;
case 4: System.out.println("FOUR");break;
case 5: System.out.println("FIVE");break;
case 6: System.out.println("SIX");break;
case 7: System.out.println("SEVEN");break;
case 8: System.out.println("EIGHT");break;
case 9: System.out.println("NINE");break; default:
System.out.println("Enter between 0-9");
}
```

Iteration Statements

LOOP STATEMENTS:

- The iteration control statements are also called as Repetition or Iteration control statements.
- A looping process includes the following four steps.
 - Setting and initialization of a counter.
 - Execution of the statements in the loop body.
 - Test for a specified condition (loop control expression) for execution of a loop
 - Incrementing or Decrementing counter.

i) Pretest and Posttest loops:

- In a **Pretest loop**, the condition is checked before we execute a loop body.
- It is also called as **entry-controlled loop**.
- In the **Posttest loop**, we always execute the loop body atleast once.
- It is also called as **exit-controlled loop**.

while Expression

a) while statement:

Syntax: while(condition)

```
{  
loop body;  
}
```

next statement;

Example:

```
n=10,i=1,sum=0;  
while(i<=n)  
{  
sum=sum+i;  
i++;  
}  
System.out.println("sum="+sum);
```

do-while Loop - forLoop

Syntax:

```
do  
{  
    loop body;  
}while(condition);  
next statement;
```

Example:

```
n=10,i=1,sum=0;  
do  
{  
sum=sum+i; i++;  
} while(i<=n);  
System.out.println("sum="+sum);
```

for Loop

for statement:

Syntax:

```
for(initialization;condition;inc or dec)  
{  
    loop body;  
}  
next statement;
```

Example:

```
n=10,i,sum=0;  
for(i=1;i<=n;i++)  
{  
    sum=sum+i;  
}  
System.out.println("sum="+sum);
```

Nested for Loop

Syntax:

```
for(initialization;condition;inc or dec)
{
    for(initialization;condition;inc or dec)
    {
        Inner loop body;
    }

    Outer loop body;
}
next statement;
```

Example:

```
public class RightTriangle
{
    public static void main(String[] args)
    {
        int rows = 5;
        for(int i = 1;i<= rows;i++)
        {
            for (int j = 1;j <= i;j++)
            {
                System.out.print("* ");
            }
            System.out.println();
        }
    }
}
```

Output:

```
*
* *
* * *
* * * *
* * * * *
```

For-Each for Loop

The Java for-each loop or enhanced for loop. It provides an alternative approach to traverse the array or collection in Java. It is mainly used to traverse the array or collection elements. The advantage of the for-each loop is that it eliminates the possibility of bugs and makes the code more readable. It is known as the for-each loop because it traverses each element one by one.

Advantages:

1. Less clutter in code, especially when iterators are used.
2. Less chances of errors.
3. Improves overall readability of program.

Limitations:

1. It is designed to iterate in forward direction only.
2. In iteration, it takes a single step at a time.
3. It cannot simultaneously traverse multiple arrays or collections. Syntax:

```
for (Object obj : Collection_name)
{
    Body of loop
}
```

Example:

```
public class ForEach
{
    public static void main (String args[])
    {
        int myArray[] = {10,20,30,40,50};

        for (int x:myArray)
        {
            System.out.println(" "+x);
        }
    }
}
```

Output:

```
C:\>javac ForEach.java
```

```
C:\>java
ForEach
10
20
30
40
50
```

Break Statement

Unconditional control statements:

- The unconditional control statements are,
 - a) break statement
 - b) continue statement

break statement:

- The break statement skips from the loop or block in which it is defined.
- The control then automatically goes to the first statement after the loop or block.
- The general format is

```
break;
```

Example:

```
public class Break
{
    public static void main (String args[])
    {
```

```

//printing the values from 1 to 10
for(int i =0; i<10;i++)
{
    if (i==5)
        break; //Break Statement
    System.out.println( i );
}
}

```

Output:

C:\>javac Break.java

C:\>ja

va

Break

0

1

2

3

4

Here loop is stopped due to break statement.

Continue Statement.

- The continue statement is used for continuing next iteration of loop statements.
- When it occurs in the loop, it does not terminate but it skips the statements after it.
- It is useful when we want to continue the program without executing any part of the program.
- The general format is

`continue;`

Example:

```

public class Continue
{
    public static void main (String args[])
    {

        //printing the values from 1 to 10
        for(int i =0; i<10;i++)
        {
            if (i==5)
                continue; // Continue statement
            System.out.println( i );
        }
    }
}

```

Output:

C:\>javac Break.java

C:\>java
Continue 0
1
2
3
4
6
7
8
9

Here 5 is not printed because of continue statement. The Iteration at the condition $i = 5$ is skipped or jumped to next statement.

UNIT-2

UNIT II: Classes and Objects: Introduction, Class Declaration and Modifiers, Class Members, Declaration of Class Objects, Assigning One Object to Another, Access Control for Class Members, Accessing Private Members of Class, Constructor Methods for Class, Overloaded Constructor Methods, Nested Classes, Final Class and Methods, Passing Arguments by Value and by Reference, Keyword this.

Methods: Introduction, Defining Methods, Overloaded Methods, Overloaded Constructor Methods, Class Objects as Parameters in Methods, Access Control, Recursive Methods, Nesting of Methods, Overriding Methods, Attributes Final and Static.

2.1 Classes Introduction:

Class is a user defined new data type. Once defined, this new type can be used to create objects of that type.

Class is a template for an object, and an object is an instance of a class.

A **class** in Java is a blueprint or template used to create objects. It defines the **data (fields/variables)** and **behavior (methods)** that objects of that class will have.

The General Form of a Class:

A class is declared by use of the class keyword class class-name

```
{
    type instance-variable1; type
    instance-variable2;
    // ...
    type instance-variableN;
    type methodname1(parameter-list)
    {
    // body of method
    }
    type methodname2(parameter-list)
    {
    // body of method
    }
    // ...
    type methodnameN(parameter-list)
    {
    // body of method
    }
}
```

The data, or variables, defined within a class are called instance variables. The code is contained within methods. Collectively, the methods and variables defined within a class are called members of the class. In most classes, the instance variables are acted upon and accessed by the methods defined for that class. Thus, it is the methods that they can be used.

A Simple Class and its members

```
class Box
{
    double width;
```

```

double height;
double depth;
void volume()
{
    System.out.print("Volume is ");
    System.out.println(width * height * depth);
}
}

```

Above class contains members as width, height, depth of double type and member function volume().

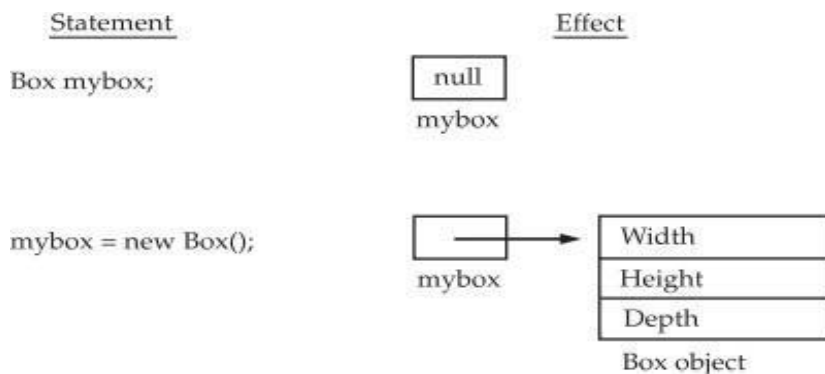
2.1.2 Declaring Objects:

Declaring objects of a class is a two-step process. First, you must declare a variable of the class type. This variable does not define an object. Instead, it is simply a variable that can refer to an object. Second, you must acquire an actual, physical copy of the object and assign it to that variable. You can do this using the **new** keyword. The new keyword dynamically allocates (that is, allocates at run time) memory for an object and returns a reference to it. This reference is, more or less, the address in memory of the object allocated by new.

Ex: Box mybox = new Box();

This statement combines the two steps just described. It can be rewritten like this to show each step more clearly:

Box mybox; // declare reference to object
mybox = new Box(); // allocate a Box object



class members :

In Java, class members are the variables and methods that belong to a class. They define the data and behavior of objects created from the class.

Types of Class Members

1. Fields (Variables) :Store data for the class or objects.

Can be: Instance variables: Each object has its own copy.

Static variables: Shared by all objects of the class.

Example:

```
class Student {
```

```

        String name;        // Instance variable
        static String school = "ABC School"; // Static variable
    }

```

Methods : Define the actions or behavior of the class.

Can be: Instance methods: Operate on object data.

Static methods: Belong to the class itself.

Example: class Student {

```

    String name;
    void display() {    // Instance method
        System.out.println(name);
    }
    static void greet() { // Static method
        System.out.println("Welcome!");
    }
}

```

Constructors :

- Special members used to initialize objects.
- Have the same name as the class and no return type.

Example:

```

class Student {
    String name;
    Student(String n) {
        name = n;
    }
}

```

Nested Classes

- A class defined inside another class.

```

class Outer {
    class Inner {
        void message() {
            System.out.println("Inside Inner Class");
        }
    }
}

```

2.1.3 Assigning One Object to Another:

Box b1 = new Box();

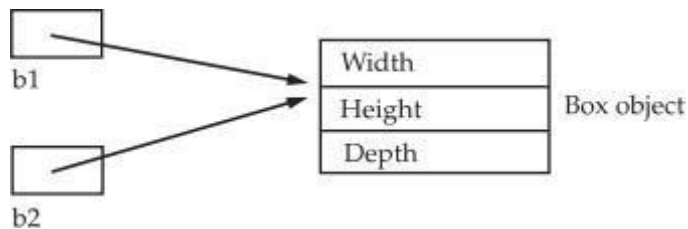
new Box() creates a new Box object in heap memory.

b1 is a reference variable that stores the memory address (reference) of that object.

Box b2 = b1;

No new Box object is created.

b2 is another reference variable that points to the **same object** as b1.



2.1.4 Access Control for Class Members:

Java supplies a rich set of access specifiers. Some aspects of access control are related mostly to inheritance or packages.

Java's access specifiers are **public**, **private**, and **protected**.

Public:

When a member of a class is modified by the public specifier, then that member can be accessed by any other code.

Accessible from **anywhere** in the program.

Example

```
class Student
{
    public String name = "Alice";
}
public class Main
{
    public static void main(String[] args)
    {
        Student s = new Student();
        System.out.println(s.name); // Accessible
    }
}
```

Private:

When a member of a class is specified as private, then that member can only be accessed by other members of its class.

Accessible **only within the same class**.

Example

```
class Student {
    private int marks = 90;

    public void display() {
        System.out.println(marks); // Accessible inside the class
    }
}
```

```

    }
}

public class Main
{
    public static void main(String[] args)
    {
        Student s = new Student();
        s.display();

        // System.out.println(s.marks); // Error: marks is private
    }
}

```

Protected:

When a member of a class is specified as protected, then that member can be accessed by its sub classes with in the package.

Within the **same package**.

In **subclasses** (even if they are in different packages).

Example:

```

class Animal {
    protected void sound()
    {
        System.out.println("Animal makes a sound");
    }
}

public class Main {
    public static void main(String[] args) {
        Animal a = new Animal();
        a.sound(); // Accessible (same package)
    }
}

```

Default :

When no access specifier is used, then by default the member of a class is public within its own package, but cannot be accessed outside of its package.

Example:

```

    class Student
    {
        void display ()
        { // default access
            System.out.println("Hello");
        }
    }

public class Main
{
    public static void main(String[] args)
    {

```

```

Student s = new Student();
s.display(); // Accessible (same package)
}
}

```

Access Modifiers In Java

Access Modifier	Within the Class	Other Classes [Within the Package]	In Subclasses [Within the package and other packages]	Any Class [In Other Packages]
public	Y	Y	Y	Y
protected	Y	Y	Y	N
default	Y	Y	Same Package – Y Other Packages - N	N
private	Y	N	N	N

Y – Accessible
N – Not Accessible

2.1.5 Accessing Private Members of Class:

In Java, private members (variables and methods) can only be accessed inside the same class. They cannot be accessed directly from another class.

Example:

```

class Test {
    int a;    // default access
    public int b; // public access
    private int c; // private access
}

```

```

// methods to access c
void setc(int i) {
    c = i;
}
int getc() {
    return c;
}
}
public class Main {
    public static void main(String args[]) {
        Test ob = new Test();
        ob.a = 1;
        ob.b = 2;
        // This is not OK and will cause an error
        // ob.c = 100; // Error!
        // You must access c through its methods
        ob.setc(100); // OK
        System.out.println("a, b, and c: " + ob.a + " " + ob.b + " " + ob.getc());
    }
}

```

2.1.6 Constructor Methods for Class:

- A constructor initializes an object immediately upon creation.
- It has the same name as the class in which it resides and is syntactically similar to a method.
- Once defined, the constructor is automatically called immediately after the object is created, before the new operator completes.

Constructors look a little strange because they have no return type, not even void. This is because the implicit return type of a class' constructor is the class type itself. It is the constructor's job to initialize the internal state of an object so that the code creating an instance will have a fully initialized, usable object immediately.

Programm:

```

class Box {
    double width;
    double height;
    double depth;
    // This is the constructor for Box.
    Box() {
        System.out.println("Constructing Box");
        width = 10;
        height = 10;
        depth = 10;
    }
    // compute and return volume
    double volume() {
        return width * height * depth;
    }
}

```

```

}

class BoxDemo6
{
public static void main(String args[])
{
// declare, allocate, and initialize Box objects
Box mybox1 = new Box();
Box mybox2 = new Box();
double vol;
// get volume of first box
vol = mybox1.volume();
System.out.println("Volume is " + vol);
// get volume of second box
vol = mybox2.volume();
System.out.println("Volume is " + vol);
}
}

```

When this program is run, it generates the following results: Constructing Box
Constructing Box Volume is 1000.0
Volume is 1000.0

Types of java constructors:

There are two types of constructors:

- Default constructor (no-arg constructor)
- Parameterized constructor

//Default constructor:

```

class Bike1
{
Bike1()
{
System.out.println("Bike is created");
}
public static void main(String args[])
{
Bike1 b=new Bike1();
}
}

```

//Parameterized constructor

```

class Student4
{
int id;
String name;
Student4(int i,String n)
{
id = i; name = n;
}
}

```

```

void display()
{
    System.out.println(id+" "+name);
}
public static void main(String args[])
{
    Student4 s1 = new Student4(111,"Karan");
    Student4 s2 = new Student4(222,"Aryan");
    s1.display();
    s2.display();
}
}

```

2.1.7 Overloaded Constructor Methods:

In addition to overloading normal methods, you can also overload constructor methods. In fact, for most real-world classes that you create, overloaded constructors will be the norm, not the exception. To understand why, let's return to the Box class developed in the preceding chapter. Following is the latest version of Box:

```

class Box
{
    double width;
    double height;
    double depth;
    // This is the constructor for Box.
    Box(double w, double h, double d)
    {
        width = w;
        height = h;
        depth = d;
    }
    // compute and return volume
    double volume()
    {
        return width * height * depth;
    }
}

```

Nested classes:

```

//Demonstrate an inner class.
class Outer
{
    int outer_x = 100; void test()
    {
        Inner inner = new Inner();
        inner.display();
    }
    // this is an inner class
    class Inner {

```

```

void display() {
System.out.println("display: outer_x = " + outer_x);
}
}
}
class InnerClassDemo
{
public static void main(String args[])
{
Outer outer = new Outer();
outer.test();
}
}

```

Output from this application is shown here:
display: outer_x = 100

In the program, an inner class named Inner is defined within the scope of class Outer. Therefore, any code in class Inner can directly access the variable outer_x. An instance method named display() is defined inside Inner. This method displays outer_x on the standard output stream. The main() method of InnerClassDemo creates an instance of class Outer and invokes its test() method. That method creates an instance of class Inner and the display() method is called.

2.1.8 Final Class and Methods:

Final Class:

When a class is declared with final keyword, it is called a final class. A final class cannot be extended(inherited).

There are two uses of a final class :

1. One is definitely to prevent inheritance, as final classes cannot be extended. For example, all Wrapper Classes like Integer,Float etc. are final classes. We can not extend them.

```

final class A
{
    // methods and fields
}
// The following class is illegal.
class B extends A
{
    // COMPILE-ERROR! Can't subclass A
}

```

2. The other use of final with classes is to create an immutable class like the predefined String class. You cannot make a class immutable without making it final.

Final Method:

When a method is declared with final keyword, it is called a final

method. A final method cannot be overridden.

The Object class does this—a number of its methods are final. We must declare methods with final keyword for which we required to follow the same implementation throughout all the derived classes. The following fragment illustrates final keyword with a method:

```
class A
{
    final void m1()
    {
        System.out.println("This is a final method.");
    }
}

class B extends A
{
    void m1()
    {
        // COMPILE-ERROR! Can't
        // override.
        System.out.println("Illegal!");
    }
}
```

2.1.2 Passing Arguments by Value and by reference:

In general, there are two ways that a computer language can pass an argument to a subroutine.

The first way is call-by-value. This method copies the value of an argument into the formal parameter of the subroutine. Therefore, changes made to the parameter of the subroutine have no effect on the argument.

The second way an argument can be passed is call-by-reference.

In this method, a reference to an argument (not the value of the argument) is passed to the parameter. Inside the subroutine, this reference is used to access the actual argument specified in the call. This means that changes made to the parameter will affect the argument used to call the subroutine. As you will see, Java uses both approaches, depending upon what is passed.

For example, consider the following program:

```
// Simple types are passed by
// value.
class Test
{
    void meth(int i, int j)
    {
        i *= 2;
        j /= 2;
    }
}

class CallByValue
{
    public static void main(String args[])
    {
        Test ob = new Test();
        int a = 15, b = 20;
        System.out.println("a and b before call: " + a + " " + b);
        ob.meth(a, b);
    }
}
```

```
System.out.println("a and b after call: " + a + " " + b);
}
}
```

The output from this program is shown here:

**a and b before
call: 15 20 a and
b after call: 15 20**

Pass by Reference: It is a process in which the actual copy of reference is passed to the function. This is called by Reference.

Talking about Java, we can say that Java is Pass by Value and not pass by reference.

2.1.9 this Keyword:

Sometimes a method will need to refer to the object that invoked it. To allow this, Java defines the this keyword. this can be used inside any method to refer to the current object. That is, this is always a reference to the object on which the method was invoked. You can use this anywhere a reference to an object of the cur what this refers to, consider the following version of Box():

```
// A redundant use of this.
Box(double w, double h, double d)
{
    this.width = w;
    this.height = h;
    this.depth = d;
}
```

This version of Box() operates exactly like the earlier version. The use of this is redundant, but perfectly correct. Inside Box(), this will always refer to the invoking object. While it is redundant in this case, this is useful in other contexts, one of which is explained in the next section.

Instance Variable Hiding:

As you know, it is illegal in Java to declare two local variables with the same name inside the same or enclosing scopes. Interestingly, you can have local variables, including formal parameters to methods, which instance variables. However, when a local variable has the same name as an instance

variable, the local variable hides the instance variable.

```
// Use this to resolve name-space
collisions. Box(double width, double
height, double depth)
{
    this.width = width;
    this.height = height;
    this.depth = depth;
}
```

A word of caution: The use of this in such a context can sometimes be confusing, and some programmers are careful not to use local variables and formal parameter names that hide instance variables.

2.2.1 Introducing Methods:

Introduction :

method in Java is a block of code that performs a specific task. Methods help organize a program into smaller, reusable, and easy-to-understand parts. Instead of writing the same code multiple times, you can write it once inside a method and call it whenever needed.

Syntax:

This is the general form of a method:

```
type name(parameter-list)
{
// body of method
}
```

type specifies the type of data returned by the method. This can be any valid type, including class types that you create. If the method does not return a value, its return type must be void.

The name of the method is specified by name. This can be any legal identifier other than those already used by other items within the current scope.

The parameter-list is a sequence of type and identifier pairs separated by commas. Parameters are essentially variables that receive the value of the arguments passed to the method when it is called. If the method has no parameters, then the parameter list will be empty. Methods that have a return type other than void return a value to the calling routine using the following form of the return statement: return value; Here, value is the value returned.

Example :

```
class Box
{
double width;
double height;
double depth;
// display volume of a box
void volume()
{
System.out.print("Volume is ");
System.out.println(width * height * depth);
}
}

class BoxDemo3
{
public static void main(String args[])
{
Box mybox1 = new Box();
Box mybox2 = new Box();
// assign values to mybox1's instance variables
mybox1.width = 10;
mybox1.height = 20;
mybox1.depth = 15;
/* assign different values to mybox2's instance variables */
}
```

```

mybox2.width = 3;
mybox2.height = 6;
mybox2.depth = 9;
// display volume of first box
mybox1.volume();
// display volume of second box
mybox2.volume();
}
}

```

Returning a Value:

While the implementation of `volume()` does move the computation of a box's volume inside the `Box` class where it belongs, it is not the best way to do it. For example, what if another part of your program wanted to know the volume of a box, but not display its value? A better way to implement `volume()` is to have it compute the volume of the box and return the result to the caller. The following example, an improved version of the preceding program, does just that:

```

class Box
{
double width; double height; double depth;
// compute and return volume double volume()
{
    return width * height * depth;
}
}
class BoxDemo4
{
public static void main(String args[])
{
    Box mybox1 = new Box();
    Box mybox2 = new Box();
    double vol;
    // assign values to mybox1's instance variables
    mybox1.width = 10;
    mybox1.height = 20;
    mybox1.depth = 15;
    /* assign different values to mybox2's instance variables */
    mybox2.width = 3;
    mybox2.height = 6;
    mybox2.depth = 9;
    // get volume of first box
    vol = mybox1.volume();
    System.out.println("Volume is " + vol);
    // get volume of second box
    vol = mybox2.volume();
    System.out.println("Volume is " + vol);
}
}

```

2.2.2 Overloaded Methods:

In Java it is possible to define two or more methods within the same class that share the same name, as long as their parameter declarations are different. When this is the case, the methods are said to be overloaded, and the process is referred to as **method overloading**. Method overloading is one of the ways that Java implements polymorphism.

```
// Demonstrate method overloading.

class OverloadDemo {
void test() {
System.out.println("No parameters");
}
// Overload test for one integer
parameter. void test(int a) {
System.out.println("a: " + a);
}
// Overload test for two integer parameters.
void test(int a, int b) {
System.out.println("a and b: " + a + " " + b);
}
// overload test for a double parameter
double test(double a) {
System.out.println("double a: " + a);
return a*a;
}
}
class Overload {
public static void main(String args[]) {
OverloadDemo ob = new OverloadDemo();
double result;
// call all versions of test()
ob.test();
ob.test(10);
ob.test(10, 20);
result = ob.test(123.25);
System.out.println("Result of ob.test(123.25): " + result);
}
}
```

This program generates the following output: No parameters a:

10

a and b: 10 20

double a: 123.25

Result of ob.test(123.25): 15190.5625

As you can see, test() is overloaded four times.

Overloaded Constructor Methods :

Constructor overloading in Java means creating two or more constructors in the same class with different parameter lists. Each constructor initializes objects in a different way.

A constructor is considered overloaded when it has:

A different number of parameters, or Different types of parameters, or a different order of parameter types.

Syntax:

```
class ClassName {
```

```

ClassName() {
    // Default constructor
}

ClassName(int a) {
    // Constructor with one parameter
}

ClassName(int a, String b) {
    // Constructor with two parameters
}
}

```

Example:

```

class Student {
    int id;
    String name;

    // Default constructor
    Student() {
        id = 0;
        name = "Unknown";
    }

    // Parameterized constructor
    Student(int i, String n) {
        id = i;
        name = n;
    }

    void display() {
        System.out.println("ID: " + id);
        System.out.println("Name: " + name);
    }

    public static void main(String[] args) {
        Student s1 = new Student();
        Student s2 = new Student(101, "Ravi");

        s1.display();
        s2.display();
    }
}

```

2.2.3 Class Objects as Parameters in methods:

```

class Test {
    int a, b;
    Test(int i, int j) {
        a = i;
    }
}

```

```

b = j;
}
// return true if o is equal to the invoking object
boolean equals(Test o) {
    if(o.a == a && o.b == b)
        return true;
    else
        return false;
}
}

class PassOb {
    public static void main(String args[]) {
        Test ob1 = new Test(100, 22);
        Test ob2 = new Test(100, 22);
        Test ob3 = new Test(-1, -1);
        System.out.println("ob1 == ob2: " + ob1.equals(ob2));
        System.out.println("ob1 == ob3: " + ob1.equals(ob3));
    }
}

```

This program generates the following output:

ob1 == ob2: true

ob1 == ob3: false

As you can see, the equals() method inside Test compares two objects for equality and returns the result. That is, it compares the invoking object with the one that it is passed. If they contain the same values, then the method returns true. Otherwise, it returns false. Notice that the parameter o in equals() specifies Test as its type

2.2.4 Recursive Methods:

Java supports recursion. Recursion is the process of defining something in terms of itself.

As it relates to Java programming, recursion is the attribute that allows a method to call itself. A method that calls itself is said to be recursive.

The classic example of recursion is the computation of the factorial of a number. The factorial of a number N is the product of all the whole numbers between 1 and N.

// A simple example of recursion(factorial).

```

class Factorial {
    // this is a recursive function
    int fact(int n) {
        int result;
        if(n==1) return 1;
        result = fact(n-1) * n;
        return result;
    }
}

class Recursion {
    public static void main(String args[]) {
        Factorial f = new Factorial();
        System.out.println("Factorial of 3 is " + f.fact(3));
        System.out.println("Factorial of 4 is " + f.fact(4));
        System.out.println("Factorial of 5 is " + f.fact(5));
    }
}

```

```
}
```

The output from this program is shown here:

Factorial of 3 is 6

Factorial of 4 is 24

Factorial of 5 is 120

2.2.5 Nesting of methods:

A method of a class can be called only by an object of that class using the dot operator. So, there is an exception to this. A method can be called by using only its name by another method of the same class that is called Nesting of Methods.

```
class test
{
    int a,b;
    test(int p, int q)
    {

        a=p; b=q;
    }
    int greatest()
    {
        if(a>=b)
            return(a);
        else
            return(b);
    }
    void display()
    {
        int great=greatest();
        System.out.println("The Greatest Value="+great);
    }
}
class temp
{
    public static void main(String args[])
    {
        test t1=new test(25,24);
        t1.display();
    }
}
```

2.2.6 Method overriding:

- In a class hierarchy, when a method in a sub class has the same name and type signature as a method in its super class, then the method in the sub class is said to be override the method in the sub class.
- When an overridden method is called from within a sub class, it will always refers to the version of that method defined by the sub class
- The version of the method defined in the super class is hidden.
- In this situation, first it checks the method is existed in super class are not. If it is existed then it executes the version of sub class otherwise it gives no such method found exception.

Note: Methods with different signatures overloading but not overriding.

// **Method overriding.**

```

class A {
int i, j;
A(int a, int b) {
i = a;
j = b;
}
// display i and j
void show() {
System.out.println("i and j: " + i + " " + j);
}
}
class B extends A {
int k;
B(int a, int b, int c) {
super(a, b);
k = c;
}
// display k – this overrides show() in A
void show() {
System.out.println("k: " + k);
}
}
class Override {
public static void main(String args[]) {
B subOb = new B(1, 2, 3);
subOb.show(); // this calls show() in B
}
}
Output:
k: 3

```

Dynamic method dispatch:

- It is a mechanism by which a call to an overridden method is resolved at run time rather than compile time.
- It is important because this is how java implements runtime polymorphism.
- Before going to that we must know about super class reference sub class object. Example:

```

// Dynamic Method Dispatch class A {
void callme() {
System.out.println("Inside A's callme method");
}
}
class B extends A {
// override callme()
void callme() {
System.out.println("Inside B's callme method");
}
}
class C extends A {
// override callme()
void callme() {
System.out.println("Inside C's callme method");
}
}

```

```

}
}
class Dispatch {
public static void main(String args[]) {
A a = new A(); // object of type A
B b = new B(); // object of type B
C c = new C(); // object of type C
A r; // obtain a reference of type A
r = a; // r refers to an A object
r.callme(); // calls A's version of callme
r = b; // r refers to a B object
r.callme(); // calls B's version of callme
r = c; // r refers to a C object

callme();
// calls C's version of callme
}
}
Output:
Inside A's callme method
Inside B's callme method
Inside C's callme method

```

2.2.7 Attributes Final and Static

Final:

The modifier final may be used with variables as well as methods and classes. If a variable is declared as final it makes it constant. Its value cannot be changed in a program. The value is assigned right in the beginning.

Example: Class PhyTest

```

{
Final int Minmarks=35; // final variable declaration
Void run()
{
Minmarks=40;           //Value of final variable changed ,error
System.out.println("The subject is cleared");

Public static void main(String args[])
{
PhyTest obj=new PhyTest();
Obj.run();
}
}

```

Since the variable Minmarks is declared as final it cannot be modified throughout the program

Static Methods:

By declaring a method as static, it becomes a class method. Then it is not necessary that it should be called through an object. Static methods belong to the class and therefore, it can be accessed without declaring an object of the class.

Example:

Class Computers

```
{  
Public static void main(String args[])  
{  
Show(); //calling method without an object  
Computers obj=new Computers();  
Obj.display(); //calling a method using an object  
}  
Static void show()  
{  
System.out.println("components details of computer");  
}  
void display()  
{  
System.out.println("Main components of computer");  
}  
}
```

UNIT-3

Syllabus

Arrays: Introduction, Declaration and Initialization of Arrays, Storage of Array in Computer Memory, Accessing Elements of Arrays, Operations on Array Elements, Assigning Array to Another Array, Dynamic Change of Array Size, Sorting of Arrays, Search for Values in Arrays, Class Arrays, Two-dimensional Arrays, Arrays of Varying Lengths, Three-dimensional Arrays, Arrays as Vectors.

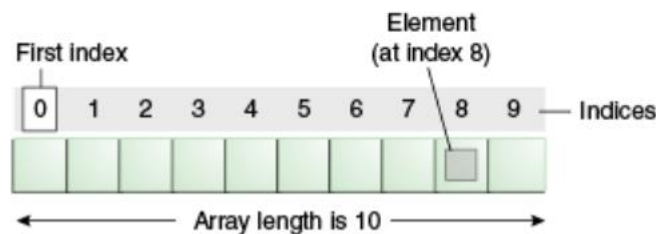
Inheritance: Introduction, Process of Inheritance, Types of Inheritances, Universal Super Class/Object Class, Inhibiting Inheritance of Class Using Final, Access Control and Inheritance, Multilevel Inheritance, Application of Keyword Super, Constructor Method and Inheritance, Method Overriding, Dynamic Method Dispatch, Abstract Classes, Interfaces and Inheritance.

Interfaces: Introduction, Declaration of Interface, Implementation of Interface, Multiple Interfaces, Nested Interfaces, Inheritance of Interfaces, Default Methods in Interfaces, Static Methods in Interface, Functional Interfaces, Annotations.

3.1.1 Introduction:

An array is a collection of elements of the same data type stored in contiguous memory locations. Arrays are used to store multiple values in a single variable, instead of declaring separate variables for each value.

Arrays of any type can be created and may have one or more dimensions. A specific element in an array is accessed by its index. Arrays offer a convenient means of grouping related information.



An array of 10 elements.

Each item in an array is called an *element*, and each element is accessed by its numerical *index*. As shown in the preceding illustration, numbering begins with 0. The 9th element, for example, would therefore be accessed at index 8.

3.1.2 Declaration and Initialization of Arrays

To declare an array, define the variable type with square brackets []

Declaration:

The general form of a one-dimensional array declaration is

type var-name[];

Here, *type* declares the element type (also called the base type) of the array. The element type determines the data type of each element that comprises the array. Thus, the element type for the array determines what type of data the array will hold.

Example:

```
int[] numbers;
```

Here:

- int → Data type of the array elements.
- [] → Indicates that numbers is an array.
- numbers → Name of the array.

At this stage, memory for the elements is **not** allocated. The array reference variable is created only.

Array Creation

Memory is allocated using the new keyword.

Syntax

```
arrayName = new dataType[size];
```

Example

```
int[] numbers;
```

```
numbers = new int[5];
```

This creates an array capable of storing **5 integer values**.

Declaration and Creation in One Statement

```
int[] numbers = new int[5];
```

Declaration, Creation and Initialization Together

```
int[] numbers = {10, 20, 30, 40, 50};
```

Example Program

```
public class ArrayDeclaration {
    public static void main(String[] args) {

        // Declaration and creation
        int[] numbers = new int[5];

        // Assigning values
        numbers[0] = 10;
        numbers[1] = 20;
        numbers[2] = 30;
        numbers[3] = 40;
        numbers[4] = 50;

        // Displaying array elements
        for (int i = 0; i < numbers.length; i++) {
            System.out.println("numbers[" + i + "] = " + numbers[i]);
        }
    }
}
```

Initialization:

Arrays can be initialized when they are declared. The process is much the same as that used to initialize the simple types. An *array initializer* is a list of comma-separated expressions surrounded by curly braces. The commas separate the values of the array elements. The array will automatically be created large enough to hold the number of elements you specify in the array initializer. There is no need to use **new**.

For example, the following code creates an initialized array of integers:

```
public class ArrayDeclaration {
    public static void main(String[] args) {
        int numbers[] = { 10,20,30,40,50 };
        or

        int[] numbers = new int[5] { 10,20,30,40,50};

        // Displaying array elements
        for (int i = 0; i < numbers.length; i++) {
            System.out.println("numbers[" + i + "] = " + numbers[i]);
        }
    }
}
```

When you run this program, you see the same output as that generated by the previous version.

Java strictly checks to make sure you do not accidentally try to store or reference values outside of the range of the array. The Java run-time system will check to be sure that all array indexes are in the correct range. For example, the run-time system will check the value of each index into **numbers** to make sure that it is between 0 and 5 inclusive. If you try to access elements outside the range of the array (negative numbers or numbers greater than the length of the array), you will cause a run-time error.

Alternative Array Declaration Syntax:

Java also supports the following type of code for array declaration. In this declaration the type is followed by square bracket and the name or identifier of the array follows the square bracket.

```
type [ ] identifier; //one dimensional
type [ ] [ ] identifier; //two dimensional
```

Example: *int [] num;*

NOTE: Use of a square bracket before and after the identifier will create a multi-dimensional array.

```
int [ ] num [ ];
```

the above declaration is equivalent to

```
int [ ] [ ] num;            or        int num [ ] [ ];
```

3.1.3 Storage of Array in Computer Memory

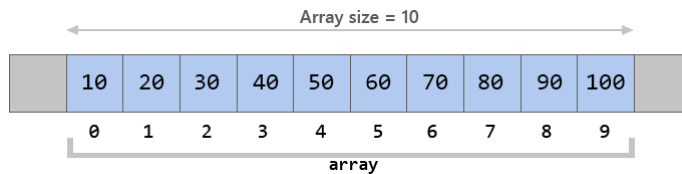
The operator `new` which is a keyword, allocates memory for storing the array elements.

For example with the following declaration

```
int [] num = new int [10];
```

The compiler allocates 10 memory spaces each equal to 4 bytes for storing the `int` type values of elements of array numbers. When an array is created as above, elements of the array are automatically initialized to 0 by the compiler.

```
int num [ ] = {10,20,30,40,50,60,70,80,90,100};
```



A two dimensional array may be declared and initialized as

```
int [ ] [ ] num = new int [ ] [ ] ({1,2,3},{4,5,6});
```

or

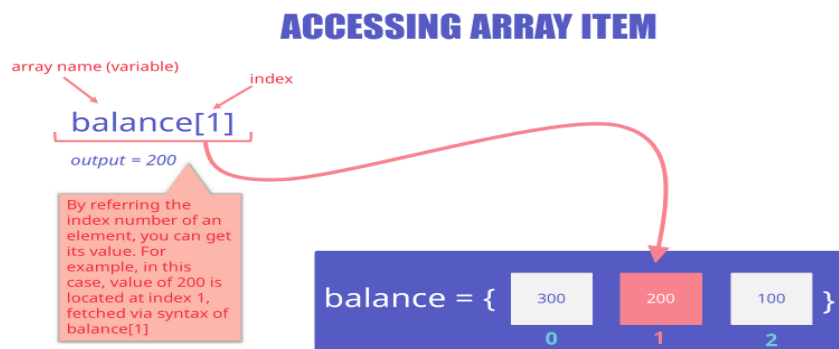
```
int [ ] [ ] num = {{1,2,3},{4,5,6}};
```

3.1.4 Accessing Elements of Arrays

Elements of an array can be accessed by their position, called the index. In Java, array indexing starts from 0 (not 1). To access an element, provide the index inside square brackets `[]` along with the array name.

Syntax

`arrayName[index]`



```
public class ArrayAccessDemo {  
    public static void main(String[] args) {
```

```
        int[] marks = {85, 90, 78, 88, 95};
```

```
        System.out.println("First Element : " + marks[0]);  
        System.out.println("Second Element : " + marks[1]);  
        System.out.println("Third Element : " + marks[2]);  
        System.out.println("Fourth Element : " + marks[3]);  
        System.out.println("Fifth Element : " + marks[4]);  
    }
```

```
}
```

Output

First Element : 85

Second Element : 90

Third Element : 78

Fourth Element : 88

Fifth Element : 95

3.1.1 Operations on Array Elements

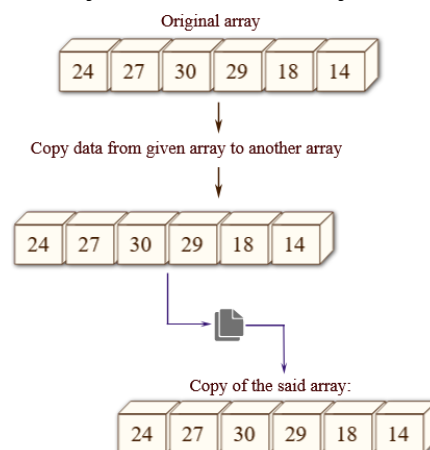
An array element is a variable of the type declared with the array. All the operations that are admissible for that type of a variable can be applied to an individual array element.

Similar to primitive types or objects of classes, the array may also be declared as a parameter of a method.

Access_modifier type method_identifier (type array [], type other_parameter, ...)

3.1.2 Assigning Array to another Array

Unlike in C and C++ languages, in Java, an array may be assigned as a whole to another array of same data type. In this process, the second array identifier, in fact, becomes the reference to the assigned array. The second array is not a new array, instead only a second reference is



created.

```
public class Test {  
    public static void main(String[] args)  
    {  
        int a[] = { 1, 8, 3 };                // Create an array b[] of same size as a[]  
        int b[] = new int[a.length];          // Doesn't copy elements of a[] to b[], only makes b refer to same location  
        b = a;  
        // Change to b[] will also reflect in a[] as 'a' and 'b' refer to same location.  
        b[0]++;  
        System.out.println("Contents of a[] ");  
        for (int i = 0; i < a.length; i++)  
            System.out.print(a[i] + " ");  
        System.out.println("\n\nContents of b[] ");  
        for (int i = 0; i < b.length; i++)  
            System.out.print(b[i] + " ");  
    }  
}
```

}

Output: Contents of a[] 2 8 3

Contents of b[] 2 8 3

3.1.5 Dynamic Change of Array Size

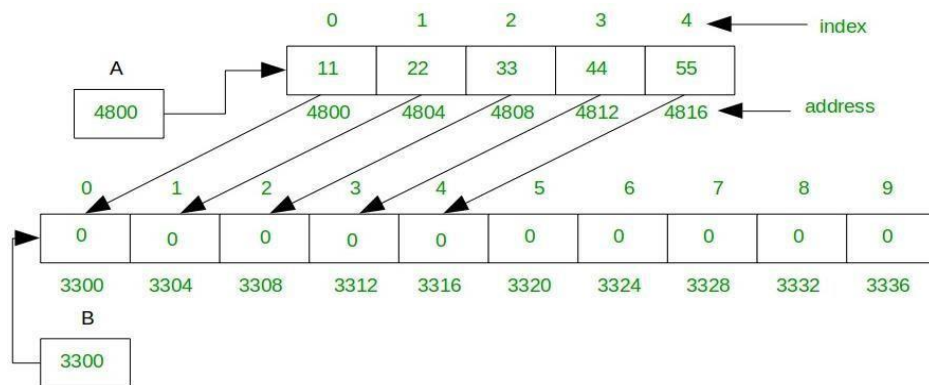
The number of elements (size) of the array may change during the execution of the program. This feature is unlike C and C++ wherein the array once declared is of fixed size, that is, the number of elements cannot be changed.

In Java, however, you may change the number of elements by dynamically retaining the array name. In this process, the old array is destroyed along with the values of elements.

Example:

```
int [ ] num = new int [5];
```

```
num = new int [10];
```



Example:

```
public class ArrayDemo {  
    public static void main(String[] args) {  
  
        int[] num = new int[5];  
  
        System.out.println("First array size: " + num.length);  
  
        num = new int[10];  
  
        System.out.println("New array size: " + num.length);  
    }  
}
```

Output

First array size: 5

New array size: 10

3.1.6 Sorting of Arrays

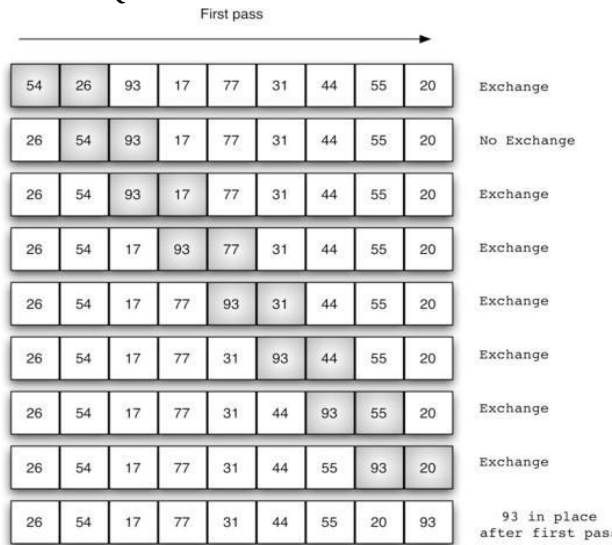
Sorting is the process of arranging the elements of an array in a specific order.

Sorting of arrays is often needed in many applications of arrays.

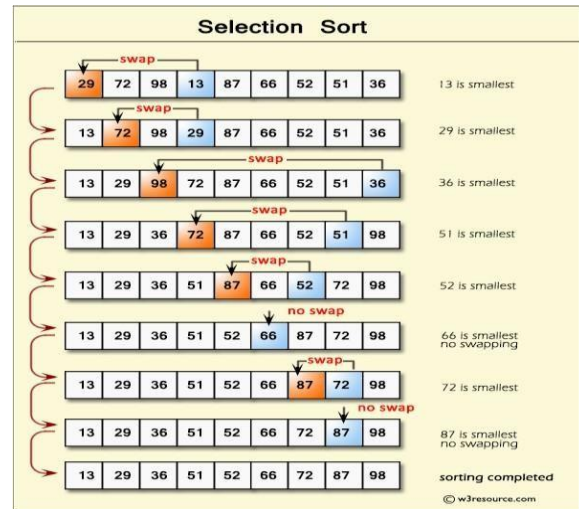
For example, in the preparation of examination results, you may require to arrange the entries in order of grades acquired by students or in alphabetical order in dictionary style. The arrays may be sorted in ascending or descending order.

Several methods are used for sorting the arrays that include the following:

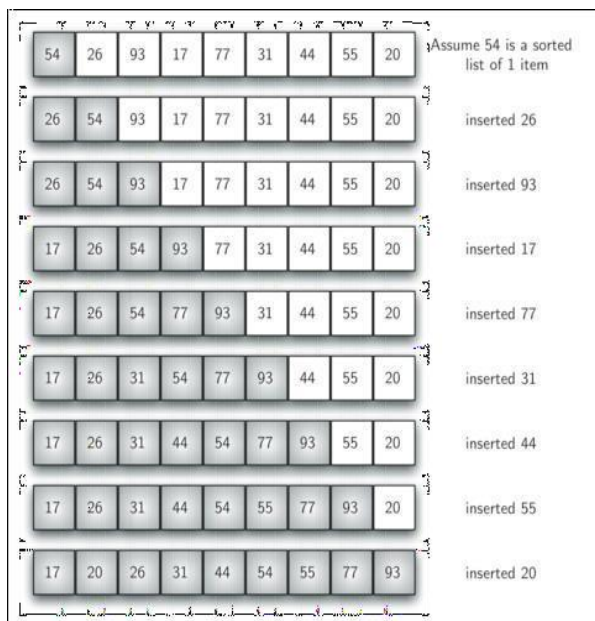
1. Bubble sort
2. Selection sort
3. Sorting by insertion method
4. Quick sort



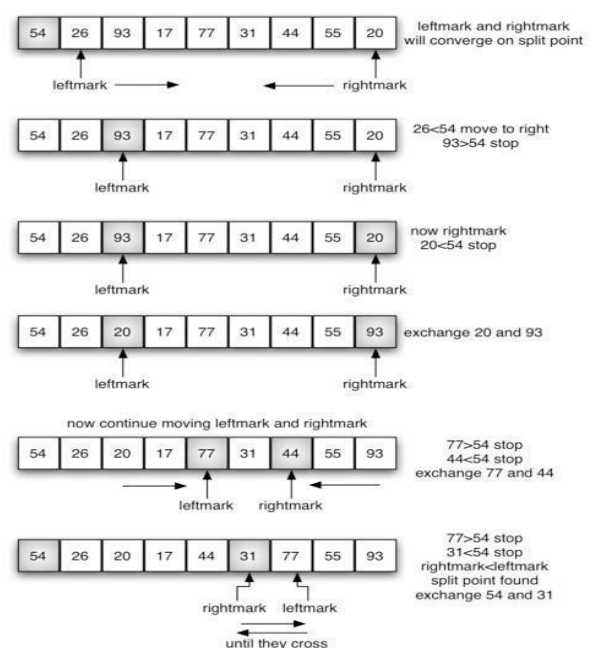
1. Bubble Sort



2. Selection Sort



3. Insertion Sort



4. Quick Sort

3.1.7 Search for Values in Arrays

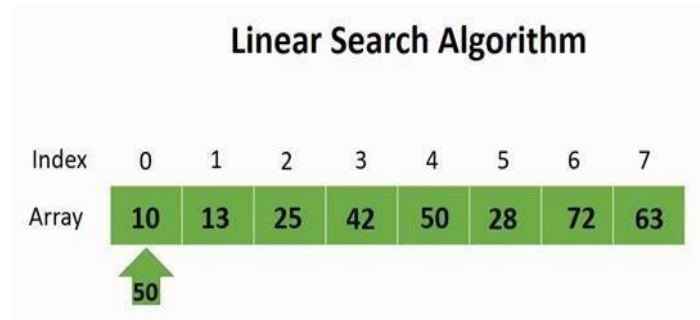
Searching is the process of finding a specific element (value) in an array. In Java, searching is used to determine whether a particular value exists in an array and, if it does, to find its position (index).

The two most common searching techniques are:

1. **Linear Search (Sequential Search)**
2. **Binary Search**

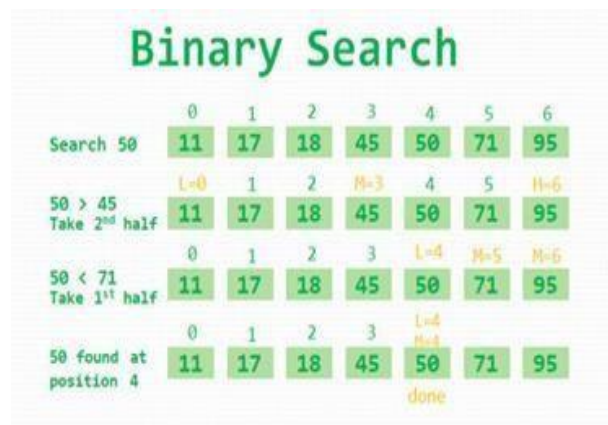
1.Linear Search (Sequential Search)

Linear Search checks each element of the array one by one until the required element is found or the end the array is reached.



2.Binary Search

Binary Search is a faster searching technique that works only on sorted arrays. It repeatedly divides the search interval into two halves until the element is found.



3.1.8 Class Arrays

The package `java.util` defines the class `Arrays` with static methods for general processes that are carried out on arrays such as sorting an array for full length of the array or for part of an array, binary search of an array for the full array or part of array, for comparing two arrays if they are equal or not, for filling a part of the full array with elements having a specified value, and for copying an array to another array. The `sort` method of `Arrays` class is based on quick sort technique.

The methods are applicable to all primitive types as well as to class objects.

The class `Arrays` are declared as

`public class Arrays extends Object`

Methods of Class `Arrays`

Common Methods of the `Arrays` Class

Method	Description
<code>Arrays.sort()</code>	Sorts the array in ascending order.
<code>Arrays.binarySearch()</code>	Searches for an element in a sorted array.

Method	Description
Arrays.fill()	Fills all array elements with a specified value.
Arrays.copyOf()	Creates a copy of an array with a specified length.
Arrays.equals()	Compares two arrays for equality.
Arrays.toString()	Returns a string representation of the array.

3.1.9 Two-dimensional or multi-dimensional Arrays

Two-dimensional arrays are used to store data in rows and columns, where each row can represent a separate individual array. In short, a two-dimensional array contains one-dimensional arrays of elements.

Creating a 2D Array

In Java, we can declare a two-dimensional array in multiple ways, but the correct and most common way to declare a two-dimensional array is as follows:

```
data_type[][] name_of_Array = new data_type[p][q];
```

Where "p" represents the number of rows and "q" represents the number of columns. The total number of elements to be stored in a 2D array is equal to the product of p and q ($p \times q$).

For example:

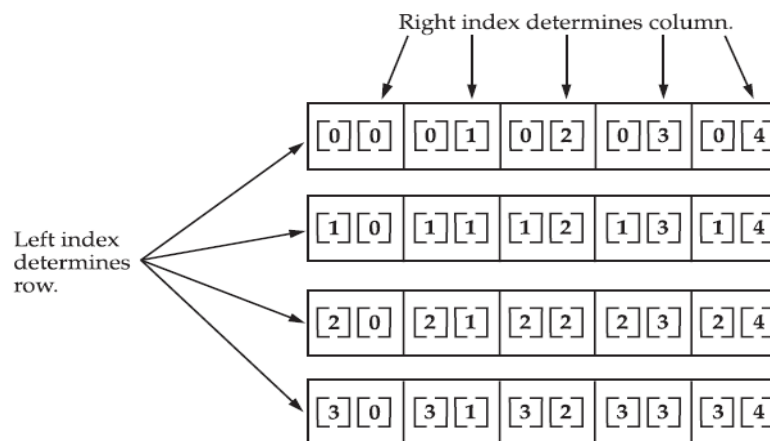
```
int[][] array1 = new int [2][2]
```

The total number of elements to be stored in array1 is $2 \times 2 = 4$.

For example, the following declares a twodimensional array variable called twoD:

```
int twoD[][] = new int[4][5];
```

This allocates a 4 by 5 array and assigns it to twoD. Internally this matrix is implemented as an array of arrays of int. Conceptually, this array will look like the one shown in below figure.



Given: `int twoD [ ] [ ] = new int [4] [5] ;`

A conceptual view of a 4 by 5, two-dimensional array

The following program numbers each element in the array from left to right, top to bottom, and then displays these values:

```
// Demonstrate a two-dimensional array.
```

```
class TwoDArray {
    public static void main(String args[]) {
        int twoD[][]= new int[4][5];
        int i, j, k = 0;
        for(i=0; i<4; i++)
```

```

        for(j=0; j<5; j++) {
            twoD[i][j] = k;
            k++;
        }
        for(i=0; i<4; i++) {
            for(j=0; j<5; j++)
                System.out.print(twoD[i][j] + " ");
            System.out.println();
        }
    }
}

```

This program generates the following output:

```

0 1 2 3 4
5 6 7 8 9
10 11 12 13 14
15 16 17 18 19

```

When you allocate memory for a multidimensional array, you need only specify the memory for the first (leftmost) dimension. You can allocate the remaining dimensions separately. For example, this following code allocates memory for the first dimension of twoD when it is declared. It allocates the second dimension manually.

```

int twoD[][] = new int[4][];
twoD[0] = new int[5];
twoD[1] = new int[5];
twoD[2] = new int[5];
twoD[3] = new int[5];

```

While there is no advantage to individually allocating the second dimension arrays in this situation, there may be in others. For example, when you allocate dimensions manually, you do not need to allocate the same number of elements for each dimension. As stated earlier, since multidimensional arrays are actually arrays of arrays, the length of each array is under your control. For example, the following program creates a two-dimensional array in which the sizes of the second dimension are unequal:

// Manually allocate differing size second dimensions.

```

class TwoDAgain {
    public static void main(String args[]) {
        int twoD[][] = new int[4][];
        twoD[0] = new int[1];
        twoD[1] = new int[2];
        twoD[2] = new int[3];
        twoD[3] = new int[4];
        int i, j, k = 0;
        for(i=0; i<4; i++)
            for(j=0; j<i+1; j++) {
                twoD[i][j] = k;
                k++;
            }
        for(i=0; i<4; i++) {

```

```

        for(j=0; j<i+1; j++)
            System.out.print(twoD[i][j] + " ");
        System.out.println();
    }
}

```

This program generates the following output:

```

0
1 2
3 4 5
6 7 8 9

```

The array created by this program looks like this:

[0][0]			
[1][0]	[1][1]		
[2][0]	[2][1]	[2][2]	
[3][0]	[3][1]	[3][2]	[3][3]

The use of uneven (or irregular) multidimensional arrays may not be appropriate for many applications, because it runs contrary to what people expect to find when a multidimensional array is encountered. However, irregular arrays can be used effectively in some situations. For example, if you need a very large two-dimensional array that is sparsely populated (that is, one in which not all of the elements will be used), then an irregular array might be a perfect solution.

It is possible to initialize multidimensional arrays. To do so, simply enclose each dimension's initializer within its own set of curly braces. The following program creates a matrix where each element contains the product of the row and column indexes. Also notice that you can use expressions as well as literal values inside of array initializers.

// Initialize a two-dimensional array.

```

class Matrix {
    public static void main(String args[]) {
        double m[][] = {
                                { 0*0, 1*0, 2*0, 3*0 },
                                { 0*1, 1*1, 2*1, 3*1 },
                                { 0*2, 1*2, 2*2, 3*2 },
                                { 0*3, 1*3, 2*3, 3*3 }
                            };

        int i, j;
        for(i=0; i<4; i++) {

```

```

        for(j=0; j<4; j++)
            System.out.print(m[i][j] + " ");
            System.out.println();
        }
    }
}

```

This program generates the following output:

```

0 0 0 0
0 0 0 0
0 0 0 0
0 0 0 0
0 0 0 0
0 0 0 0
0 1 2 3 4
0 2 4 6 8
0 3 6 9 12
0 0 0 0
0 2 4 6 8
0 4 8 12 16
0 6 12 18 24

```

Alternative Array Declaration Syntax

There is a second form that may be used to declare an array:

```
type[ ] var-name;
```

Here, the square brackets follow the type specifier, and not the name of the array variable. For example, the following two declarations are equivalent:

```
int a1[] = new int[3];
int[] a2 = new int[3];
```

The following declarations are also equivalent:

```
char twod1[][] = new char[3][4];
char[][] twod2 = new char[3][4];
```

This alternative declaration form offers convenience when declaring several arrays at the same time. For example,

```
int[] nums, nums2, nums3;    // create three arrays
```

creates three array variables of type int. It is the same as writing

```
int nums[], nums2[], nums3[];    // create three arrays
```

The alternative declaration form is also useful when specifying an array as a return type for a method.

3.1.10 Arrays as Vectors

Similar to arrays, vectors are another kind of data structure that is used for storing information. Using vector, we can implement a dynamic array. As we know, an array can be declared in the following way:

```
int marks[ ] = new int [7];
```

the basic difference between arrays and vectors is that vectors are dynamically allocated, where as arrays are static. The size of vector can be changed as and when required, but this is not true for arrays.

The vector class is contained in java.util package. Vector stores pointers to the objects and not objects themselves. The following are the vector constructors.

```
Vector vec = new Vector( 5 ); // size of vector is 5
```

3.2 INHERITANCE

3.2.1 Introduction:

Inheritance is the technique which allows us to inherit the data members and methods from base class to derived class.

- **Base class is one which always gives its features to derived classes.**
- **Derived class is one which always takes features from base class.**

A Derived class is one which contains some of features of its own plus some of the data members from base class.

3.2.1 Process of inheritance

Syntax for INHERITING the features from base class to derived class:

```
class <clsname-2> extends <clsname-1>
{
    Variable declaration;
    Method definition;
}
```

Here, **clsname-1** and **clsname-2** represents derived class and base class respectively.

Extends is a keyword which is used for inheriting the data members and methods from base class to the derived class and it also improves functionality of derived class.

For example:

```
class c1
{
    int a;
    void f1()
    {
        .....;
    }
}
class c2 extends c1
{
    int b;
    void f2()
    {
        .....;
    }
}
```

Whenever we inherit the base class members into derived class, when we creates an object of derived class, JVM always creates the memory space for base class members first and later memory space will be created for derived class members.

Example:

Write a JAVA program computes sum of two numbers using inheritance? Answer:

```
class Bc
{
    int a;
}
class Dc extends Bc
{
    int b;
    void set (int x, int y)
    {
        a=x; b=y;
    }
    void sum ()
    {
        System.out.println ("SUM = "+(a+b));
    }
}
class InDemo
{
    public static void main (String k [])
    {
        Dc do1=new Dc ();
        do1.set (10,12);
        do1.sum ();
    }
}
```

3.2.1 Types of Inheritances



(a) Single Inheritance

Single Inheritance: It means when a base class acquired the properties of super class

```
class Animal
{
    void eat()
    {
        {
```

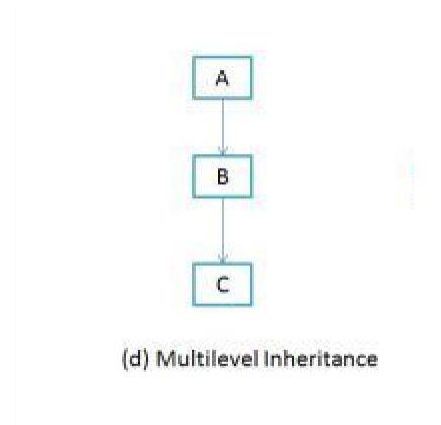
```

    System.out.println("eating...");
}
}
class Dog extends Animal
{
void bark()
{
    System.out.println("barking...");
}
}
class TestInheritance
{
public static void main(String args[])
{
    Dog d=new Dog(); d.bark();
    d.eat();
}
}

```

In this example base class is Dog and super class is Animal:

Multilevel Inheritance:



Multilevel inheritance refers to a mechanism in OO technology where one can inherit from a derived class, thereby making this derived class the base class for the new class. As you can see in below flow diagram C is subclass or child class of B and B is a child class of A

Example:

```

Class X
{
    public void methodX()
    {
        System.out.println("Class X method");
    }
}
Class Y extends X
{
    public void methodY()

```

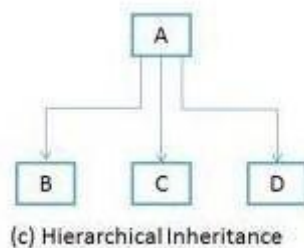
```

{
System.out.println("class Y method");
}
}
Class Z extends Y
{
    public void methodZ()
    {
        System.out.println("class Z method");
    }
    public static void main(String args[])
    {
        Z obj = new Z();
        obj.methodX(); //calling grand parent class method
        obj.methodY(); //calling parent class method
        obj.methodZ(); //calling local method
    }
}

```

Hierarchical Inheritance

In such kind of inheritance one class is inherited by many sub classes. In below example class B,C and D inherits the same class A. A is parent class (or base class) of B,C & D



Example:

```

class A
{
    public void methodA()
    {
        System.out.println("method of Class A");
    }
}
class B extends A
{
    public void methodB()
    {
        System.out.println("method of Class B");
    }
}

```

```

class C extends A
{
    public void methodC()
    {
        System.out.println("method of Class C");
    }
}
class D extends A
{
    public void methodD()
    {
        System.out.println("method of Class D");
    }
}
class JavaExample
{
    public static void main(String args[])
    {
        B obj1 = new B();
        C obj2 = new C();
        D obj3 = new D();
    }
}

```

Output:

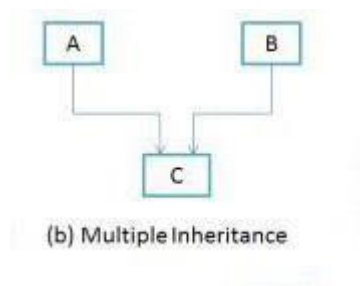
```

method of Class A
method of Class A
method of Class A

```

Multiple Inheritance:

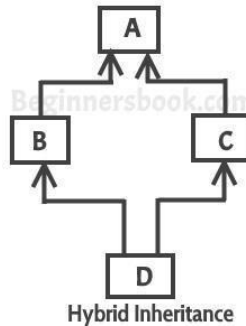
Multiple Inheritance” refers to the concept of one class extending (Or inherits) more than one base class. The inheritance we learnt earlier had the concept of one base class or parent. The problem with “multiple inheritance” is that the derived class will have to manage the dependency on two base classes.



Note: Multiple Inheritance is very rarely used in software projects. Using Multiple inheritance often leads to problems in the hierarchy. This results in unwanted complexity when further extending the class.

Hybrid Inheritance in Java

A hybrid inheritance is a combination of more than one types of inheritance. For example when class A and B extends class C & another class D extends class A then this is a hybrid inheritance, because it is a combination of single and hierarchical inheritance.



The diagram is just for the representation, since multiple inheritance is not possible in java

```
class C
{
    public void disp()
    {
        System.out.println("C");
    }
}

class A extends C
{
    public void disp()
    {
        System.out.println("A");
    }
}

class B extends C
{
    public void disp()
    {
        System.out.println("B");
    }
}

class D extends A
{
    public void disp()
    {
        System.out.println("D");
    }
    public static void main(String args[]){
```

```

        D obj = new D();
        obj.disp();
    }
}

```

This example is just to demonstrate the hybrid inheritance in Java. Although this example is meaningless, you would be able to see that how we have implemented two types of inheritance(single and hierarchical) together to form hybrid inheritance.

Class A and B extends class C → Hierarchical inheritance

Class D extends class A → Single inheritance

3.2.2 Inhibiting Inheritance of Class Using Final

The final keyword in java is used to restrict the user. The java final keyword can be used in many context.

Final can be:

1. variable
2. method
3. class

The main purpose of using a class being declared as final is to prevent the class from being subclassed. If a class is marked as final then no class can inherit any feature from the final class.

Example:

```

final class Bike
{
}
class Honda1 extends Bike
{
    void run()
    {
        System.out.println("running safely with 100kmph");
    }
    public static void main(String args[])
    {
        Honda1 honda= new Honda1(); honda.run();
    }
}

```

Output:

Compile time error

Since class **Bike** is declared as final so the derived class **Honda** cannot extend **Bike**

3.2.1 Access Control and Inheritance

Although a subclass includes all of the members of its superclass, it cannot access those members of the superclass that have been declared as private.

```
class A
{
int i; // public by default
private int j; // private to A
    void setij(int x, int y)
    {
        i = x;
        j = y;
    }
}
// A's j is not accessible here.
class B extends A
{
    int total;
    void sum()
    {
        total = i + j; // ERROR, j is not accessible here
    }
}
class Access
{
    public static void main(String args[])
    {
        B subOb = new B();
        subOb.setij(10, 12);
        subOb.sum();
        System.out.println("Total is " + subOb.total);
    }
}
```

This program will not compile because the reference to j inside the sum() method of B causes an access violation. Since j is declared as private, it is only accessible by other members of its own class. Subclasses have no access to it.

3.2.1 Application of Keyword Super

Super keyword is used for differentiating the base class features with derived class features. Super keyword is placing an important role in three places.

- variable level
- method level

- constructor level

Super at variable level:

Whenever we inherit the base class members into derived class, there is a possibility that base class members are similar to derived class members.

In order to distinguish the base class members with derived class members in the derived class, the base class members will be preceded by a keyword super.

For example:

```
class Bc
{
    int a;
}
class Dc extends Bc
{
    int a;
    void set (int x, int y)
    {
        super.a=x;
        a=y; //by default 'a' is preceded with 'this.' since 'this.' represents current class
    }
    void sum ()
    {
        System.out.println ("SUM = "+(super.a+a));
    }
}
class InDemo1
{
    public static void main (String k [])
    {

        Dc do1=new Dc (); do1.set (20, 30);
        do1.sum ();
    }
}
```

Super at method level:

Whenever we inherit the base class methods into the derived class, there is a possibility that base class methods are similar to derived methods.

To differentiate the base class methods with derived class methods in the derived class, the base class methods must be preceded by a keyword super.

Syntax for super at method level: super. base class method name For example:

```

class Bc
{
    void display ()
    {
        System.out.println ("BASE CLASS - DISPLAY...");
    }
};
class Dc extends Bc
{
    void display ()
    {
        super.display (); //refers to base class display method
        System.out.println ("DERIVED CLASS- DISPLAY...");
    }
}
class InDemo2
{
    public static void main (String k [])
    {
        Dc do1=new Dc ();
        do1.display ();
    }
}

```

3.2.3 Constructor Method and Inheritance

Super at constructor level:

super() can be used to invoke immediate parent class constructor.

```

class A
{
    int i,j;
    A(int a,int b)
    {
        i=a; j=b;
    }
    void show()
    {
        System.out.println("i and j values are"+i+" "+j);
    }
}
class B extends A
{
    int k;
    B(int a, int b, int c)
    {
        super(a, b); //super class constructor
    }
}

```

```

    k = c;
}
// display k – this overrides show() in A
void show()
{
    super.show();
    System.out.println("k: " + k);
}

}

```

```

class Override
{

public static void main(String args[])
{
    B subOb = new B(1, 2, 3);
    subOb.show(); // this calls show() in B

}
}

```

3.2.4 Method Overriding // Refer 2nd unit

3.2.5 Dynamic Method Dispatch // Refer 2nd unit

3.2.6 Abstract Classes:

In JAVA we have two types of classes. They are concrete classes and abstract classes.

- A concrete class is one which contains fully defined methods. Defined methods are also known as implemented or concrete methods. With respect to concrete class, we can create an object of that class directly.
- An abstract class is one which contains some defined methods and some undefined methods. Undefined methods are also known as unimplemented or abstract methods. Abstract method is one which does not contain any definition.

To make the method as abstract we have to use a keyword called abstract before the function declaration.

Syntax for ABSTRACT CLASS: abstract return_type method_name (parameters list);

// A Simple demonstration of abstract.

```

abstract class A
{
    abstract void callme();
}

```

```

// concrete methods are still allowed in abstract classes
void callmetoo()
{
    System.out.println("This is a concrete method.");
}
}
class B extends A
{
    void callme()
    {
        System.out.println("B's implementation of callme.");
    }
}
class AbstractDemo
{
    public static void main(String args[])
    {
        B b = new B();
        b.callme();
        b.callmetoo();
    }
}

```

Notice that no objects of class A are declared in the program. As mentioned, it is not possible to instantiate an abstract class.

One other point: class A implements a concrete method called callmetoo()

3.3 INTERFACE

3.3.1 Introduction:

An interface in java is a blueprint of a class.

It has static constants and abstract methods. The interface in java is a mechanism to achieve abstraction. There can be only abstract methods in the java interface not method body.

It is used to achieve abstraction and multiple inheritance in Java.

- Using the keyword interface, you can fully abstract a class' interface from its implementation.
- That is, using interface, you can specify what a class must do, but not how it does it.
- Interfaces are syntactically similar to classes, but they lack instance variables, and their methods are declared without any body
- Variables can be declared inside of interface declarations.
- They are implicitly final and static, meaning they cannot be changed by the implementing class.
- They must also be initialized. All methods and variables are implicitly public.

3.3.1 Declaring an Interface

An interface is defined much like a class. This is the general form of an interface:

```
interface interface_name {  
    return-type method-name1(parameter-list);  
    return-type method-name2(parameter-list);  
    type final-varname1 = value;  
    type final-varname2 = value;  
    // ...  
    return-type method-nameN(parameter-list);  
    type final-varnameN = value;  
}
```

Here is an example of an interface definition. It declares a simple interface that contains one method called `callback()` that takes a single integer parameter.

```
interface Callback  
{  
    void callback(int param);  
}
```

3.3.2 Implementing Interfaces

Once an interface has been defined, one or more classes can implement that interface. To implement an interface, include the implements clause in a class definition, and then create the methods defined by the interface. The general form of a class that includes the implements clause looks like this:

```

class classname [extends superclass] [implements interface [,interface...]] {
class Client implements Callback
{
// Implement Callback's interface
public void callback(int p)
{
System.out.println("callback called with " + p);
}
}
}

```

Notice that callback() is declared using the public access specifier.

It is both permissible and common for classes that implement interfaces to define additional members of their own. For example, the following version of Client implements callback() and adds the method nonIfaceMeth():

```

class Client implements Callback
{
// Implement Callback's interface public void callback(int p)
{
System.out.println("callback called with " + p);
}
void nonIfaceMeth()
{
System.out.println("Classes that implement interfaces " + "may also define other members, too.");
}
}

```

Accessing Implementations Through Interface References:

You can declare variables as object references that use an interface rather than a class type. Any instance of any class that implements the declared interface can be referred to by such a variable. When you call a method through one of these references, the correct version will be called based on the actual instance of the interface being referred to.

The following example calls the callback() method via an interface reference variable:

```

class TestIface
{
public static void main(String args[])
{
Callback c = new Client();
c.callback(42);
}
}

```

The output of this program is shown here:
callback called with 42

3.3.3 Multiple Interfaces

Multiple inheritance in java can be achieved through interfaces:

Example:

interface Printable

```
{  
void print();  
}
```

interface Showable

```
{  
void show();  
}
```

class A7 implements Printable, Showable

```
{  
    public void print()  
    {  
        System.out.println("Hello");  
    }  
    public void show()  
    {  
        System.out.println("Welcome");  
    }  
    public static void main(String args[])  
    {  
        A7 obj = new A7();  
        obj.print();  
        obj.show();  
    }  
}
```

Output:

Hello Welcome

3.3.4 Nested Interfaces:

An interface i.e. declared within another interface or class is known as nested interface. The nested interfaces are used to group related interfaces so that they can be easy to maintain. The nested interface must be referred by the outer interface or class. It can't be accessed directly.

There are given some points that should be remembered by the java programmer.

- Nested interface must be public if it is declared inside the interface but it can have any access modifier if declared within the class.
- Nested interfaces are declared static implicitly.

Example:

```
interface Showable
```

```
{  
    void show();  
        interface Message  
        {  
            void msg();  
        }  
}
```

```
class TestNestedInterface1 implements Showable.Message
```

```
{  
    public void msg()  
    {  
        System.out.println("Hello nested interface");  
    }  
}
```

```
public static void main(String args[])
```

```
{  
    Showable.Message message=new TestNestedInterface1();//upcasting here  
    message.msg();  
}  
}
```

As you can see in the above example, we are accessing the Message interface by its outer interface Showable because it cannot be accessed directly. It is just like almirah inside the room, we cannot access the almirah directly because we must enter the room first.

3.3.5 Inheritance of Interfaces

One interface can inherit another by use of the keyword extends. The syntax is the same as for inheriting classes. When a class implements an interface that inherits another interface, it must provide implementations for all methods defined within the interface inheritance chain

```
// One interface can extend another. interface A
```

```
{  
    void meth1(); void meth2();  
}
```

```
// B now includes meth1() and meth2() -- it adds meth3().
```

```
interface B extends A
```

```
{  
    void meth3();  
}
```

```
// This class must implement all of A and B
class MyClass implements B
{
    public void meth1()
    {
        System.out.println("Implement meth1().");
    }
    public void meth2()
    {
        System.out.println("Implement meth2().");
    }
    public void meth3()
    {
        System.out.println("Implement meth3().");
    }
}
class IFExtend
{
    public static void main(String arg[])
    {
        MyClass ob = new MyClass();
        ob.meth1();
        ob.meth2();
        ob.meth3();
    }
}
```

3.3.6 Default methods in interfaces

Before Java 8, interfaces could have only abstract methods. The implementation of these methods has to be provided in a separate class. So, if a new method is to be added in an interface, then its implementation code has to be provided in the class implementing the same interface. To overcome this issue, Java 8 has introduced the concept of default methods which allow the interfaces to have methods with implementation without affecting the classes that implement the interface

Example:

```
interface TestInterface
{
    // abstract method
    public void square(int a);

    // default method default
    void show()
    {
```

```

        System.out.println("Default Method Executed");
    }
}

```

class TestClass implements TestInterface

```

{
    // implementation of square abstract method
    public void square(int a)
    {
        System.out.println(a*a);
    }

    public static void main(String args[])
    {
        TestClass d = new TestClass();
        d.square(4);

        // default method executed
        d.show();
    }
}

```

Output:

16

Default Method Executed

Default methods are also known as defender methods or virtual extension methods.

3.3.7 Static methods in interfaces

The interfaces can have static methods as well which is similar to static method of classes.

Example:

```

interface TestInterface
{
    // abstract method

    public void square (int a);

    // static method static
    void show()
    {
        System.out.println("Static Method Executed");
    }
}

```

class TestClass implements TestInterface

```

{

```

```

// Implementation of square abstract method
public void square (int a)
{
    System.out.println(a*a);
}

public static void main(String args[])
{
    TestClass d = new TestClass();
    d.square(4);

    // Static method executed
    TestInterface.show();
}
}

```

Output:

16

Static Method Executed

3.3.8 Functional Interfaces

A functional interface is an interface that contains only one abstract method. They can have only one functionality to exhibit. From Java 8 onwards, lambda expressions can be used to represent the instance of a functional interface.

A functional interface can have any number of default methods. Runnable, ActionListener, Comparable are some of the examples of functional interfaces.

3.3.9 FunctionalInterface annotation is used to ensure that the functional interface can't have more than one abstract method. In case more than one abstract methods are present, the compiler flags an 'Unexpected @FunctionalInterface annotation' message.

@FunctionalInterface interface Square

```

{
    int calculate(int x);
}

```

class Test

```

{
    public static void main(String args[])
    {
        int a = 5;

        // lambda expression to define the calculate method
        Square s = (int x)->x*x;

        // parameter passed and return type must be
        // same as defined in the prototype
        int ans = s.calculate(a);
        System.out.println(ans);
    }
}

```

}

Output:

25

UNIT IV

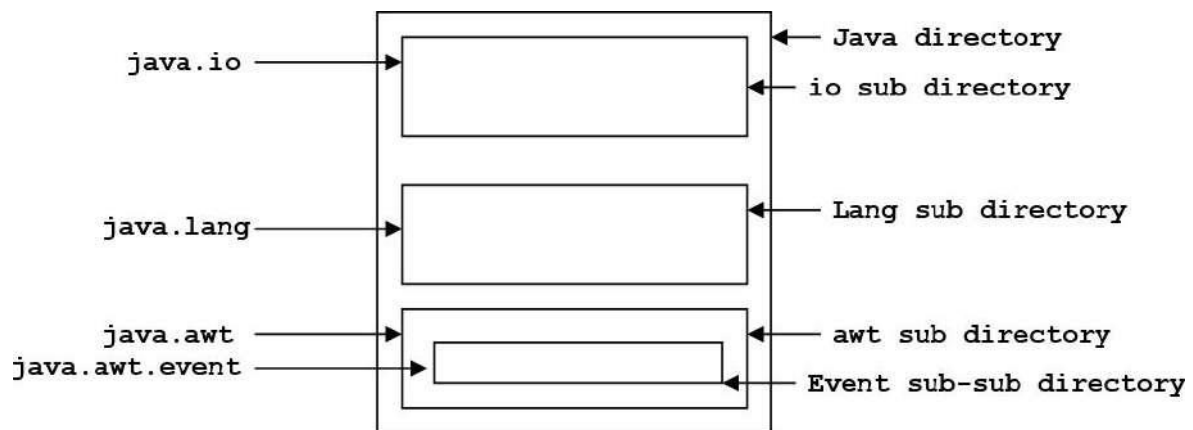
Packages and Java Library: Introduction, Defining Package, Importing Packages and Classes into Programs, Path and Class Path, Access Control, Packages in Java SE, Java.lang Package and its Classes, Class Object, Enumeration, class Math, Wrapper Classes, Autoboxing and Auto-unboxing, Java util Classes and Interfaces, Formatter Class, Random Class, Time Package, Class Instant (java.time.Instant), Formatting for Date/Time in Java, Temporal Adjusters Class, Temporal Adjusters Class.

Exception Handling: Introduction, Hierarchy of Standard Exception Classes, Keywords throws and throw, try, catch, and finally Blocks, Multiple Catch Clauses, Class Throwable, Unchecked Exceptions, Checked Exceptions.

Java I/O and File: Java I/O API, standard I/O streams, types, Byte streams, Character streams, Scanner class, Files in Java. Packages and Java Library:

INTRODUCTION

- It is necessary in software development to create several classes and interfaces.
- After creating these classes and interfaces, it is better if they are divided into some groups depending on their relationship.
- So these classes and interfaces are stored in some directory.
- This directory or folder is also known as package.



Advantages of packages:

- packages hide the classes and interfaces in a separate sub directory, so accidental deletion of classes and interfaces will not take place.
- Two classes in two different packages can have the same name.
- A group of packages is called a library. The reusability nature of packages makes programming easy.

There are two types of packages.

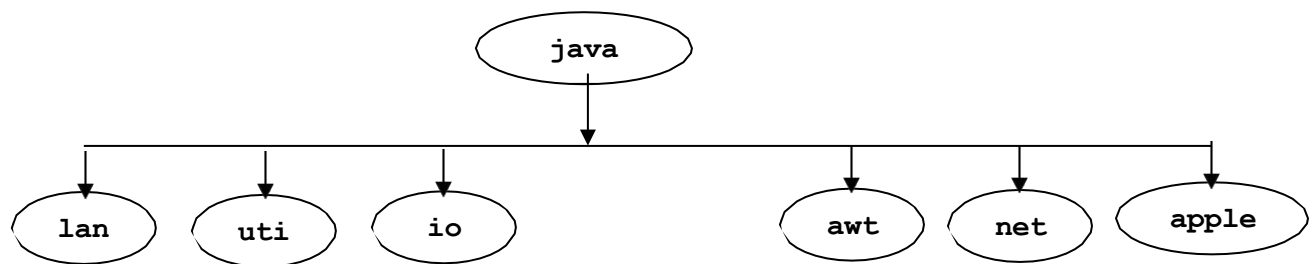
Built-in packages/Java API packages.

User Defined Packages. Java

API Packages:

Java API provides a large number of classes grouped into different packages according to functionality.

Packages from java API are:



Package name	Contents
java.lang	lang package contains language support classes. These classes are imported by java compiler automatically for its usage. lang package contains classes for primitive types, string, threads, exceptions, etc.....
java.util	Utility classes such as vectors, hash tables, etc.....
java.io	Contains classes for input/output of data.
java.awt	Contains for graphical user interface, classes for windows, buttons, lists, menus, etc.....
java.net	Classes for networking, contains classes for communicating local computers and internet servers.
Java.applet	Classes for creating and implementing applets

DEFINING PACKAGE

- A package is created using keyword package
 - Syntax:
package <package>;
- Package statement must be the first statement in a java source file.
 - Ex:
package studentpack; // package declaration
class Student // class declaration
{

}

Create a package which contains Addition class in that.

```
package pack;  
public class Addition  
{  
    private double a,b;  
    public Addition(double a,double b)  
    {  
        this.a=a;  
        this.b=b;  
    }  
    public void sum()  
    {  
        System.out.println("Sum="+a+b);  
    }  
}
```

```
}  
}
```

Compiling the program:

> javac -d . Addition.java

- -d tells java compiler to create a separate sub directory and place the .class file there.
- Dot (.) indicates that the package should be created in the current directory.

Program to use Addition.class from mypack:

```
class UsePack  
{  
    public static void main(String args[])  
    {  
        pack.Addition obj=new pack.Addition(10,20.5);  
        obj.sum();  
    }  
}
```

Note:

- Instead of referring the package name every time, we can import the package like this
 - import pack.Addition;
- Then the program can be written asS....

```
import pack.Addition;  
class UsePack  
{  
    public static void main(String args[])  
    {  
        Addition obj=new Addition(10,20.5);  
        obj.sum();  
    }  
}
```

IMPORTING PACKAGES AND CLASSES INTO PROGRAMS

There are two ways of accessing classes stored in a package.

1. Fully qualified class name
2. Using Import Statement

1. Fully qualified class name:

```
double y=java.lang.Math.sqrt(x);
```

2. Using Import Statement:

syntax:-

```
import packagename.classname;
```

or

```
import packagename.*;
```

```
import java.awt.Font;
```

- imports Font class in our program.

(or)

```
import java.awt.*;
```

- imports all classes of awt package in our program.

PATH AND CLASS PATH

PATH	CLASSPATH
PATH is an environment variable.	CLASSPATH is an environment variable that tells the java compiler where to look for class files to import. Generally CLASSPATH is set to a directory or JAR file.
It is used by the operating system to find the executable files (.exe).	It is used by Application Class Loader to locate the .class file.

You are required to include the directory which contains .exe files.	You are required to include all the directories which contain .class and JAR files.
PATH environment variable once set, cannot be overridden.	The CLASSPATH environment variable can be overridden by using the command line option -cp or -CLASSPATH to both javac and java command.

How to Set CLASSPATH in Windows Using Command Prompt

CLASSPATH is an environment variable that tells the java compiler where to look for class files to import. Generally CLASSPATH is set to a directory or JAR file.

Type the following command in your Command Prompt and press enter.

set CLASSPATH=%CLASSPATH%;C:\Program Files\Java\jre1.8\rt.jar;

In the above command, the set is an internal DOS command that allows the user to change the variable value. CLASSPATH is a variable name. The variable enclosed in percentage sign (%) is an existing environment variable. The semicolon is a separator, and after the (;) there is the PATH of rt.jar file.

ACCESS CONTROL:

Access modifier Access Location	Private	Public	Default/ Nomodifier/ Friendly	protected	Private Protected
Same package Same class	Yes	Yes	Yes	Yes	Yes
same package subclass	No	Yes	yes	Yes	Yes
Same package Non-subclass	No	Yes	Yes	Yes	No
Different package Subclass	No	Yes	No	Yes	Yes
Different package Non-subclass	No	Yes	No	No	no

- This table only applies to members of classes
- A class has only two access levels

Public – accessible any where

Default – accessible with in the same package.

PACKAGES IN JAVA

Java Standard Edition provides 14 packages namely –

- **applet** – This package provides classes and methods to create and communicate with the applets.
- **awt**– This package provides classes and methods to create user interfaces.
- **io**– This package contains classes and methods to read and write data standard input and output devices, streams and files.
- **lang**– This package contains the fundamental classes, methods, and, interfaces of Java language.
- **math**– This package contains classes and methods which helps you to perform arithmetic operations using the Java language.
- **net**– This package provides classes to implement networking applications.
- **rmi**– This package provides classes, methods, and interfaces for Remote Method Invocation.
- **security**– This package provides classes and interfaces for security framework.

- **sql**– This package provides classes and methods to access and process the data stored in a data source.
- **text**– This package provides classes and interfaces to handle text.
- **time**– This package provides API for dates, times, instants, and durations.
- **util**– This package contains collection framework, collection classes, classes related to date and time, event model, internationalization, and miscellaneous utility classes.

JAVA. LANG PACKAGE AND ITS CLASSES:

The most important classes of lang are

- Object, which is the root of the class hierarchy, and Class, instances of which represent classes at runtime.
 protected Object clone()
 boolean equals(Object obj)
 protected void finalize()
 Class getClass()
 int hashCode()
 void notify()
 void notifyAll()
 void wait()
 String toString()
- The wrapper classes
 Boolean
 Character
 Integer
 Short
 Byte
 Long
 Float
 Double
- The classes String, StringBuffer, and StringBuilder similarly provide commonly used operations on character strings.
- Class Throwable encompasses objects that may be thrown by the throw statement. Subclasses of Throwable represent errors and exceptions.

ENUMERATION IN JAVA:

class EnumExample

```
{
    //defining enum within class
    public enum Season { WINTER, SPRING,
        SUMMER, FALL } public static void main(String[]
        args)
    {
        //printing all enum
        for (Season s : Season.values())
        {
            System.out.println(s);
        }
        System.out.println("Value of WINTER is: "+Season.valueOf("WINTER"));
        System.out.println("Index of WINTER is:
        "+Season.valueOf("WINTER").ordinal()); System.out.println("Index of
        SUMMER is: "+Season.valueOf("SUMMER").ordinal());
    }
}
```

CLASS MATH:

class MathClass

```

{
    public static void main(String args[])
    {
        System.out.println("Absolute value-"+Math.abs(-90));
        System.out.println("Minimum value-"+Math.min(90,20));
        System.out.println("Maximum value-"+Math.max(90,20));
        System.out.println("round value-"+Math.round(79.52));
        System.out.println("sqrtroot value-"+Math.sqrt(25));
        System.out.println("cuberoot value-"+Math.cbrt(125));
        System.out.println("power value-"+Math.pow(2,5));
        System.out.println("ceil value-"+Math.ceil(2.2));
        System.out.println("ceil value-"+Math.floor(2.8));
        System.out.println("floorDiv value-"+Math.floorDiv(25,3));
        System.out.println("random value-"+Math.random());
        System.out.println("rint value-"+Math.rint(81.68));
        System.out.println("subtractExact value-"+Math.subtractExact(732,
190));
        System.out.println("multiplyExact value-"+Math.multiplyExact(732, 190));
        System.out.println("incrementExact value-"+Math.incrementExact(732));
        System.out.println("decrementExact value-"+Math.decrementExact(732));
        System.out.println("negateExact value-"+Math.negateExact(90));
    }
}

```

WRAPPER CLASSES:

Primitive data types can be converted into object types by using the wrapper classes contained in java.lang package.

Below table shows the simple data types and their corresponding wrapper classes:

<u>Simple type</u>	<u>Wrapper class</u>
Int	Integer
Char	Character
Float	Float
Double	Double
Long	Long
Boolean	Boolean

The wrapper classes have a number of unique methods for handling primitive data types and objects.

Converting primitive type to object type using Constructor methods:

<u>Constructor</u>	<u>Conversion Action</u>
Integer x=new Integer(i)	Primitive integer to Integer object
Float x=new Float(f)	Primitive float to Float object
Double x=new Double(d)	Primitibe double to Double object
Long x=new Long(l)	Primitive long to Long object

Converting object numbers to Primitive numbers using typeValue() method:

<u>Method</u>	<u>Conversion Action</u>
int i=x.intValue()	Object to primitive integer
float f=x.floatValue()	Object to primitive float
long l=x.longValue()	Object to primitive long
double d=x.doubleValue()	Object to primitive double

Converting numbers to string using toString() method:

<u>Method</u>	<u>Conversion Action</u>
str=Integer.toString(i)	primitive integer to String
str=Float.toString(f)	Primitive float to String
str=Double.toString(d)	Primitive double to String
str=Long.toString(l)	Primitive long to String

Converting string Objects to Numeric objects using static method valueOf().

<u>Method</u>	<u>Conversion Action</u>
X=Double.valueOf(str)	Converts string to Double object
X=Float.valueOf(str)	Converts string to Float object
X=Integer.valueOf(str)	Converts string to Integer Object
X=Long.valueOf(str)	Converts string to Long object

Converting Numeric string to primitive type using parsing methods:

<u>Method</u>	<u>Conversion Action</u>
int i=parseInt(Str)	Converts string to integer
long l=parseLong(str)	Converts string to long

Ex:-

A simple integer can be changed to object type:

```
int x=100;
```

```
Integer ob1=new Integer(x);
```

Now ob1 is an integer object which contains the 100 as its value.

Object can be converted into primitive type:

```
int x=ob1.intValue();
```

now x will contain the value 100.

Byte class:

The following are the Byte class constructs

Byte(byte N);

Byte(String str);

Ex:

Byte b1=new Byte(100);

Byte b2=new Byte("Hello");

Note:

Byte class constructor accepts number as well as string as its parameters.

The following are the methods of Byte class:

byte byteValue() – returns byte value of the object.

int compareTo(Byte b) – it accepts byte object and compares with invoking object.

static byte parseByte(String str) throws NumberFormatException

- it accepts string object and converts into byte. It is static so it can be called directly using the class name.

static Byte valueOf(String str) throws NumberFormatException

- it accepts a String object and converts into Byte class object.

Integer class:

The following are the Integer class constructs

Integer(int N);

Integer(String str) throws NumberFormatException

The following are the methods of Integer class:

int intValue() – To get integer from Integer object.

int compareTo() -

AUTO-BOXING AND AUTO-UNBOXING:

The automatic conversion of primitive data types into its equivalent Wrapper type is known as boxing and opposite operation is known as unboxing. This is the new feature of Java5. So java programmer doesn't need to write the conversion code.

Advantage of Autoboxing and Unboxing:

No need of conversion between primitives and Wrappers manually so less coding is required.

AutoBoxing Example:

```
class BoxingExample1
{
    public static void main(String args[])
    {
        int a=50;
        Integer a2=new Integer(a);//Boxing
        Integer a3=5;//Boxing
        System.out.println(a2+" "+a3);
    }
}
```

UnBoxing Example

```
class UnboxingExample1
{
    public static void main(String args[])
    {
        Integer i=new Integer(50);
        int a=i;
        System.out.println(a);
    }
}
```

```
}  
}
```

JAVA UTIL CLASSES AND INTERFACES:

Classes of util package:

- ArrayDeque
- **ArrayList**
- **Arrays**
- BitSet
- Calendar
- **Collections**
- Currency
- Date
- Dictionary
- EnumMap
- EnumSet
- **Formatter**
- GregorianCalendar
- HashMap
- HashSet
- Hashtable
- IdentityHashMap
- LinkedHashMap
- LinkedHashSet
- LinkedList
- ListResourceBundle
- Locale
- Observable
- PriorityQueue
- Properties
- PropertyPermission
- PropertyResourceBundle
- **Random**
- ResourceBundle
- ResourceBundle.Control
- Scanner
- ServiceLoader
- SimpleTimeZone
- Stack
- StringTokenizer
- Timer
- TimerTask
- TimeZone
- TreeMap
- TreeSet
- UUID
- Vector
- WeakHashMap

Interfaces of util package

- Collection<E>
- Comparator<T>
- Deque<E>
- Enumeration<E>
- EventListener
- Formattable
- Iterator<E>
- List<E>
- ListIterator<E>
- Map<K,V>
- Map.Entry<K,V>
- NavigableMap<K,V>
- NavigableSet<E>
- Observer
- Queue<E>
- RandomAccess()
- Set<E>
- SortedMap<K,V>
- SortedSet<E>

FORMATTER CLASS:

The java.util.Formatter class provides support for layout justification and alignment, common formats for numeric, string, and date/time data, and locale-specific output.

Once the Formatter object is created, it may be used in many ways. The format specifier specifies the way the data is formatted.

A few common format specifiers are:

- **%S or %s**: Specifies String
- **%X or %x**: Specifies hexadecimal integer
- **%o**: Specifies Octal integer
- **%d**: Specifies Decimal integer
- **%c**: Specifies character
- **%T or %t**: Specifies Time and date
- **%n**: Inserts newline character
- **%B or %b**: Specifies Boolean
- **%A or %a**: Specifies floating point hexadecimal
- **%f**: Specifies Decimal floating point

Example:

```
Formatter f=new Formatter();
f.format("%3$s %2$s %1$s", "fear","strengthen", "weakness");
System.out.println(f);
```

RANDOM CLASS:

```
//all the functions of Random class will generate Pseudo random numbers
import java.util.Random;
public class Test
{
    public static void main(String[] args)
    {
        Random random = new Random();
        System.out.println(random.nextInt(10));
    }
}
```

```

        System.out.println(random.nextBoolean());
        System.out.println(random.nextDouble());
        System.out.println(random.nextFloat());
        System.out.println(random.nextGaussian());
    }
}
Time Package
Class Instant (java. time. Instant)

```

FORMATTING FOR DATE/TIME IN JAVA:

```

//Formatting for Date/Time in Java
import java.text.SimpleDateFormat;
import java.util.Date;
class SimpleDateFormatExample2
{
    public static void main(String[] args)
    {
        Date date = new Date();
        SimpleDateFormat formatter = new SimpleDateFormat("MM/dd/yyyy");
        String strDate = formatter.format(date);
        System.out.println("Date Format with MM/dd/yyyy : "+strDate);

        formatter = new SimpleDateFormat("dd-M-yyyy hh:mm:ss");
        strDate = formatter.format(date);
        System.out.println("Date Format with dd-M-yyyy hh:mm:ss : "+strDate);

        formatter = new SimpleDateFormat("dd MMMM yyyy");
        strDate = formatter.format(date);
        System.out.println("Date Format with dd MMMM yyyy : "+strDate);

        formatter = new SimpleDateFormat("dd MMMM yyyy zzzz");
        strDate = formatter.format(date);
        System.out.println("Date Format with dd MMMM yyyy zzzz : "+strDate);
        formatter = new SimpleDateFormat("E, dd MMM yyyy HH:mm:ss z");
        strDate = formatter.format(date);
        System.out.println("Date Format with E, dd MMM yyyy HH:mm:ss z
        :"+strDate);
    }
}

```

```

    }
}
TEMPORAL ADJUSTERS CLASS:

```

```

import
java.time.DayOfWeek;
import java.time.LocalDate;
import java.time.temporal.TemporalAdjusters;
public class TemporalAdjusterExample
{
    public static void main(String[] args)
    {
        LocalDate now = LocalDate.now();
        System.out.println("Current date : " + now);

        LocalDate output = null;

        output =
        now.with(TemporalAdjusters.firstDayOfMonth());
        System.out.println("firstDayOfMonth :: " + output);
        output = now.with(TemporalAdjusters.firstDayOfNextMonth());
        System.out.println("firstDayOfNextMonth :: " + output);

        output = now.with(TemporalAdjusters.firstDayOfNextYear());
        System.out.println("firstDayOfNextYear :: " + output);

        output =
        now.with(TemporalAdjusters.firstDayOfYear());
        System.out.println("firstDayOfYear :: " + output);
        output =
        now.with(TemporalAdjusters.lastDayOfYear());
        System.out.println("lastDayOfY ear :: " + output);

        output = now.with(TemporalAdjusters.dayOfWeekInMonth(2, DayOfWeek.FRIDAY));
        System.out.println("dayOfWeekInMonth(1, DayOfWeek.FRIDAY) :: " + output);

        output =
        now.with(TemporalAdjusters.lastDayOfMonth());
        System.out.println("lastDayOfMonth :: " + output);
        output =
        now.with(TemporalAdjusters.lastDayOfYear());
        System.out.println("lastDayOfY ear :: " + output);
    }
}

```

EXCEPTION HANDLING:

INTRODUCTION

- An exception is an abnormal condition that arises in a code sequence at run time. Exception is a run time error.
- Java and other programming languages have mechanisms for handling exceptions that you can use to keep your program from crashing. In Java, this is known as *catching an exception/exception handling*.
- When java interpreter encounters an error, it creates an exception object and

throws it(informs us that an error occurred).

- If the exception object is not caught and handled properly, the interpreter will display a message and stops the program execution.
- If we want the program to continue with the execution of the remaining code, then we should try to catch the exception object thrown by the error condition and then display an appropriate message for taking corrective actions.

Errors are broadly classified into two categories.

1. Compile time exception(error)
2. Run Time exception(error)

Compile-time errors:

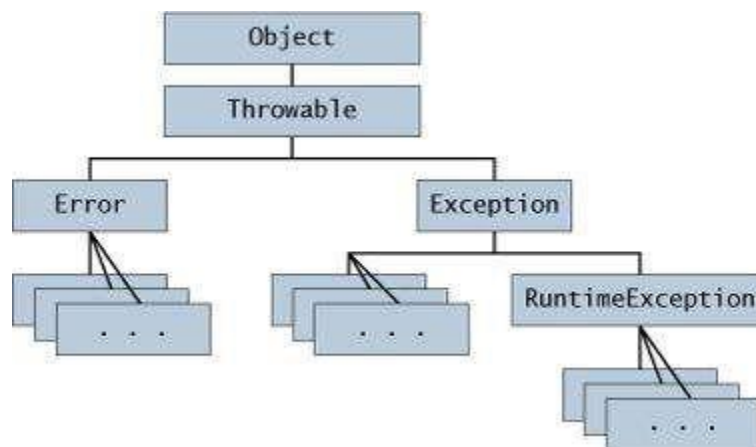
- All syntax errors will be detected and displayed by the java compiler and therefore these errors are known as compile-time errors.
- Whenever the compiler displays an error, it will not create the .class file.

Run-time errors:

- Sometimes a program may compile successfully creating the .class file but may not run properly because of abnormal conditions.
- Common run-time errors are
 - The file you try to open may not exist.
 - Dividing an integer by Zero.
 - Accessing an element that is out of the bonds of an array type.
 - Trying to store a value into an array of an incompatiable class or type.
 - Trying to cast an instance of a class to one of its subclasses.
 - And many more.....

Each exception is a class, part of java.lang package and it is derived from Throwable class.

HIERARCHY OF STANDARD EXCEPTION CLASSES:



There are two types of exceptions in Java:

- **Unchecked Exceptions**
- **Checked Exceptions**

UNCHECKED EXCEPTIONS:

- Unchecked exceptions are RuntimeExceptions and any of its subclasses.
- Error class and its subclasses are also called unchecked exceptions.
- Compiler does not force the program to catch the exception or declare in a throws clause.
 - Ex: ArithMeticException
- Unchecked exceptions can occur anywhere in a program and in a typical program

can be very numerous.

CHECKED EXCEPTIONS:

- A checked exception is any subclass of Exception (or Exception class itself), excluding class RuntimeException and its subclasses.
- Checked exceptions must be handled by the programmer to avoid a compile-time error.
- There are two ways to handle checked exceptions.
Declare the exception using a throws clause.
catch the exception.

The compiler requires a [throws clause](#) or a [try-catch statement](#) for any call to a method that may cause a checked exception to occur.

- Checked Exceptions are checked at compile time, where as unchecked exceptions are at runtime.

We can know whether the exception is Checked or Unchecked in two ways:

- Using Java API
- By Experience

java.io

Class IOException

```
java.lang.Object
|
+--java.lang.Throwable
|
+--java.lang.Exception
|
+--java.io.IOException
```

doesn't extend
RuntimeException
means checked exception

java.lang

Class NumberFormatException

```
java.lang.Object
|
+--java.lang.Throwable
|
+--java.lang.Exception
|
+--java.lang.RuntimeException
|
+--java.lang.IllegalArgumentException
|
+--java.lang.NumberFormatException
```

extends RuntimeException
means unchecked exception

Some common exceptions Unchecked (Runtime Exceptions):

Exception type	Cause of exception
Arithmetic Exception	Caused by math errors such as division by zero
ArrayIndexOutOfBoundsException	Caused by bad array indexes
ArrayStoreException	Caused when a program tries to store the wrong type of data in an array
ClassCastException	Invalid cast
NullPointerException	Invalid use of a null reference

NumberFormatException	Invalid conversion of a string to a numeric format
StringIndexOutOfBoundsException	Attempt to index outside the bounds of a string

Some common exceptions Checked (Compiletime Exceptions):

Exception type	Cause of exception
ClassNotFoundException	Class not found
IllegalAccessException	Access to a class denied
InstantiationException	Attempt to create an object of an abstract class or interface
NoSuchFieldException	A requested field does not exist
NoSuchMethodException	A requested method does not exist

JAVA EXCEPTION HANDLING IS MANAGED BY 5 KEYWORDS:

1. try
2. catch
3. throw
4. throws
5. finally

If these abnormal conditions are not handled properly, either the program will be aborted or the incorrect result will be carried on causing more and more abnormal conditions.

Example to see what happens if the exceptions are not handled: class NoException
{

```

    public static void main(String[] args)
    {
        int no=10; int
        r=0; r=no/0;
        System.out.println("Result is:"+r);
    }
}
```

The above program gives the following error:

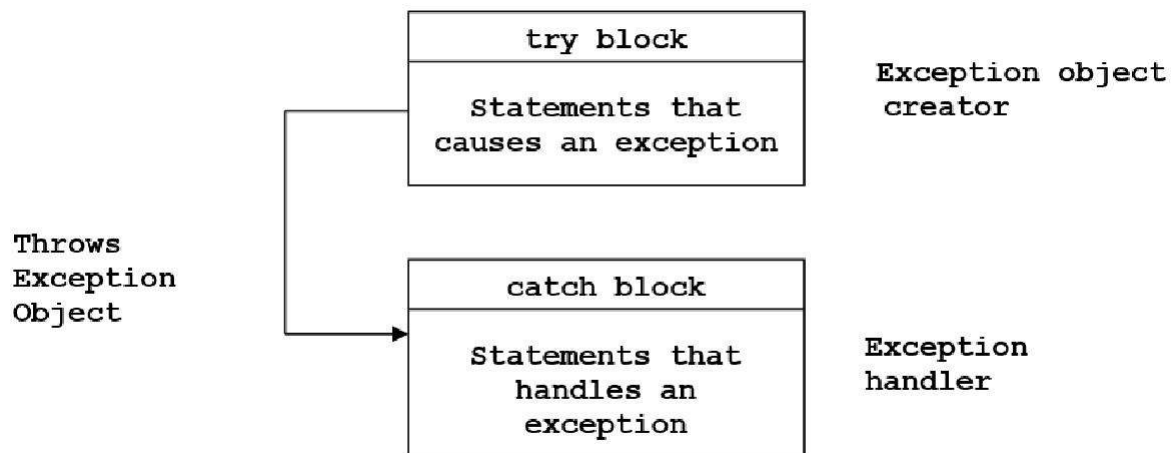
Exception in thread "main" java.lang.ArithmeticException: / by zero at
NoException.main(NoException.java:8)

In the above program the error is handled by the default exception handler which is provided by java run-time system.

If the exception is handled by the user then it gives two benefits.

1. **Fixes the error.**
2. **Avoids automatic termination of program.**

Syntax for exception handling code:



```
try
{
    statement;    // generates exception
}
catch(Exception-type e)
{
    statement;    // process the exception
}
```

- Java uses a keyword try to preface a block of code that is likely to cause an error and “throw” an exception.
- A catch block is defined by the keyword catch “catches” the exception “thrown” by the exception block.
- Catch block should be immediately after the try block.
- try block can have one or more statements that could generate an exception. If any one statement generates an exception, the remaining statements in the block are skipped and control jumps to the catch block that is placed next to the try block.
- The catch block too can have one or more statements that are necessary to process the exception.
- Every try block should be followed by at least one catch statement.
- Catch statement works like a method definition, contains one parameter, which is reference to the exception object thrown by the try block.

Program:

Class Trycatch

```

{
    public static void main(String[] args)
    {
        Int a=10; int b=5; int c=5; int r;
try
    {
        r=a/(b-c); // exception here
        System.out.println("This will not be executed.....");
    }
    catch(ArithmeticException e)
    { System.out.println("Cannot divide by 0....."); }
    System.out.println("After catch statement.....");
}
}

```

Note: Once an exception is thrown, control is transferred to catch block and never returns to try block again.

***Statements after the exception statement(try block) never get executed

MULTIPLE CATCH CLAUSES:

Syntax:

```

try
{
    statement;
}
catch(Exception-type 1 e)
{
    statement; // process exception type 1
}
catch(Exception-type 2 e)
{
    statement; // process exception type 2
}
catch(Exception-type N e)
{statement; // process exception type N
}

```

- When an exception in a try block is generated, java treats the multiple catch statements like cases in switch statement.
- The first statement whose parameter matches with the exception object will be executed, and the remaining statements will be skipped.
- Code in the catch block is not compulsory.
 catch (Exception e){}
 the catch statement simply ends with a curly braces({}), which does nothing, this statement will catch an exception and then ignore it.

```
import java.util.*;
class MultiCatch
{
    public static void main(String args[])
    {
        int no,d,r;
        no=d=r=0;

        try
        {
            no=Integer.parseInt(args[0]);
            d=Integer.parseInt(args[1]);
            r=no/d;
            System.out.println("Result:"+r);
        }
        catch (NumberFormatException nf)
        {
            System.out.println(nf);
        }
        catch (ArrayIndexOutOfBoundsException ai)
        {
            System.out.println(ai);
        }
        catch (ArithmeticException ae)
        {
            System.out.println(ae);
        }
    }
}
```

try-with-resources:

In Java, the try-with-resources statement is a try statement that declares one or more resources. The resource is as an object that must be closed after finishing the program. The try- with-resources statement ensures that each resource is closed at the end of the statement execution.

Syntax:

```
try(resource code)
```

```

{
    use resources}
catch()
{
    handle exceptions
}
Example:
import
java.io.*;
class
TryWithRes
{
    public static void main(String args[])
    {

        try (BufferedReader br=new BufferedReader(new FileReader("Sample.java")))
        {
            String line; while((line=br.readLine())!=null)
            {
                System.out.println(line);
            }
        }
        catch(IOException e)
        {
            System.out.println(e);
        }
    }
}

```

Custom Exceptions:

we can create our own exceptions that are derived classes of the Exception class. Creating our own Exception is known as custom exception or user-defined exception. Basically, Java custom exceptions are used to customize the exception according to user need.

Example:

```

public class WrongFileNameException extends Exception
{ public WrongFileNameException(String errorMessage)
    { super(errorMessage);
    }
}

```

NESTED TRY AND CATCH BLOCKS:

In Java, using a try block inside another try block is permitted. It is called as nested try block.

Syntax:

```

....
//main try block
try
{
    statement 1;
    statement 2;
    //try catch block within another try block
    try

```

```

    {
        statement 3;
        statement 4;
    //try catch block within nested try block
        try
        {
            statement 5;
            statement 6;
        }
        catch(Exception e2)
        {
            //exception message
        }

    }
    catch(Exception e1)
    {
        //exception message
    }
}
//catch block of parent (outer) try block
catch(Exception e3)
{
    //exception message
}
....

```

Example:

```

class NestedExcep
{
    public static void main(String args[])
    {
        try
        {
            try
            {
                System.out.println("going to divide");
                int b =39/0;
            }
            catch(ArithmeticException e)
            {
                System.out.println("A number can't be divide with zero");
            }

            try
            {
                int a[]=new int[5];
                a[5]=4;
            }
            catch(ArrayIndexOutOfBoundsException e)
            {
                System.out.println(e);
            }
        }
    }
}

```

```

        }

        }
        catch(Exception e)
        {
            System.out.println("handeled");
        }
    }
}

```

THROWS CLAUSE OR RETHROWING EXCEPTION

```

import
java.io.*;
class Test
{
    void doWork() throws IOException
    {
        throw new IOException();
    }
}

class ThrowsTest1
{
    public static void main(String args[]) throws IOException
    {
        Test t1=new
        Test();
        t1.doWork();
    }
}

```


UNIT V

String Handling in Java: Introduction, Interface Char Sequence, Class String, Methods for Extracting Characters from Strings, Comparison, Modifying, Searching; Class String Buffer.

Multithreaded Programming: Introduction, Need for Multiple Threads Multithreaded Programming for Multi-core Processor, Thread Class, Main Thread-Creation of New Threads, Thread States, Thread Priority-Synchronization, Deadlock and Race Situations, Inter-thread Communication - Suspending, Resuming, and Stopping of Threads.

Java Database Connectivity: Introduction, JDBC Architecture, Installing MySQL and MySQL Connector/J, JDBC Environment Setup, Establishing JDBC Database Connections, ResultSet Interface

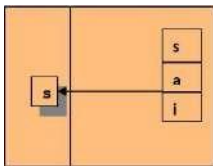
Java FX GUI: Java FX Scene Builder, Java FX App Window Structure, displaying text and image, event handling, laying out nodes in scene graph, mouse events

STRING HANDLING IN JAVA:

String: String is a group of characters surrounded by double quotes, in java String class is used to represent these group of characters.

Ex:

```
String s="sai"
```



1. Address of the object is assigned to the variable s. This object will be like a character array.
2. Once a string object is created, the data inside the object cannot be modified, that's why we say strings are immutable.

Different ways of creating String objects:

```
String s1=new
```

```
String(); String s2=new
```

```
String("sai"); String
```

```
s3="sai";
```

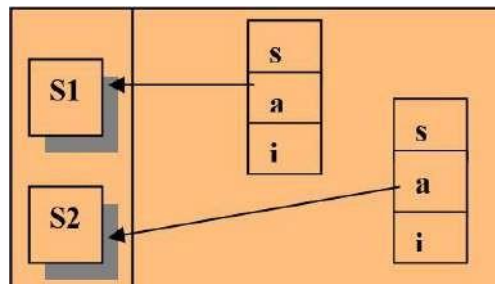
Difference between new and " "

1. Using new keyword any number of objects are created for the same class.

```
String s1=new String("sai");
```

```
String s2=new String("sai");
```

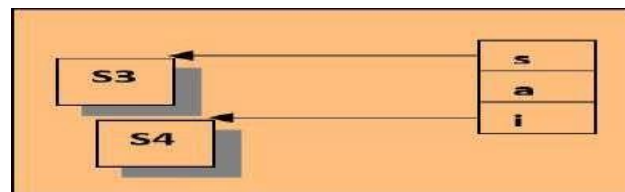
The above statements will create two different objects, with different address(same contents) assigned to different variable s1 and s2 respectively.



2. Assignment operator is used to assign the string to the variable s3. in this case, JVM first of all checks whether the same object is already available in the string constant pool or not. If it is available, then it creates another reference to it. If same object is not available, then it creates another object with the with the content “PREM” and stores it into the string constant pool.

```
String s3="sai";
```

```
String s4="sai";
```

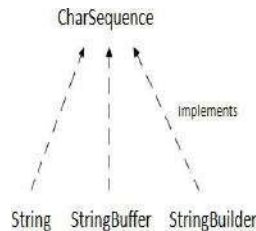


```
class StringDiff
{
    public static void main(String args[])
    {
        String s1=new String("sai");
        String s2=new String("sai");
        String s3="sai";
        String s4="sai";
        if(s1==s2)
            System.out.println("Equal..");
        else
            System.out.println("NOT Equal..");
        if(s3==s4)
            System.out.println("Equal..");
        else
            System.out.println("NOT Equal..");
    }
}
```

Note: The java. lang. String class implements *Serializable*, *Comparable* and *Char Sequence* interfaces.

INTERFACE CHAR SEQUENCE:

The CharSequence interface is used to represent the sequence of characters. String, StringBuffer and StringBuilder classes implement it. It means, we can create strings in Java by using these three classes.



The Java String is immutable which means it cannot be changed. Whenever we change any string, a new instance is created. For mutable strings, you can use StringBuffer and StringBuilder classes.

CLASS STRING:

The java. lang. String class provides many useful methods to perform operations on sequence of char values.

No.	Method	Description
1	char charAt(int index)	returns char value for the particular index
2	int length()	returns string length
3	static String format(String format, Object... args)	returns a formatted string.
4	static String format(Locale l, String format, Object... args)	returns formatted string with given locale.
5	String substring(int beginIndex)	returns substring for given begin index.
6	String substring(int beginIndex, int endIndex)	returns substring for given begin index and end index.
7	boolean contains(CharSequence s)	returns true or false after matching the sequence of char value.
8	static String join(CharSequence delimiter, CharSequence... elements)	returns a joined string.
9	boolean equals(Object another)	checks the equality of string with the given object.
10	boolean isEmpty()	checks if string is empty.
11	String concat(String str)	concatenates the specified string.
12	String replace(char old, char new)	replaces all occurrences of the specified char value.

13	String replace(CharSequence old, CharSequence new)	replaces all occurrences of the specified CharSequence.
14	static String equalsIgnoreCase(String another)	compares another string. It doesn't check case.
15	String[] split(String regex)	returns a split string matching regex.
16	int indexOf(int ch)	returns the specified char value index.
17	int indexOf(int ch, int fromIndex)	returns the specified char value index starting with given index.
18	int indexOf(String substring)	returns the specified substring index.
19	int indexOf(String substring, int fromIndex)	returns the specified substring index starting with given index.
20	String toLowerCase()	returns a string in lowercase.
21	String toLowerCase(Locale l)	returns a string in lowercase using specified locale.
22	String toUpperCase()	returns a string in uppercase.
23	String toUpperCase(Locale l)	returns a string in uppercase using specified locale.
24	String trim()	removes beginning and ending spaces of this string.
25	static String valueOf(int value)	converts given type into string. It is an overloaded method.

String concatenation:

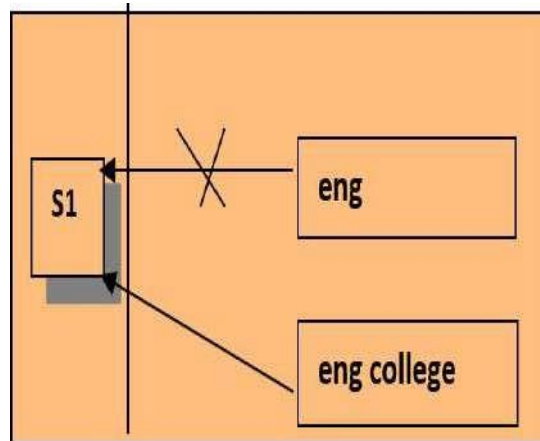
Appending a string to another String is known as String concatenation. The + symbol acts as concatenation operator.

```
class SCat
{
    public static void main (String args[])
    {
        String s1="eng"; s1=s1+"college";
        System.out.println(s1);
    }
}
```

We said that String objects immutable(contents of the object are not changed once created).

s1=s1+" Computers";

when this statement is executed a new string is created with the contents of LHS and RHS of the + operator. And the old object will eligible for garbage collection.



String functions:

```
class StringClass
```

```
{
```

```
    public static void main(String args[])
```

```
    {
```

```
        String name="ADITYA";
```

```
        String name1="college";
```

```
        String name3="";
```

```
        String name4="Aditya College of Engineering and Technology";
```

```
        String name5="    Aditya College of Engineering and Technology    ";
```

```
        char ch=name.charAt(4);//returns the char value at the 4th index
```

```
        System.out.println(ch);
```

```
        System.out.println("string length is: "+name.length());
```

```
        System.out.println("sub string example: "+name.substring(2,4));
```

```
        System.out.println("contains example: "+name.contains("IT"));
```

```
        System.out.println("join example: "+String.join("-", "02", "07", "2021"));
```

```
        System.out.println("equals example: "+name.equals(name1));
```

```
        System.out.println("concat example: "+name.concat(name1));
```

```
        System.out.println("replace example: "+name.replace('a','A'));
```

```
        System.out.println("isEmpty example: "+name3.isEmpty());
```

```
        System.out.println("isEmpty example: "+name1.isEmpty());
```

```
        System.out.println("name 4 before spliting: "+name4);
```

```

String s[]=name4.split("\\s");
for(String word:s)
    System.out.println(word);

System.out.println("indexof example: "+name.indexOf('t'));
System.out.println("tolowercase example: "+name.toLowerCase());
System.out.println("touppercase example: "+name1.toUpperCase());
System.out.println("name 5 is:-"+name5);
System.out.println("trim example: "+name5.trim());
}
}

```

METHODS FOR EXTRACTING CHARACTERS FROM STRINGS

The following methods are used to extract Characters from Strings

1. charAt()
2. getChars()
3. toCharArray()
4. getBytes()

Example:

```

class Extract
{
    public static void main(String args[])
    {
        String s=new String("Aditya College of Engineering and technology");
        System.out.println("character at 7th index is-"+s.charAt(7));
        System.out.println(s.length());
        char[] des=new char[44]; s.getChars(1,21,des,0);
        System.out.println("Extracted characters-"+des);
        String s1="ABCDEF";
        byte b[]=s1.getBytes(); for(byte a:b) System.out.println(a);

        String s2="Aditya";
        char[] ch=s2.toCharArray();
        for(char c:ch)
            System.out.print(c);
    }
}

```

```
}
```

METHODS FOR COMPARISON OF STRINGS

The following methods are used to compare Strings

1. equals()
2. compareTo()

Example:

```
class Compare
```

```
{  
  
    public static void main(String args[])  
    {  
        String s1="Aditya";  
        String s2="Sai";  
        String s3="aditya";  
        String s4="Aditya";  
        System.out.println(s1.compareTo(s4));  
        System.out.println(s1.compareTo(s2));  
        System.out.println(s2.compareTo(s1));  
        if(s1.equals(s4))  
            System.out.println("Equals"); else  
            System.out.println("Not Equals");  
            if(s1==s4)  
                System.out.println("Equals");  
            else  
                System.out.println("Not Equals");  
    }  
}
```

METHODS FOR MODIFYING STRINGS

The following methods are used to modify Strings

1. concat()
2. replace()
3. trim()
4. substring()

Example:

```
class StringClass
```

```
{  
  
    public static void main(String args[])  
    {  
        String name="ADITYA";
```

```

String name1="college";
String name3="";
String name4="Aditya College of Engineering and Technology";
String name5="    Aditya College of Engineering and Technology    ";
System.out.println("sub string example: "+name.substring(2,4));
System.out.println("concat example: "+name.concat(name1));
System.out.println("replace example: "+name.replace('a','A'));
System.out.println("trim example: "+name5.trim());
}
}

```

METHODS FOR SEARCHING STRINGS

The following methods are used searching Strings

1. indexOf()
2. lastIndexOf()
3. charAt()
4. contains()

Example:

```

class StringClass
{
    public static void main(String args[])
    {
        String name="ADITYA";
        String name1="college";
        String name3="";
        String name4="Aditya College of Engineering and Technology";
        String name5="    Aditya College of Engineering and Technology    ";
        char ch=name.charAt(4);//returns the char value at the 4th index
        System.out.println("contains example: "+name.contains("IT"));
        System.out.println("indexof example: "+name.indexOf('t'));
    }
}

```

DATA CONVERSION AND MISCELLANEOUS METHODS

Example:

```
class StringClass
{
    public static void main(String args[])
    {
        String name="ADITYA";
        String name1="college";
        String name3="";
        String name4="Aditya College of Engineering and Technology";
        String name5="    Aditya College of Engineering and Technology    ";
        char ch=name.charAt(4);//returns the char value at the 4th index
        System.out.println(ch);
        System.out.println("string length is: "+name.length());
    }
}
```

```

System.out.println("sub string example: "+name.substring(2,4));
System.out.println("contains example: "+name.contains("IT"));
System.out.println("join example: "+String.join("-", "02", "07", "2021"));
System.out.println("equals example: "+name.equals(name1));
System.out.println("concat example: "+name.concat(name1));
System.out.println("replace example: "+name.replace('a','A'));
System.out.println("isEmpty example: "+name3.isEmpty());
System.out.println("isEmpty example: "+name1.isEmpty());
System.out.println("name 4 before splitting: "+name4);
String s[]=name4.split("\\s");
for(String word:s)
    System.out.println(word);
System.out.println("indexOf example: "+name.indexOf('t'));
System.out.println("toLowerCase example: "+name.toLowerCase());
System.out.println("toUpperCase example: "+name1.toUpperCase());
System.out.println("name 5 is:-"+name5);
System.out.println("trim example: "+name5.trim());
}
}

```

STRINGBUFFER CLASS:

1. String buffer is a peer class of string. The string buffer class creates mutable strings of flexible length which can be modified in terms of both length and content.
2. StringBuffer reserves space for 16 characters.
3. EnsureCapacity () adds double+2 spaces.
4. The following are methods of StringBuffer Class
 1. length()
 2. capacity()
 3. charAt()
 4. append()
 5. toString()
 6. reverse()
 7. ensureCapacity()
 8. setCharAt()

- 9. delete();
- 10. deleteCharAt();
- 11. replace();
- 12. substring();
- 13. insert()

Example 1:

```
class StringBufferDemo
{
    public static void main(String args[])
    {
        StringBuffer sb=new StringBuffer("Engineering Student");
        System.out.println("Buffer:"+sb);
        System.out.println("Length:"+sb.length());
        System.out.println("Capacity:"+sb.capacity());
        // Displaying all the characters of the string
        for(int i=0;i<s.length();i++)
        {
            System.out.println("Character at position:" +i+" is "+s.charAt(i));
        }
        // Appending at the end
        s.append(" CSE");
        System.out.println("New string is:"+s);
    }
}
```

Example 2:

```
class StringBufferDemo1
{
    public static void main(String args[])
    {
        StringBuffer sb1=new StringBuffer();
        StringBuffer sb2=new
        StringBuffer(10);
```

```

    StringBuffer sb3=new StringBuffer("Prem");
    System.out.println("sb3 toString:"+sb3.toString());
    System.out.println("sb1 length is:"+sb1.length());
    System.out.println("sb2 length is:"+sb2.length());
    System.out.println("sb3 length is:"+sb3.length());
    System.out.println("sb1 capacity is:"+sb1.capacity());
    System.out.println("sb2 capacity is:"+sb2.capacity());
    System.out.println("sb3 capacity is:"+sb3.capacity());
    sb1.ensureCapacity(50);
    System.out.println("sb1 capacity is:"+sb1.capacity());
    StringBuffer sb4=new StringBuffer("hi are u");
    System.out.println("sb4 character is:"+sb4.charAt(0));
    System.out.println("sb4 character is:"+sb4.charAt(4));
    sb4.setCharAt(0,'H');
    sb4.setCharAt(4,'H');
    sb4.reverse();
    System.out.println(sb4);
    sb4.insert(4," Prem");
    String s="Good Bye";
    boolean b=true;
    int i=99;

    double d=99.99;
    StringBuffer sb5=new StringBuffer();
    sb5.append('r');
    sb5.append('a');
    sb5.append('j');
    System.out.println(sb5);
    StringBuffer sb6=new StringBuffer("Raj Jain");
    sb6.insert(4," Kumar ");
    System.out.println(sb6);
    sb6.delete(1,3);
    sb6.deleteCharAt(3);
    sb6.replace(0,4,"Hello");
    String s1=sb6.substring(4);
    String s2=sb6.substring(0,4);
    System.out.println(s1);
    System.out.println(s2);
}

};

```

equals() and ==:

equals() and == performs two different operations. equals() method compares the characters inside a String object.

The == operator compares two object references to see whether they refer to the same instance.

StringBuilder class:

StringBuilder class has been added in JDK1.5 which has same features like

StringBuffer class. StringBuilder class objects are also mutable as the stringBuffer Objects.

Difference:

StringBuffer is class is synchronized and StringBuilder is not.

MULTITHREADED PROGRAMMING:

INTRODUCTION,

NEED FOR MULTIPLE THREADS MULTITHREADED PROGRAMMING FOR MULTI-CORE PROCESSOR

1. Multithreading in Java is a process of executing multiple threads simultaneously.
2. A thread is a lightweight sub-process, the smallest unit of processing. Multiprocessing and multithreading, both are used to achieve multitasking.

Advantages of Java Multithreading

1. It **doesn't block the user** because threads are independent and you can perform multiple operations at the same time.
2. You **can perform many operations together, so it saves time**.
Threads are **independent**, so it doesn't affect other threads if an exception occurs in a single thread.

Multitasking

Multitasking is a process of executing multiple tasks simultaneously. We use multitasking to utilize the CPU. Multitasking can be achieved in two ways:

- Process-based Multitasking (Multiprocessing)
- Thread-based Multitasking (Multithreading)

1) Process-based Multitasking (Multiprocessing)

- Each process has an address in memory. In other words, each process allocates a separate memory area.
- A process is heavyweight.
- Cost of communication between the process is high.
- Switching from one process to another requires some time for saving and loading **registers**, memory maps, updating lists, etc.

2) Thread-based Multitasking (Multithreading)

- Threads share the same address space.
- A thread is lightweight.
- Cost of communication between the thread is low.

THREAD CLASS

Thread class provide constructors and methods to create and perform operations on a thread. Thread class extends Object class and implements Runnable interface.

Commonly used Constructors of Thread class:

- Thread()
- Thread(String name)
- Thread(Runnable r)
- Thread(Runnable r, String name)

Commonly used methods of Thread class:

- **public void run():** is used to perform action for a thread.
- **public void start():** starts the execution of the thread. JVM calls the run() method on the thread.

public void sleep(long milliseconds): Causes the currently executing thread to sleep

- (temporarily cease execution) for the specified number of milliseconds.
- **public void join():** waits for a thread to die.
- **public void join(long milliseconds):** waits for a thread to die for the specified milliseconds.
- **public int getPriority():** returns the priority of the thread.
- **public int setPriority(int priority):** changes the priority of the thread.
- **public String getName():** returns the name of the thread.
- **public void setName(String name):** changes the name of the thread.
- **public Thread currentThread():** returns the reference of currently executing thread.
- **public int getId():** returns the id of the thread.
- **public Thread.State getState():** returns the state of the thread.
- **public boolean isAlive():** tests if the thread is alive.
- **public void yield():** causes the currently executing thread object to temporarily pause and allow other threads to execute.
- **public void suspend():** is used to suspend the thread(deprecated).
- **public void resume():** is used to resume the suspended thread(deprecated).
- **public void stop():** is used to stop the thread(deprecated).
- **public boolean isDaemon():** tests if the thread is a daemon thread.
- **public void setDaemon(boolean b):** marks the thread as daemon or user thread.
- **public void interrupt():** interrupts the thread.
- **public boolean isInterrupted():** tests if the thread has been interrupted.

public static boolean interrupted(): tests if the current thread has been interrupted

MAIN THREAD:

- A Thread represents a separate path of execution.
- Group of statements executed by JVM one by one.

Program to find the thread used by JVM to execute the statements(Main Thread):

class ThreadName

```
{
    public static void main(String args[])
    {
        System.out.println("Welcome ");
        Thread t=Thread.currentThread();
        System.out.println("Current Thread is : "+t);
        System.out.println("Current Thread Name is : "+t.getName());
    }
}
```

In the above program currentThread() is a static method of Thread Class, which returns the reference of the current running thread.

Thread[main,5,main]

- Here Thread indicates that t is thread class object.
- The First main indicates the name of the thread that is executing the current code.

- 5 the is priority of thread.

The next main indicates the thread group name to which this thread belongs.

CREATION OF NEW THREADS

Creating threads in java is simple. Threads are created in the form of objects that contain a method called run(). The run() is heart and soul of any thread. The whole code that constitutes a new thread is written inside the run() method.

Threads can be created by the user in two ways

1. By extending Thread class
2. By implementing Runnable interface

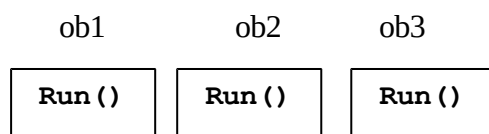
Creating thread by extending Thread class

Syntax:-

```
class <class name> extends Thread
{
    public void run()
    {
        -----

        -----override run() method
        -----

    }
}
```



Note:

- Methods of Thread class are inherited in our own class, so every object created for our class is a thread.
- Every object(thread) has its own start(),run() and many more methods.
- Whenever start() method is called run() is executed.
- The run() method is just like your main() method where you can declare variables, use other classes, and can call other methods.
- The run() is an entry point for another thread. This thread will end when run() returns.

Example:

```

class NewThread1 extends Thread
{
    public void run()
    {
        try
        {
            for(int i=1;i<=5;i++)
            { System.out.println(Thread.currentThread().getName()+" : "+i);
              Thread.sleep(1000);
            }
        }
        catch(InterruptedException e){}
        System.out.println("Child thread exiting... ");
    }
};

class NewThreadDemo1
{
    public static void main(String args[])throws InterruptedException
    {
        NewThread1 t1=new NewThread1();
        t1.setName("Child Thread");
        t1.start();
    }
};

```

BY IMPLEMENTING RUNNABLE INTERFACE:

- We can't extend more than one class to our class because multiple inheritance is not possible, in such situations we can implement runnable interface.
- Runnable implementation is slightly less simple. To run a separate thread, you still need a thread instance.
- To implement Runnable interface, a class need to implement a single method called run().

Syntax: Public void run()

Example:

```

class NewThread3 implements Runnable
{
    public void run()

```

```

{
    try
    {

        for(int i=1;i<=5;i++)
        {
            System.out.println(Thread.currentThread().getName()+" : "+i);
            Thread.sleep(1000);
        }
    }

    catch(InterruptedExceoption e){}
    System.out.println("Child thread exiting... ");
};
}
class NewThreadDemo3
{
    public static void main(String args[])throws InterruptedException
    {
        NewThread3 obj=new NewThread3();
        Thread t1=new Thread(obj);
        t1.setName("Child Thread"); t1.start();
    }
};

```

THREAD PRIORITIES:

When the threads are created and started, a ‘thread scheduler’ program in JVM will load them into memory and execute them. This scheduler will allot more JVM time to those thread which are having priority.

The priority numbers will change from 1 to 10.

Thread.MAX_PRIORITY – 10

Thread.MIN_PRIORITY – 1

Thread.NORM_PRIORITY - 5

Example:

```
class MyClass extends Thread
```

```

{
    int count=0;
    public void
    run()
    {
        for(int
        i=1;i<=10000;i++)
        count++;

```

```

        System.out.println("Completed
        Thread:"+Thread.currentThread().getName()); System.out.println("It's
        Priority:"+Thread.currentThread().getPriority());
    }
}

```

```

class Prior
{
public static void main(String args[])
{

    MyClass m=new MyClass();
    Thread t1=new
    Thread(m,"One"); Thread
    t2=new Thread(m,"Two");
    t1.setPriority(2);
    t2.setPriority(Thread.NORM_PRIORITY);

    t1.start();
    t2.start();
}
}

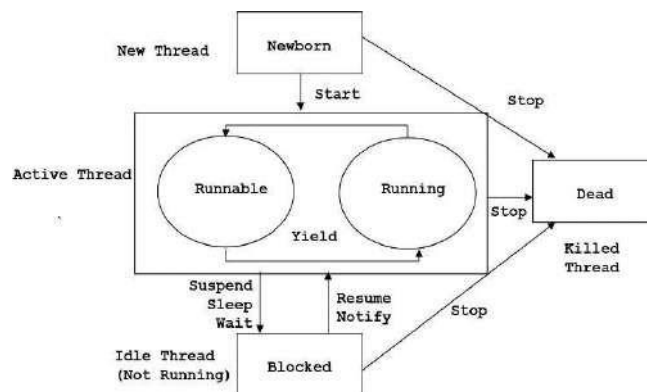
```

THREAD STATES (OR) LIFE CYCLE OF THE THREAD:

During the life time of a thread, there are many states it can enter. They are

1. Newborn State
2. Runnable State
3. Running State
4. Blocked State
5. Dead State

A thread is always in one of these five states. It can move from one state to another state via a variety of ways.

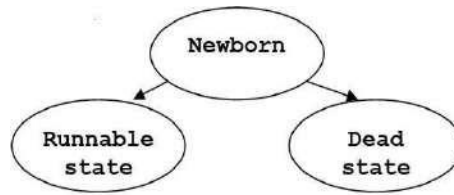


Newborn state:

When a thread object is created it is said to be in newborn state. It is not yet scheduled for running. At this stage we can do only following things.

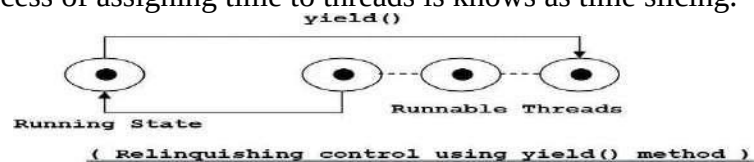
- Schedule it for running using start() method.
- Kill it using Stop() method.

If scheduled, it moves to the runnable state. If any other method is used an exception will be thrown.



Runnable state:

- Runnable state means the thread is ready for execution and is waiting for the availability of the processor. i.e The thread has joined the queue of threads that are waiting for execution.
- If all threads have equal priority, they are given time slots for execution in round robin fashion. i.e first come first serve manner.
- The thread that relinquishes control joins the queue at the end and again waits for its turn. This process of assigning time to threads is known as time slicing.



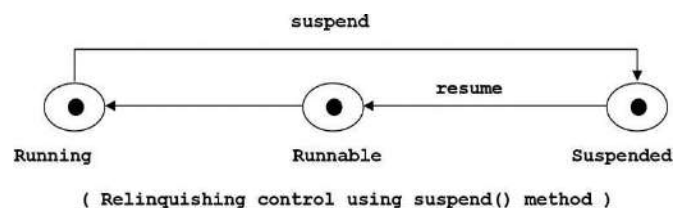
If we want a thread to relinquish control to another thread of equal priority before its turn comes, we can do so by using the yield().

Running State:

- Running means the processor has given its time to the thread for execution.
- **A running thread may relinquish its control in one of the following**

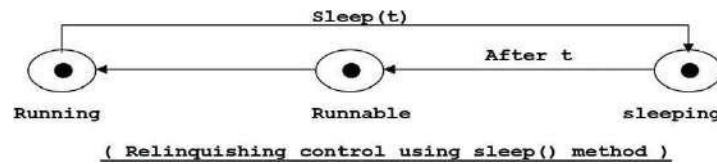
situations. Suspend:

It has been suspended using suspend() method. A suspended thread can be revived using resume() method. This is useful when we want to suspend a thread for some time due to certain reason. But do not want to kill.



Sleep:

It has been made to sleep. We put a thread to sleep for a specified time period using the method sleep(time) where time is in milliseconds. This means that the thread is out of the queue during this time period. It re-enters the runnable state as soon as time period is elapsed.



Wait:

It had been told to wait until some event occurs. This is done using wait() method. The thread can be scheduled to run again using notify() method.

Blocked State:

- A thread is said to be blocked when it is prevented from entering into the Runnable state. Subsequently the running state.

Dead state:

- This happens when the thread is suspended, sleeping or waiting in order to satisfy certain requirements.
- A blocked thread is considered “NOT RUNNABLE” but not dead and therefore fully qualified to run again.
- Every thread has a life cycle.
- A running thread ends its life when it has completed executing its run() method.
- It is a natural death.
- We can kill a thread by sending stop message to it at any state thus causing a premature death.
- A thread can be killed as soon as it is born, or while it is running or even when it is in “NOT RUNNABLE”(blocked) condition.

DAEMON THREADS:

- Threads that work continuously without any interruption to provide services to other threads (works in the background to support the runtime environment) are called *daemon threads*.
Eg: the clock handler thread, the idle thread, the garbage collector thread, the screen updater thread etc.
- By default a thread is not a daemon thread.
 - *setDaemon(true)* turns a thread into daemon thread.
 - *isDaemon()* to know whether a thread is Daemon or not.

SYNCHRONIZATION

- When a thread is already acting on an object, preventing any other thread from acting on the same object is called “Thread synchronization” or “Thread safe”.
- Synchronized object is like a locked object, When a thread enters the object, it locks it, so that the next thread cannot enter till it comes out.
- This means the object is locked mutually on threads, so this object is called mutex(Mutually exclusive lock).

How to synchronize the object:

There are two ways

- Synchronized block
- Synchronized keyword

Synchronized block:

Here we can embed a group of statements of the object(inside run() method) within a synchronized block.

```
synchronized(this)
```

```
{  
    Statements;
```

```
}
```

Synchronized keyword:

We can synchronize an entire method by using synchronized keyword.

```
synchronized void display()
```

```
{  
    Statements;
```

```
}
```

Example for Synchronized block can be executed only by one thread:

```
class Reserve implements Runnable
```

```
{  
    int available=1;  
    int wanted;  
    Reserve(int i)  
    {  
        wanted=i;  
    }  
    public void run()  
    {  
        synchronized(this)  
        {  
            System.out.println("Available Berths= " +available);  
            if(available>=wanted)  
            {
```

```

        String name=Thread.currentThread().getName();
        System.out.println(wanted+" berths reserved for "+name);
        try
        {
            Thread.sleep(1000);
            available=available-wanted;
        }
        catch(InterruptedException ie)
        { ie.printStackTrace(); }
        }
        else
            System.out.println("Sorry no berths....");
    }
}
}
class Sync
{
    public static void main(String args[])
    {
        Reserve obj=new Reserve(1);
        Thread t1=new Thread(obj);
        Thread t2=new Thread(obj);

        t1.setName("First person...");
        t2.setName("Second Person. ..");
        t1.start();
        t2.start();
    }
};

```

DEADLOCK AND RACE SITUATIONS

- Deadlock in Java is a condition where two or more threads are blocked forever, waiting for each other.
- This usually happens when multiple threads need the same locks but obtain them in different orders.
- Multithreaded Programming in Java suffers from the deadlock situation because of the synchronized keyword.

Example:

```

public class Example
{
    public static void main(String[] args)
    {
        final String r1 = "surya";
        final String r2 = "java";
        Thread t1 = new Thread()
        {
            public void run()
            {
                synchronized(r1)

```

```

        {
            System.out.println("Thread 1: Locked r1");
            try
            {
                Thread.sleep(100);
            }
            catch(exception e)
            {
            }
        }
    }
};
Thread t2 = new Thread()
{
    public void run()
    {
        synchronized(r1)
        {
            System.out.println("Thread 2: Locked r1");
            try{ Thread.sleep(100);} catch(exception e) {}
        }
    }
};
t1.start();
t2.start();
}
}

```

INTER-THREAD COMMUNICATION:

In some cases, two or more threads should communicate with each other. For example, a consumer thread is waiting for a producer to produce the data. When the producer completes the production of data, then the consumer thread should take that data and use it.

Methods used for inter -thread communication:

obj.wait() : This method makes a thread wait for the object(obj) till it receives a notification from a notify() or notifyAll() method.

obj.notify() : This method releases an object(obj) and sends a notification to a waiting thread that the object is available.

obj.notifyAll() : This method is useful to send notification to all waiting threads at once that the object is available.

Example:

```

class Communicate1
{
    public static void main(String args[])throws Exception
    {
        Producer obj1=new Producer();
    }
}

```

```

// pass producer object to Consumer
Consumer obj2=new Consumer(obj1);
//Create two threads and attach to producer and consumer
Thread t1=new Thread(obj1);
Thread t2=new Thread(obj2);
t2.start();//consumer waits
t1.start();//producer starts production
}
}
class Producer extends Thread
{
    StringBuffer sb;
    Producer()
    {
        sb=new StringBuffer();
    }
    public void run()
    {
        synchronized(sb)
        {
            for(int i=1;i<=10;i++)
            {
                try{
                    sb.append(i+":");
                    Thread.sleep(100);
                    System.out.println("Appending...");
                }
            }
        }
    }
}

```

```

        catch(Exception e){}
    }
    //Production is over, so it is sending a notification to consumer
    //thread that sb object is
    available sb.notify();
}
} //end of run
} //end of producer class
class Consumer extends Thread
{
    Producer
    prod;
    Consumer(
    Producer p)
    {
        prod=p;
    }
    public void run()
    {
        synchronized(prod.sb)
        {
            // wait till a notification is received from producer thread. Here
            //there is no wastage of time of even a
            single millisecond try{
                prod.sb.wait();
            }
            catch(Exception e){}
            System.out.println(prod.sb);
        } //end of sync
    } //end of run
} //end of consumer class

```

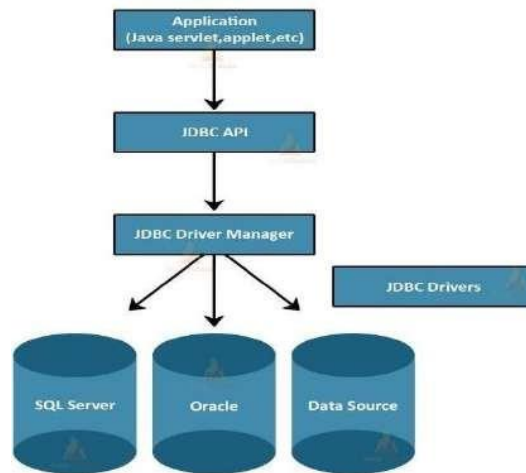
JAVA DATABASE CONNECTIVITY:

INTRODUCTION

1. The term JDBC stands for Java Database Connectivity and it is a specification from Sun microsystems.
2. JDBC is an API (Application programming interface) in Java that helps users to interact or communicate with various databases.
3. The classes and interfaces of JDBC API allow the application to send the request to the specified database.
4. Using JDBC, we can write **programs required to access databases**. JDBC and the database driver are capable of accessing databases and spreadsheets.
5. Interacting with the database requires efficient database connectivity, which we can achieve using ODBC (Open database connectivity) driver. We can use this ODBC Driver with the JDBC to interact or communicate with various kinds of databases, like Oracle, MS Access, MySQL, and SQL, etc.
6. The **java.sql** package contains classes and interfaces for JDBC API. A list of popular *interfaces* of JDBC API are given below:
 - Driver interface
 - Connection interface
 - Statement interface
 - PreparedStatement interface
 - CallableStatement interface
 - ResultSet interface
 - ResultSetMetaData interface
 - DatabaseMetaData interface
 - RowSet interfaceA list of popular classes of JDBC API are given below:
 - DriverManager class
 - Blob class
 - Clob class
 - Types class
7. We can use JDBC API to handle database using Java program and can perform the following activities:
 1. Connect to the database

2. Execute queries and update statements to the database
3. Retrieve the result received from the database.

Architecture of JDBC



CONNECTIVITY WITH MYSQL

To connect Java application with the MySQL database, we need to follow 5 following steps. In this example we are using MySql as the database. So we need to know following informations for the mysql database:

1. **Driver class:** The driver class for the mysql database is **com.mysql.jdbc.Driver**.
2. **Connection URL:** The connection URL for the mysql database is **jdbc:mysql://localhost:3306/aditya** where jdbc is the API, mysql is the database, localhost is the server name on which mysql is running, we may also use IP address, 3306 is the port number and aditya is the database name. We may use any database, in such case, we need to replace the aditya with our database name.
3. **Username:** The default username for the mysql database is **root**.
4. **Password:** It is the password given by the user at the time of installing the mysql database. In this example, we are going to use root as the password.

Example:

Id	Name	City
2415	Vahida	KKD
117	Ail	RJY

Example:

impor

t

```

java.s
ql.*;
class
Mysq
lCon
{
    public static void main(String args[])
    {
        try
        {
            Class.forName("com.mysql.jdbc.Driver");
            Connection con=DriverManager.getConnection(
                "jdbc:mysql://localho
st:3306/aditya","root","root");
            //here sai is database name, root is
            username and password Statement
            stmt=con.createStatement();
            ResultSet rs=stmt.executeQuery("select *
            from emp"); while(rs.next())
            System.out.println(rs.getInt(1)+"
            "+rs.getString(2)+" "+rs.get String(3));
            con.close();
        }
        catch(Exception e)
        {
            System.out.println(e);
        }
    }
}
}

```

JDBC BATCH PROCESSING:

- Batch Processing allows you to group related SQL statements into a batch and submit them with one call to the database.
- When you send several SQL statements to the database at once, you reduce the amount of communication overhead, thereby improving performance.

- JDBC drivers are not required to support this feature. You should use the `DatabaseMetaData.supportsBatchUpdates()` method to determine if the target database supports batch update processing. The method returns true if your JDBC driver supports this feature.
- The **`addBatch()`** method of `Statement`, `PreparedStatement`, and `CallableStatement` is used to add individual statements to the batch. The **`executeBatch()`** is used to start the execution of all the statements grouped together.
- The **`executeBatch()`** returns an array of integers, and each element of the array represents the update count for the respective update statement.
- Just as you can add statements to a batch for processing, you can remove them with the **`clearBatch()`** method. This method removes all the statements you added with the `addBatch()` method. However, you cannot selectively choose which statement to remove.

Step to Batching with Statement Object

Here is a typical sequence of steps to use Batch Processing with Statement Object –

- Statement object using either `createStatement()` methods.
- Set auto-commit to false using `setAutoCommit()`.
- Add as many as SQL statements you like into batch using `addBatch()` method on created statement object.
- Execute all the SQL statements using `executeBatch()` method on created statement object.
- Finally, commit all the changes using `commit()` method.

Example:

```
// Create statement object
Statement stmt = conn.createStatement();
// Set
auto-
commi
t to
false
```

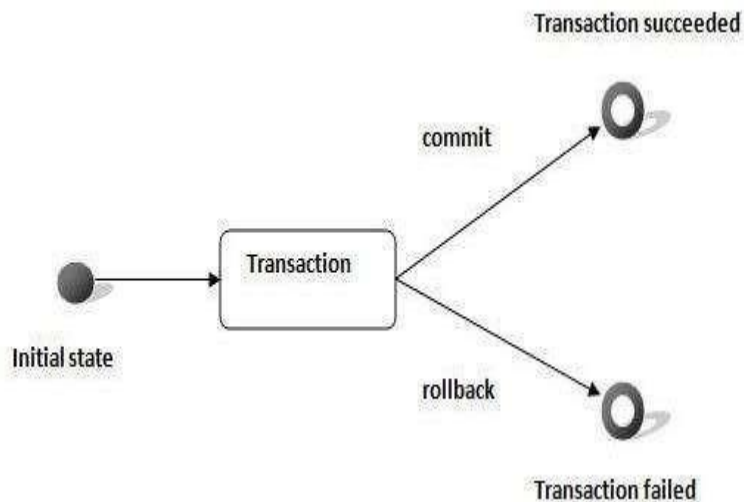
```

conn.setAutoCommit(
    false);
// Create SQL statement
String SQL = "INSERT INTO Employees (id, first, last, age) " + "VALUES(200,'ravi', 'Ali',
30)";
// Add above SQL
statement in the
batch.
stmt.addBatch(SQL);
// Create one more SQL statement
String SQL = "INSERT INTO Employees (id, first, last, age) " + "VALUES(201,'Raj',
'Kumar', 35)";
// Add above SQL
statement in the
batch.
stmt.addBatch(SQL);
// Create one more SQL statement
String SQL = "UPDATE Employees SET age = 35 " + "WHERE id = 100";
// Add above SQL
statement in the
batch.
stmt.addBatch(SQL);
// Create an int[]
to hold returned
values int[]
count =
stmt.executeUpdate();
//Explicitly commit statements to
apply changes conn.commit();

```

Transaction Management in JDBC

- Transaction represents a **single unit of work**.
- properties describes the transaction management well. ACID stands for Atomicity, Consistency, isolation and durability.



In JDBC, **Connection interface** provides methods to manage transaction.

Method	Description
void setAutoCommit(boolean status)	It is true by default means each transaction is committed by default.
void commit()	commits the transaction.
void rollback()	cancels the transaction.

Java FX GUI:

JAVAFX SCENE BUILDER

JavaFX Scene Builder is a visual layout tool that lets you design user interfaces (UIs) for JavaFX applications without writing tedious layout code. Instead of hand-coding every button, text field, and container, you can drag and drop UI components into a canvas.

Behind the scenes, Scene Builder automatically generates **FXML**—an XML-based markup language used to define the structure of your JavaFX UI.

How JavaFX Scene Builder Works

Scene Builder promotes a clean separation of concerns using the **Model-View-Controller (MVC)** pattern:

1. **The View (FXML):** Created entirely using Scene Builder. It defines *what* the UI looks like.
2. **The Controller (Java Code):** A standard Java class where you write the logic (e.g., what happens when a button is clicked).

3. **The Application (Java Code):** The main entry point that loads the FXML file and displays it in a window (Stage).

Key Features of Scene Builder

- **Drag-and-Drop Interface:** Easily place buttons, menus, tables, grids, and layouts onto your canvas.
- **Property Panels:** Modify fonts, colors, sizes, and alignments on the fly without looking at code.
- **CSS Integration:** Apply custom styling using CSS stylesheets to give your app a modern look.
- **Live Preview:** Press `Ctrl + P` (or `Cmd + P` on Mac) to instantly preview how your application will look and behave when running.
- **IDE Integration:** Seamlessly integrates into popular IDEs like IntelliJ IDEA, Eclipse, and NetBeans.

Here is how you bridge Scene Builder with your Java code.

1. Design the UI in Scene Builder

You drag a `Button` onto the canvas. In the **Code** section of the properties panel, you give it two important identifiers:

- **fx:id:** `myButton` (This acts as the variable name in Java).
- **On Action:** `handleButtonClick` (This is the method name that will run when clicked).

Save this file as `scene.fxml`.

2. Create the Controller Class

In your Java project, you create a controller class to handle the button click. You use the `@FXML` annotation to link the Java variables and methods to the FXML file.

```
import javafx.fxml.FXML;
import javafx.scene.control.Button;

public class MyController {

    @FXML
    private Button myButton; // Matches fx:id="myButton"

    @FXML
    private void handleButtonClick() { // Matches On Action="handleButtonClick"
        myButton.setText("You clicked me!");
    }
}
```

3. Load the FXML in Main Application

Finally, your main Java application loads the FXML file to display it to the user.

```

import javafx.application.Application;
import javafx.fxml.FXMLLoader;
import javafx.scene.Parent;
import javafx.scene.Scene;
import javafx.stage.Stage;

public class MainApp extends Application {

    @Override
    public void start(Stage primaryStage) throws Exception {
        // Load the FXML file generated by Scene Builder
        Parent root = FXMLLoader.load(getClass().getResource("scene.fxml"));

        primaryStage.setTitle("My JavaFX App");
        primaryStage.setScene(new Scene(root, 400, 300));
        primaryStage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }
}

```

3. Load the FXML in Main Application

Finally, your main Java application loads the FXML file to display it to the user.

```

import javafx.application.Application;
import javafx.fxml.FXMLLoader;
import javafx.scene.Parent;
import javafx.scene.Scene;
import javafx.stage.Stage;

public class MainApp extends Application {

    @Override
    public void start(Stage primaryStage) throws Exception {
        // Load the FXML file generated by Scene Builder
        Parent root = FXMLLoader.load(getClass().getResource("scene.fxml"));

        primaryStage.setTitle("My JavaFX App");
        primaryStage.setScene(new Scene(root, 400, 300));
        primaryStage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }
}

```

JAVA FX APP WINDOW STRUCTURE

JavaFX application window is structured, it helps to use a **theater analogy**.

A JavaFX application mimics a real-world theater production, consisting of a **Stage**, a **Scene**, and a **Scene Graph** (containing nodes/actors).

The Hierarchical Structure

Here is how the hierarchy breaks down from the outermost window to the individual UI components:

Component	Theater Analogy	What it represents in JavaFX
Stage	The actual physical theater stage.	The native OS window (with the minimize, maximize, and close buttons).
Scene	The specific set/backdrop currently on stage.	The container for all physical content. It defines the background color, dimensions, and styling.
Scene Graph	The actors, props, and lighting arranged on the set.	A tree-like data structure that holds all the UI elements (Layouts, Buttons, Text, Images).

Understanding the Components

1. The Stage (`javafx.stage.Stage`)

The Stage is the top-level container.

- Your application receives a `primaryStage` automatically from the JavaFX runtime when the app starts.
- You can create additional stages if your app needs multiple windows (like a popup or settings dialog).
- **Key methods:** `stage.setTitle("My App")`, `stage.setScene(scene)`, `stage.show()`.

2. The Scene (`javafx.scene.Scene`)

The Scene holds the actual content.

- A Stage can only display **one Scene at a time**, but you can easily swap scenes to change what the user sees (e.g., swapping from a "Login Scene" to a "Dashboard Scene").
- You must pass a **Root Layout** to a Scene when you create it.

3. The Scene Graph and Nodes (javafx.scene.Node)

Everything inside the Scene is a **Node**. The Scene Graph is a strict tree structure:

- **Root Node:** The base layout container that holds everything else (e.g., a VBox, BorderPane, or AnchorPane).
- **Branch Nodes:** Parent nodes that can have "children" nodes. These are typically layout containers used to nest other elements.
- **Leaf Nodes:** Elements that cannot have children. These are your actual interactive UI controls (e.g., Button, TextField, Label, ImageView).

```
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.scene.control.Label;
import javafx.scene.layout.VBox;
import javafx.stage.Stage;
```

```
public class WindowStructureApp extends Application {

    @Override
    public void start(Stage primaryStage) {
        // 1. Create Leaf Nodes (UI Controls)
        Label messageLabel = new Label("Welcome to JavaFX!");
        Button actionButton = new Button("Click Me");

        // 2. Create a Branch/Root Node (Layout Container) and add leaves to it
        VBox rootLayout = new VBox(10); // Spacing of 10px between elements
        rootLayout.getChildren().addAll(messageLabel, actionButton);

        // 3. Create the Scene and attach the Root Node to it
        Scene mainScene = new Scene(rootLayout, 400, 300); // Width: 400px, Height: 300px

        // 4. Configure the Stage and place the Scene onto it
        primaryStage.setTitle("JavaFX Architecture Demo");
        primaryStage.setScene(mainScene);

        // 5. Open the window
        primaryStage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```

Common Layout Panes (The Root/Branch Nodes)

Because the structural hierarchy relies heavily on how you arrange nodes, JavaFX provides various `Pane` classes to manage layouts automatically:

- **BorderPane:** Divides the window into five regions: Top, Bottom, Left, Right, and Center. Great for standard desktop app structures.
- **VBox / HBox:** Arranges its children in a single vertical column or horizontal row.
- **GridPane:** Arranges elements in a flexible grid of rows and columns.
- **AnchorPane:** Allows you to anchor UI elements to the edges of the window, making it easy to create resizable layouts.

DISPLAYING TEXT AND IMAGE

To display text and images in JavaFX, you primarily use two leaf nodes: Label (or Text) and ImageView.

Because a JavaFX Label is highly versatile, it can actually display both text and an image simultaneously, making it perfect for buttons, icons, or captioned graphics.

- **Image (javafx.scene.image.Image):** This class handles loading the actual image file from a local path or a URL. It runs in the background and holds the raw pixel data.
- **ImageView (javafx.scene.image.ImageView):** This is the visual UI node. Think of the Image as the photograph and the ImageView as the picture frame. You can resize, rotate, and scale the frame without altering the original file.
- **Label (javafx.scene.control.Label):** The standard control for displaying non-editable text. It includes a `.setGraphic(Node)` method, allowing you to attach an ImageView directly to it.

```
import javafx.application.Application;
import javafx.geometry.Pos;
import javafx.scene.Scene;
import javafx.scene.control.Label;
import javafx.scene.image.Image;
import javafx.scene.image.ImageView;
import javafx.scene.layout.VBox;
import javafx.scene.text.Font;
import javafx.scene.text.FontWeight;
import javafx.scene.text.Text;
import javafx.stage.Stage;
```

```
public class MediaDisplayApp extends Application {
```

```
    @Override
```

```
    public void start(Stage primaryStage) {
```

```
        // 1. Displaying Text using the Text class (great for styled paragraphs)
```

```
        Text titleText = new Text("Welcome to JavaFX Media Display");
```

```
        titleText.setFont(Font.font("Arial", FontWeight.BOLD, 20));
```

```
        // 2. Loading and Displaying a Standalone Image
```

```
        // Replace the URL with your own local path (e.g., "file:images/photo.png") or web link
```

```
        Image image = new Image("https://placeholder.pics/svg/150", true);
```

```
        ImageView imageView = new ImageView(image);
```

```

// Customizing the image frame
imageView.setFitWidth(150); // Set target width
imageView.setPreserveRatio(true); // Maintain aspect ratio
imageView.setSmooth(true); // Use higher quality filtering

// 3. Displaying Text AND Image together using a Label
Label explicitLabel = new Label("This is a caption below an icon");

Image iconImage = new Image("https://placeholder.pics/svg/30", true);
ImageView iconView = new ImageView(iconImage);

explicitLabel.setGraphic(iconView); // Attach the image to the label
explicitLabel.setGraphicTextGap(10); // Pixel gap between image and text
// explicitLabel.setContentDisplay(ContentDisplay.TOP); // Optional: Put image
ABOVE text

// 4. Arrange everything in a Layout Container
VBox rootLayout = new VBox(20); // 20px vertical spacing between elements
rootLayout.setAlignment(Pos.CENTER); // Center elements horizontally and vertically
rootLayout.getChildren().addAll(titleText, imageView, explicitLabel);

// 5. Setting up the Stage and Scene
Scene scene = new Scene(rootLayout, 450, 400);
primaryStage.setTitle("Displaying Text and Images");
primaryStage.setScene(scene);
primaryStage.show();
}

public static void main(String[] args) {
    launch(args);
}
}

```

EVENT HANDLING

JavaFX, **Event Handling** is the mechanism that allows your application to respond to user interactions, such as mouse clicks, key presses, window resizing, or touchscreen gestures.

When a user interacts with the UI, an **Event** object is created. JavaFX uses a delegation model where the event is captured, routed through the scene graph, and delivered to an **Event Handler** (a listener) that executes your custom Java code.

Every event-driven action in JavaFX relies on three pieces:

1. **Event Source:** The UI component that the user interacted with (e.g., a `Button`, `TextField`, or the `Scene` itself).
2. **Event Object:** The object containing details about the interaction (e.g., `MouseEvent` tells you the coordinates of a click; `KeyEvent` tells you which keyboard key was pressed).
3. **Event Handler / Listener:** The functional block of code that waits for the event and decides how to respond.

Four Ways to Implement Event Handling:

Depending on your coding style and whether you use Scene Builder, you can implement event handlers in a few different ways.

1. The Modern Way: Lambda Expressions (Recommended)

Because `EventHandler` is a functional interface, you can write clean, concise code using Java lambdas.

```
Button btn = new Button("Click Me");

// Using a Lambda expression to handle the action event
btn.setOnAction(event -> {
    System.out.println("Button clicked!");
});
```

2.The FXML Way (Using Scene Builder)

If you design your UI in Scene Builder, you can link a button directly to a method in your Controller class.

In Scene Builder, select your button, go to the **Code** panel, and set **On Action** to `#handleBtnClick`. Then write the matching method in your controller:

```
public class MyController {

    @FXML
    private void handleBtnClick(ActionEvent event) {
        System.out.println("Button clicked via FXML!");
    }
}
```

3. Handling Specific Events (Mouse, Keyboard, etc.)

For events other than standard button clicks, you use specialized registration methods like `.setOnMouseEntered()` or `.setOnKeyPressed()`.

```
Label hoverLabel = new Label("Hover over me!");
```

```
// Triggered when the mouse enters the label area
hoverLabel.setOnMouseEntered(event -> {
    hoverLabel.setStyle("-fx-text-fill: red;");
});
```

```
// Triggered when the mouse leaves the label area
hoverLabel.setOnMouseExited(event -> {
    hoverLabel.setStyle("-fx-text-fill: black;");
});
```

4. The Generic Way: `addEventHandler()`

If you need to listen for a broad category of events or attach multiple handlers to the same node, you can use the generic method:

```
// Attaching a generic handler for any type of Mouse Click
btn.addEventHandler(MouseEvent.MOUSE_CLICKED, event -> {
    System.out.println("Mouse clicked at coordinates: " + event.getX() + ", " + event.getY());
});
```

LAYING OUT NODES IN SCENE GRAPH

In JavaFX, you never position UI elements (nodes) by manually typing coordinate points like `$X = 100$` and `$Y = 200$`. If you do, your user interface will break, overlap, or look distorted the moment a user resizes the window.

Instead, JavaFX uses **Layout Panes** (containers) to manage the positions and sizes of your nodes automatically based on rules you define.

The Layout Process in the Scene Graph

When a scene graph is rendered, the layout process moves through the tree in a parent-to-child pattern:

1. **The Parent Queries:** The layout pane asks its child nodes for their sizing preferences:
 - **Minimum size:** The smallest size the node needs to avoid being unusable.
 - **Preferred size:** The ideal size the node wants to look best.
 - **Maximum size:** The largest size the node is allowed to scale up to.
2. **The Parent Arranges:** The layout pane calculates its own available space and arranges the children according to its specific layout rules (e.g., in a row, a grid, or anchored to edges).

Core JavaFX Layout Panes

Choosing the right layout pane depends entirely on how you want your UI to adapt. Here are the most commonly used panes:

1. **HBox and VBox (Linear Layouts)**

These arrange components in a single line. `HBox` places them horizontally in a row, and `VBox` stacks them vertically in a column.

```
VBox verticalLayout = new VBox(10); // 10px spacing between nodes
verticalLayout.setAlignment(Pos.CENTER); // Center elements
verticalLayout.getChildren().addAll(new Button("Top"), new Button("Bottom"));
```

2. BorderPane (Window Framework)

Perfect for classic desktop application layouts. It divides the window into five distinct zones: **Top**,

Bottom, Left, Right, and Center.

```
BorderPane layout = new BorderPane();
layout.setTop(new MenuBar());
layout.setLeft(new VBox(new Label("Sidebar Options")));
layout.setCenter(new TextArea()); // Center scales dynamically to fill remaining space
```

3. GridPane (Form & Tabular Layouts)

Arranges components in a flexible grid of rows and columns. It is ideal for data-entry forms or calculators.

```
GridPane grid = new GridPane();
grid.setHgap(10); // Horizontal gap
grid.setVgap(10); // Vertical gap

// Arguments: Node, Column Index, Row Index
grid.add(new Label("Username:"), 0, 0);
grid.add(new TextField(), 1, 0);
grid.add(new Label("Password:"), 0, 1);
grid.add(new PasswordField(), 1, 1);
```

4. AnchorPane (Responsive Scaling)

Allows you to anchor child elements to the edges of the window (Top, Bottom, Left, or Right). When the window stretches, the distance between the node and its anchored edge stays exactly the same.

```
AnchorPane anchorPane = new AnchorPane();
Button saveButton = new Button("Save");

// Secure the button to the bottom-right corner of the window
AnchorPane.setBottomAnchor(saveButton, 15.0);
AnchorPane.setRightAnchor(saveButton, 15.0);

anchorPane.getChildren().add(saveButton);
```

MOUSE EVENTS

In JavaFX, **Mouse Events** track any physical interaction a user has with a mouse, trackpad, or touchscreen relative to the UI nodes. Whether it's clicking a button, dragging a slider, or tracking cursor movement, JavaFX packages these inputs into a **MouseEvent** object.

You can listen for several distinct variations of mouse movements and actions:

- `MOUSE_CLICKED (setOnMouseClicked)`: Fires when a mouse button is pressed and released quickly over the exact same node.
- `MOUSE_PRESSED (setOnMousePressed)`: Fires the exact millisecond a mouse button goes down.
- `MOUSE_RELEASED (setOnMouseReleased)`: Fires when the mouse button is let go.
- `MOUSE_ENTERED (setOnMouseEntered)`: Fires once when the cursor moves inside the boundary of a node. Great for creating hover effects.
- `MOUSE_EXITED (setOnMouseExited)`: Fires once when the cursor leaves a node's boundary.
- `MOUSE_MOVED (setOnMouseMoved)`: Fires continuously as long as the mouse is moving *inside* a node.
- `MOUSE_DRAGGED (setOnMouseDragged)`: Fires continuously when the mouse is moved *while holding down* a mouse button.

Extracting Information From a MouseEvent

When a mouse event triggers your code, the `MouseEvent` object carries crucial data about *how* and *where* the action happened. Here are the most useful methods:

1. Tracking Coordinates

You can fetch where the click occurred using three different coordinate spaces:

- `getX()` / `getY()`: Coordinates relative to the **Node** that was clicked (e.g., `$(0,0)$` is the top-left corner of the button).
- `getSceneX()` / `getSceneY()`: Coordinates relative to the entire **Application Window**.
- `getScreenX()` / `getScreenY()`: Coordinates relative to your physical **Monitor screen**.

2. Identifying the Mouse Button

You can check whether the user used a left-click, right-click, or middle-wheel click:

- `event.getButton() == MouseButton.PRIMARY` (Left Click)
- `event.getButton() == MouseButton.SECONDARY` (Right Click / Context Menu)
- `event.getButton() == MouseButton.MIDDLE` (Scroll Wheel Click)

3. Detecting Click Counts (Double Clicks)

- `event.getClickCount() == 2` allows you to easily isolate a double-click event (like opening a file).

```
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.layout.Pane;
import javafx.scene.paint.Color;
import javafx.scene.shape.Circle;
import javafx.stage.Stage;
```

```
public class MouseEventDemo extends Application {
```

```

@Override
public void start(Stage primaryStage) {
    Pane drawingCanvas = new Pane();
    drawingCanvas.setStyle("-fx-background-color: #f4f4f4;");

    // 1. MOUSE_CLICKED: Detect regular or double clicks
    drawingCanvas.setOnMouseClicked(event -> {
        if (event.getClickCount() == 2) {
            System.out.println("Double-clicked the canvas at: " + event.getX() + ", " +
event.getY());
        } else if (event.getButton() == MouseButton.SECONDARY) {
            // Right-click clears the canvas
            drawingCanvas.getChildren().clear();
        }
    });

    // 2. MOUSE_DRAGGED: Draw circles continuously as the user drags
    drawingCanvas.setOnMouseDragged(event -> {
        // Only draw if using the primary (left) mouse button
        if (event.getButton() == MouseButton.PRIMARY) {
            Circle brushStroke = new Circle(event.getX(), event.getY(), 5);
            brushStroke.setFill(Color.BLUEVIOLET);

            drawingCanvas.getChildren().add(brushStroke);
        }
    });

    // 3. MOUSE_ENTERED & EXITED: Visual feedback for window focus
    drawingCanvas.setOnMouseEntered(event -> {
        drawingCanvas.setStyle("-fx-background-color: #ffffff;"); // Brighten up
    });

    drawingCanvas.setOnMouseExited(event -> {
        drawingCanvas.setStyle("-fx-background-color: #eaddff;"); // Tint when cursor leaves
    });

    Scene scene = new Scene(drawingCanvas, 600, 400);
    primaryStage.setTitle("Mouse Event Workshop");
    primaryStage.setScene(scene);
    primaryStage.show();
}

public static void main(String[] args) {
    launch(args);
}
}

```

equals() and ==:

equals() and == performs two different operations. equals() method compares the characters inside a String object.

The == operator compares two object references to see whether they refer to the same instance.

StringBuilder class:

StringBuilder class has been added in JDK1.5 which has same features like StringBuffer class. StringBuilder class objects are also mutable as the stringBuffer Objects.

Difference:

StringBuffer is class is synchronized and StringBuilder is not.